Cricket Shot Recognition: Comparing Gradient Features and a Frozen CNN

Hrushikesh Roop Avvari (230249) Voora Rakesh (231174)
Gokavarapu Manikanta Sri Akshith (230418)
Lokesh Kumar Moond (230606) Sharath (230303)
Indian Institute of Technology Kanpur
EE604, October 2026

Abstract

We study a small, practical task: classify one cricket image as drive, leg glance/flick, pull or sweep. We compare a hand-crafted histogram of oriented gradients (HOG) with frozen MobileNetV2 features followed by a linear classifier. Our implemented extension combines global and fixed regional CNN features with their absolute difference, and exposes disagreement between separate view classifiers. On an unchanged test set of 945 images in 70 visual groups, HOG obtains 35.3% accuracy and 34.5% macro F1; the global CNN obtains 63.0% and 62.9%. Contrast obtains 65.5% and 64.9%, only three additional correct predictions over concatenation. Group-based uncertainty does not establish a clear advantage. A duplicate audit substantially reduces the optimistic preliminary result, and temperature scaling worsens test calibration. We report these limitations alongside a local demo and reproducible predictions. The study is exploratory because grouping was revised after earlier evaluation.

1. Introduction

A collection of labeled cricket images can help retrieve examples of batting shots. Our task takes one image containing a shot and returns one of four supplied labels. It assumes that someone has already chosen the image. It does not locate the batter, identify a contact frame or grade batting technique.

Visual differences can be subtle. The bat may be blurred, a frame may show preparation rather than contact, and camera angle or background may dominate. We ask two focused questions. How does a gradient descriptor compare with pretrained CNN features on the same images? Does comparing a global view with a fixed central region provide evidence beyond simply concatenating them?

Our foundation is feature extraction followed by classification. HOG supplies an explicit descriptor; a frozen pretrained network supplies a learned one. The course-level extension is a two-view absolute-difference feature and a visible agreement diagnostic. We implement and ablate this design, but do not claim its components or combination are unprecedented. The small gain leaves algorithmic novelty a weakness. Auditing modified copies is useful evaluation work, rather than a new recognition algorithm.

2. Related work and positioning

HOG accumulates gradient orientations into normalized spatial blocks [3]. Our documented variant uses this idea for whole-image classification; it does not reproduce the original pedestrian detector. MobileNetV2 is an established efficient CNN [9]. We select public ImageNet ONNX weights [8] and freeze them. This follows the pretrained-feature-extractor plus classifier approach in Lecture 15; the architecture itself need not have been taught by name.

Cricket recognition already has datasets and model studies, including KUCricShot [6] and Kang's preprint [7]. We did not import those models, and their scores use different data and protocols. They are prior work, not numerical baselines for this experiment. Temperature scaling [5] and selective classification [4] are established. Our threshold heuristic provides no guaranteed risk bound.

3. Data and evaluation

We use version 2 of Aneesh's *Cricket Shots Dataset* on Kaggle [2]. Its 4,724 files have four upstream labels. Drive and leg glance/flick each combine related stroke types. We retain these labels and do not claim original collection or annotation. The downloaded metadata reports an unknown license. Provenance and checksums are recorded; source-control defaults exclude raw images and examples.

Decoded RGB hashes remove one excess exact copy. A visual audit connects qualifying near copies, including rotated and mirrored versions, then keeps each connected

Label	Train	Val.	Test
Drive	734	244	245
Glance/flick	672	224	224
Pull	756	252	252
Sweep	672	224	224
Total	2834	944	945

Table 1. Retained image counts; all visual groups remain in one split.

group in one split. It leaves 4,723 representatives in 363 visual groups. The initial hash-only contrast accuracy was 96.3%. Visual review found missed transformed pictures, so grouping was strengthened and the comparison rerun. The corrected result is 65.5%. Both predictions and both grouping records remain available. This history makes the final study exploratory; the new HOG comparison uses the corrected split unchanged.

Training has 2,834 images in 221 groups, validation 944 in 72 groups, and test 945 in 70 groups. These are fixed stratified group folds with seed 604: fold 0 is test, fold 1 validation, and the others training. This is one holdout, not five-fold cross-validation. The source has no verified match or player identities. Visual groups are not necessarily independent photographs or matches. Appendix A.1 gives detailed audit criteria.

4. Proposed method

4.1. Common geometry

We apply EXIF orientation, convert to RGB and bound the image to a 1,024-pixel box. The CNN uses the image extent and a central crop with $x \in [0.15W, 0.85W]$, $y \in [0.05H, 0.95H]$. Each view is fitted to 224 by 224 while preserving aspect ratio and cropping the center. Even the global branch loses peripheral pixels at this step. The crop can miss an off-center batter.

4.2. HOG baseline

HOG uses the identical global geometry, converted to 8-bit luminance and scaled to $[0, 1]$. Central differences give g_x, g_y, magnitude $m = \sqrt{g_x^2 + g_y^2}$ and unsigned angle $\theta = \operatorname{atan2}(g_y, g_x) \bmod 180°$. Boundary derivatives are zero. Each 16 by 16 cell accumulates *magnitudes*, not pixel counts, into nine bins centered at $0, 20, \ldots, 160°$. A pixel splits its vote linearly between adjacent orientation bins, wrapping at $180°$. Votes do not interpolate across cell boundaries.

A 2 by 2 cell block has 36 values and moves one cell per step. Each block is divided by $\sqrt{\sum_i h_i^2 + 10^{-10}}$. The 14 by 14 cells yield 13 by 13 blocks: $13 \cdot 13 \cdot 36 = 6,084$ concatenated values. Final per-image L2 normalization gives the classifier input. There is no learned scaling, PCA or feature selection. These fixed settings are not claimed to be optimal HOG settings. The lecture's signed-angle example and our unsigned bins are distinct variants of the same principle.

4.3. Frozen CNN and contrast

The CNN uses ImageNet RGB mean $(0.485, 0.456, 0.406)$ and standard deviation $(0.229, 0.224, 0.225)$. We remove the ImageNet classifier and retain the pooled 1,280-value output. No CNN parameters are updated on cricket images. For separately normalized global and regional vectors u, v, compare u, v, $\mathrm{Norm}([u; v])$, and

$$z = \mathrm{Norm}([u; v; |u - v|]) \in \mathbb{R}^{3840}. \tag{1}$$

Absolute difference gives a linear classifier a nonlinear relationship between the views. A signed difference would already be expressible by a linear map of concatenation. This motivates the construction without proving it improves recognition. Coordinates have no claimed physical interpretation, and the crop vector is not a local keypoint descriptor.

All five representations use class-balanced, L2-regularized multinomial logistic regression. Logits are $a = Wz + b$ and scores are $p_c = \exp(a_c/T)/\sum_j \exp(a_j/T)$. Training minimizes weighted multiclass cross-entropy with regularization. Contrast stores a 4×3840 matrix and four biases: 15,364 parameters. These cricket parameters are learned; the CNN and HOG extraction rules stay fixed.

5. Experiments

5.1. Controlled comparison

Every head uses the corrected manifest and $C \in \{0.1, 1, 10, 100\}$. Validation macro F1 selects C, with smaller values winning ties. HOG's configuration and selection are saved before test scoring. No new split, seed, feature setting or threshold was searched for a better test result. We retain the original CNN heads and predictions. HOG is an exploratory extension after prior results on this archive were known.

The original validation set also selected CNN temperatures and the review threshold. This reuse can overfit validation. We disclose it and preserve the choices; separate calibration data is future work. HOG uses $T = 1$ and does not alter the review gate.

HOG selects $C = 0.1$ and reaches 49.4% validation macro F1, falling to 34.5% on test. Its 35.3% accuracy is below all four CNN configurations. This supports pretrained features *for this configuration and split*. It does not isolate color, pretraining or architecture effects, or establish universal HOG inferiority.

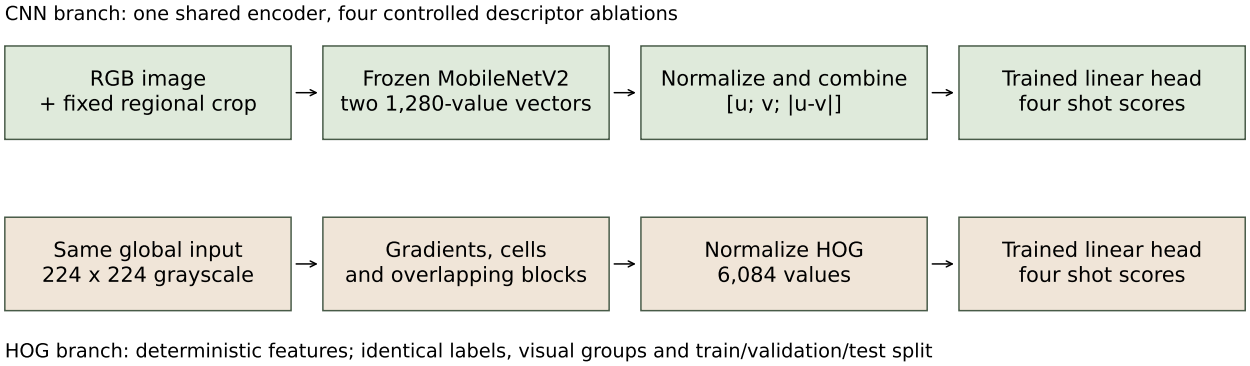


Figure 1. Two feature approaches on the same split. Only the linear cricket classifiers learn from training images. CNN ablations share a frozen encoder. The fixed crop is a regional view, not a detected or segmented batter.

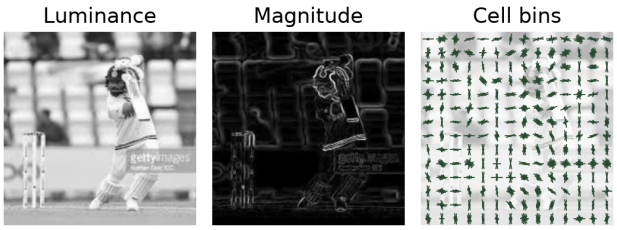


Figure 2. A selected test image traced through luminance, gradient magnitude and magnitude-weighted cell bins. This explains HOG; it is not a localization result or a representative performance sample.

Feature	Dim.	C	Acc.	F1
HOG	6084	0.1	35.3	34.5
CNN global	1280	10	63.0	62.9
CNN crop	1280	10	62.4	61.7
Concat.	2560	10	65.2	64.6
Contrast	3840	10	65.5	64.9

Table 2. All-test accuracy and macro F1 in percent. Same images, labels, groups, split and regularization grid; HOG has no ImageNet pretraining.

Contrast gets 619/945 correct, versus 616/945 for concatenation: only +0.32 accuracy points. A paired group-bootstrap interval is -1.46 to +2.38 points, spanning zero. Concatenation has the highest validation macro F1. We retain the pre-existing contrast deployment, but do not present a substantial or reliable advantage.

Intervals resample the 70 visual groups with replacement in 2,000 draws, seed 17, retaining each drawn group's images. Identical draws give paired comparisons. Statistics remain image-weighted within each resample. These conditional percentile intervals omit training and split variability and cannot repair unknown match correlation or missed copies. They do not treat 945 images as independent original photographs.

Feature	Acc. 95%	F1 95%
HOG	25.9–48.1	24.9–44.6
CNN global	49.5–74.2	44.6–72.0
CNN crop	45.3–76.7	42.1–72.3
Concat.	48.9–77.9	43.7–74.9
Contrast	49.9–77.6	45.8–74.2

Table 3. Conditional visual-group bootstrap intervals, in percent, from saved individual predictions.

5.2. Errors and model scores

Glance/flick recall is 45.5% on 224 examples, while pull recall is 78.2% on 252. HOG predicts pull for 110 of 224 sweep images. Similar stances, fixed spatial cells and camera layout are plausible limitations, not verified causes for every error.

Temperature scaling retains the top label but worsens test ECE from 0.043 to 0.094 and NLL from 0.859 to 0.898 (ten equal-width ECE bins). It did not demonstrate improved calibration. A softmax score is relative to the supported labels, not a guarantee that an arbitrary image is correct or belongs to these classes.

The unchanged secondary review rule requires score at least 0.70 and agreement of global, crop and contrast heads. It accepts 557/945 images, with 433 correct: 58.9% coverage and 77.7% accepted accuracy. The 90% validation target is not a test result or guarantee. Correlated heads can agree on a wrong label. Appendix A.2 retains the detailed diagnostics.

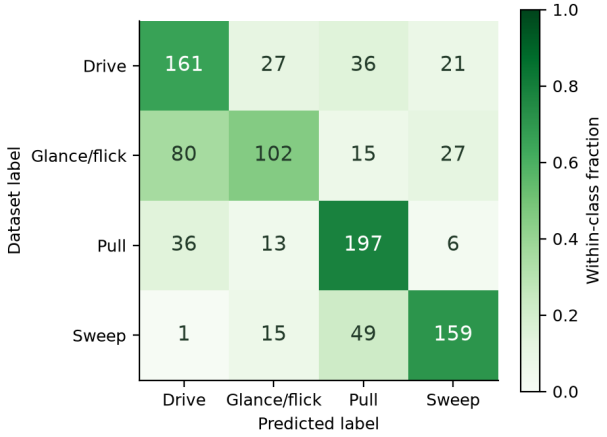


Figure 3. Contrast confusion counts on all test images. Rows are dataset labels; color is within-class fraction. Review does not remove errors from this matrix.

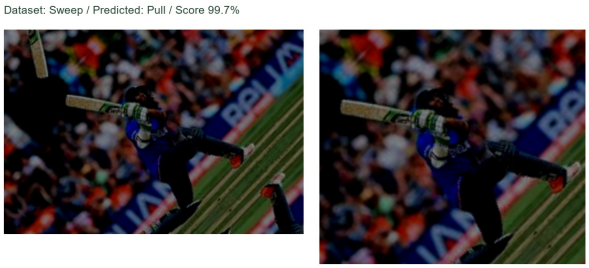


Figure 4. Dataset label: sweep; prediction: pull (99.7% score). This error was selected after evaluation and retained in the demo, alongside successes and disagreement.

5.3. Implementation and demonstration

Training uses scikit-learn; inference uses CPU ONNX Runtime and numeric NPZ/JSON heads. Shared functions implement training and demo preprocessing. The image page shows both actual inputs, four scores, review reasons and a HOG comparison. Uploads use temporary storage; inference needs no cloud service or API key after installation. JSON export identifies temperature-scaled softmax scores. Selected examples include a correct result, a confident error and disagreement.

Warm CNN plus HOG inference takes 23 ms median and 52 ms at the 95th percentile over 50 fixed test images on the recorded host. This includes separate image decodes, but excludes initialization, HTTP transport and preview encoding. Preserved older CNN-only timings describe a different operation.

6. Limitations and conclusion

The project implements the course's feature-extractor/classifier structure and tests a small two-view extension. HOG gives an explicit gradient baseline; pretrained features work better on this fixed benchmark. Contrast adds only three correct test predictions over concatenation, with uncertainty spanning no gain. Novelty remains modest; we make no state-of-the-art or first-in-literature claim.

The duplicate-audit effect is an evaluation warning, not proof of independent groups. Unknown match identities, unverified upstream labels, validation reuse and exploratory development limit generalization claims. An independently labeled test collection with known match/player identities would answer a more useful question than searching this split for a higher score. The weak video study remains in Appendix A.3.

References

[1] amittalmale. CRICKET_SHOTS, version 1. Kaggle dataset, 2025. Accessed 17 October 2026.

[2] Aneesh. Cricket shots dataset, version 2. Kaggle dataset, 2022. Accessed 17 October 2026; uploader identifier aneesh10.

[3] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *CVPR*, pages 886–893, 2005.

[4] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *NeurIPS*, 2017.

[5] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML*, pages 1321–1330, 2017.

[6] Sk. Wasiqul Hoque, Md Ahsan Habib, Atanu Shome, and Sadman Sakib. KUCricShot: A dataset for developing detection model of cricket shots. In *ICCIT*, 2023. Dataset description checked against the authors' repository.

[7] Sungwoo Kang. Modern deep learning approaches for cricket shot classification: A comprehensive baseline study. arXiv:2510.09187, preprint, 2025.

[8] ONNX Model Zoo contributors. MobileNetV2 pretrained onnx models. Accessed 17 October 2026.

[9] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *CVPR*, pages 4510–4520, 2018.

[10] Limin Wang, Yuanjun Xiong, Zhe Wang, Yu Qiao, Dahua Lin, Xiaoou Tang, and Luc Van Gool. Temporal segment networks: Towards good practices for deep action recognition. In *ECCV*, 2016.

A. Supplementary evidence

A.1. Visual-copy audit

The initial audit uses decoded RGB hashes, 64-bit perceptual hash Hamming distance at most six, and 16 by 16 RGB thumbnail mean absolute difference at most 18. Exact copies collapse; conflicting-label components are quarantined. The stronger audit adds quarter-turn/mirror hashes (same bit cutoff, aligned thumbnail difference at most 24), then proposes the nearest 15 frozen-feature neighbors within cosine distance 0.20.

ORB binary descriptors and a RANSAC homography verify proposed geometric links. Requirements are a 0.72 match-ratio cutoff, at least 16 filtered matches, 12 inliers, 65% inlier fraction and 8% convex-hull coverage in both images. This adds 948 transform and 1,797 geometric edges. There are no final conflicting components. All linked copies and initial hash components stay in one split. These checks reduce leakage but can still miss related photographs or join visually similar ones. Full criteria and preserved edges are in the commented audit script and data directory.

A.2. Secondary diagnostics

Scores	NLL	ECE	Brier
Unscaled	0.859	0.043	0.470
Scaled	0.898	0.094	0.482

Table 4. Unchanged contrast scores on test; lower is better. Scaling worsens NLL and ECE.

The original temperature is approximately 0.802. Validation is reused for regularization, temperature and review selection. The review search chooses the lowest threshold from 0.50 to 0.95 in steps of 0.05 attaining 90% validation accuracy with at least 20 accepted images and 20% coverage, with 0.80 fallback. Its chosen 0.70 threshold stays unchanged. At that threshold confidence alone covers 62.2% with 77.6% accepted accuracy; adding agreement covers 58.9% with 77.7%. This modest difference is descriptive, not a guaranteed improvement.

Head	Clean	Dark	Blur	Mirror
CNN global	63.0	62.8	48.0	62.9
Contrast	65.5	67.1	49.7	65.2

Table 5. Preserved descriptive stress-test accuracy in percent; no retraining or parameter choice.

Dark multiplies source brightness by 0.6; blur uses Gaussian radius two at source resolution; mirror reflects horizontally while retaining the four broad labels. Both CNN views are recomputed. These conditions do not represent a new match or camera, and blur severity varies with source resolution. Original predictions, stress evidence and CNN-only timings remain under `results/images/`; preliminary hash-only evidence remains under `results/images_initial_hash/`.

A.3. Exploratory video pilot

The separate Kaggle CRICKET_SHOTS archive [1] contains 562 clips in eight labels and 25 normalized source-title groups. Its test set has 116 clips from five groups. Nine uniformly sampled frames feed the frozen encoder. Three ordered phase means, late-minus-early difference and temporal standard deviation form 6,400 features. This sparse temporal summary is motivated by segment-based representations [10], without reproducing TSN training.

The full-frame pilot obtains 18.1% accuracy. A fixed upper-center crop, introduced after pilot test feedback, reaches 30.2% accuracy and 21.3% macro F1. The training-majority baseline also reaches 30.2% accuracy (5.8% macro F1). Equal majority accuracy is not a successful general video recognizer. Defence has only three test clips; validation has one unorthodox clip. The reused holdout makes this exploratory. Original and revised predictions remain available; different data and labels prevent a direct image-versus-video comparison.

A.4. Code, data and repositories

The complete report appends every current custom executable source and configuration, with a SHA-256 inventory. Public libraries, pretrained weights and the unmodified CVPR style are not original code. The project includes dependency files, manifests, numeric heads, saved predictions, tests and reproduction commands. The separate course mapping and viva guide explain lecture references, advanced audit tools and one measured image trace.

Project repository: `https://github.com/hrushikeshroop/cricket-shot-recognition-ee604`.

Data: Cricket Shots Dataset v2 and CRICKET_SHOTS v1. Pretrained weights: ONNX Model Zoo. Libraries: scikit-learn, ONNX Runtime, OpenCV, Flask, and NumPy. Complete dependency links and attribution accompany `docs/sources.md`. Our NumPy HOG follows the cited method and supplied lectures; no external cricket classifier code was imported.

Original code and configuration. Line numbers identify source lines. Wrapped lines continue without a number.

app.py

```
"""Local-only Flask demo. No uploads or predictions leave this computer."""

from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parent
sys.path.insert(0, str(ROOT / "src"))
import base64
import csv
import io
import json
import tempfile
import threading
import numpy as np
from PIL import Image, ImageOps
from flask import Flask, jsonify, render_template, request, send_file
from werkzeug.exceptions import RequestEntityTooLarge
from cricketshot.core import Recognizer, DISPLAY, crop_frame
from cricketshot.images import ImageRecognizer
from cricketshot.hog import HOGRecognizer


def create_app(recognizer=None):
    app = Flask(__name__)
    app.config["MAX_CONTENT_LENGTH"] = 40 * 1024 * 1024
    inference_lock = threading.Lock()
    model_holder = [recognizer]
    image_model_holder = [None]
    hog_model_holder = [None]

    def model():
        if model_holder[0] is None:
            model_holder[0] = Recognizer()
        return model_holder[0]

    def examples():
        path = ROOT / "demo/examples.json"
        entries = json.loads(path.read_text(encoding="utf-8")) if path.exists() else []
        return [e for e in entries if (ROOT / e["path"]).is_file() and (ROOT / e["thumbnail"]).is_file()]

    def infer(path):
        with inference_lock:
            prediction, frames = model().predict(path)
        previews = []
        for frame in frames:
            im = ImageOps.fit(
                crop_frame(frame, model().config.get("view", "batter_roi")),
                (224, 224),
                method=Image.Resampling.BILINEAR,
            )
            im.thumbnail((240, 180))
            buffer = io.BytesIO()
            im.save(buffer, format="JPEG", quality=85)
            previews.append(
                ""
                + base64.b64encode(buffer.getvalue()).decode("ascii")
            )
        prediction["previews"] = previews
        return prediction

    @app.get("/")
```

app.py

```
    def index():
        return render_template("images.html")

    @app.context_processor
    def project_identity():
        return {"submission": json.loads((ROOT / "configs/submission.json").read_text(encoding="utf-8"))}

    @app.get("/video")
    def video_index():
        return render_template("index.html", classes=DISPLAY)

    def image_examples():
        p = ROOT / "demo/image_examples.json"
        entries = json.loads(p.read_text(encoding="utf-8")) if p.exists() else []
        return [e for e in entries if (ROOT / e["path"]).is_file()]

    def infer_image(path):
        with inference_lock:
            if image_model_holder[0] is None:
                image_model_holder[0] = ImageRecognizer()
            result, views = image_model_holder[0].predict(path)
            if hog_model_holder[0] is None:
                hog_model_holder[0] = HOGRecognizer()
            result["hog_baseline"] = hog_model_holder[0].predict(path)
        result["previews"] = []
        for view in views:
            im = ImageOps.fit(view, (224, 224), method=Image.Resampling.BILINEAR)
            buffer = io.BytesIO()
            im.save(buffer, format="PNG")
            result["previews"].append(
                ""
                + base64.b64encode(buffer.getvalue()).decode("ascii")
            )
        return result

    @app.get("/image-api/examples")
    def image_example_list():
        return jsonify(
            [{k: v for k, v in e.items() if k != "path"} for e in image_examples()]
        )

    @app.get("/image-api/example/<example_id>/thumbnail")
    def image_thumbnail(example_id):
        entry = next((e for e in image_examples() if e["id"] == example_id), None)
        if not entry:
            return jsonify(error="Example not found"), 404
        return send_file(ROOT / entry["path"])

    @app.post("/image-api/example/<example_id>")
    def predict_image_example(example_id):
        entry = next((e for e in image_examples() if e["id"] == example_id), None)
        if not entry:
            return jsonify(error="Example not found"), 404
        result = infer_image(ROOT / entry["path"])
        result["example"] = {k: v for k, v in entry.items() if k != "path"}
        return jsonify(result)

    @app.post("/image-api/predict")
    def predict_image_upload():
        uploaded = request.files.get("image")
        if not uploaded or not uploaded.filename:
            return jsonify(error="Choose an image first."), 400
        suffix = Path(uploaded.filename).suffix.lower()
        if suffix not in {".jpg", ".jpeg", ".png", ".webp", ".bmp"}:
```

app.py

```
            return jsonify(error="Use JPEG, PNG, WebP or BMP."), 400
        with tempfile.TemporaryDirectory(prefix="cricketshot_image_") as folder:
            path = Path(folder) / ("image" + suffix)
            uploaded.save(path)
            return jsonify(infer_image(path))

    @app.get("/image-api/evidence")
    def image_evidence():
        return jsonify(
            json.loads(
                (ROOT / "results/images/metrics.json").read_text(encoding="utf-8")
            )
        )

    @app.get("/api/status")
    def status():
        ready = all(
            (ROOT / "models" / f).exists()
            for f in ("classifier.json", "classifier.npz", "mobilenetv2_features.onnx")
        )
        image_ready = all(
            (ROOT / f).exists()
            for f in (
                "models/image/classifier.json",
                "models/image/heads.npz",
                "models/hog/head.npz",
                "models/mobilenetv2_features.onnx",
            )
        )
        return jsonify(
            ready=image_ready,
            video_pilot_ready=ready,
            version="2.0.0",
            scope="One image, four dataset classes; video is an exploratory backup",
        )

    @app.get("/api/evidence")
    def evidence():
        return jsonify(
            json.loads((ROOT / "results/metrics.json").read_text(encoding="utf-8"))
        )

    @app.get("/api/examples")
    def example_list():
        return jsonify(
            [{k: v for k, v in e.items() if k != "path"} for e in examples()]
        )

    @app.get("/api/example/<example_id>/thumbnail")
    def thumbnail(example_id):
        entry = next((e for e in examples() if e["id"] == example_id), None)
        if not entry:
            return jsonify(error="Example not found"), 404
        return send_file(ROOT / entry["thumbnail"], mimetype="image/jpeg")

    @app.post("/api/example/<example_id>")
    def predict_example(example_id):
        entry = next((e for e in examples() if e["id"] == example_id), None)
        if not entry:
            return jsonify(error="Example not found"), 404
        result = infer(ROOT / entry["path"])
        result["example"] = {k: v for k, v in entry.items() if k != "path"}
        return jsonify(result)

```

app.py

```
    @app.post("/api/predict")
    def predict_upload():
        uploaded = request.files.get("video")
        if not uploaded or not uploaded.filename:
            return jsonify(error="Choose a trimmed video first."), 400
        suffix = Path(uploaded.filename).suffix.lower()
        if suffix not in {".avi", ".mp4", ".mov", ".mkv", ".webm"}:
            return jsonify(error="Use an AVI, MP4, MOV, MKV or WebM video."), 400
        # A private temporary file avoids trusting user-supplied filenames.
        with tempfile.TemporaryDirectory(prefix="cricketshot_") as folder:
            path = Path(folder) / ("upload" + suffix)
            uploaded.save(path)
            return jsonify(infer(path))

    @app.errorhandler(ValueError)
    def invalid_video(exc):
        return jsonify(error=str(exc)), 400

    @app.errorhandler(FileNotFoundError)
    def missing_model(exc):
        return (
            jsonify(
                error="Model files are missing. Follow the setup steps in README.md."
            ),
            503,
        )

    @app.errorhandler(RequestEntityTooLarge)
    def too_large(exc):
        return (
            jsonify(error="The upload limit is 40 MB. Trim or compress the clip."),
            413,
        )

    return app


if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser()
    parser.add_argument("--port", type=int, default=6040)
    args = parser.parse_args()
    create_app().run(host="127.0.0.1", port=args.port, debug=False, threaded=True)
```

configs/experiment.json

```
{
  "seed": 17,
  "frames": 9,
  "split": "5 source-grouped folds: test=0, val=1, train=2,3,4",
  "descriptors": ["middle", "mean", "phase", "phase_delta"],
  "C_candidates": [0.1, 1, 10, 100],
  "selection_metric": "validation macro F1, smallest C wins ties",
  "deployment_descriptor": "phase_delta",
  "view": "batter_roi",
  "temperature_bounds": [0.1, 10.0],
  "review_target_validation_accuracy": 0.9,
  "review_min_validation_coverage": 0.2,
  "bootstrap_source_resamples": 2000,
  "comment": "The original full-frame specification preceded pilot test scores. The ROI revision followed the failed pilot and keeps the same split, so final results are exploratory on a reused holdout. ROI bounds were fixed before ROI scoring. The proposed model remains phase_delta even if an ablation scores higher."
```

configs/experiment.json

```
}
```

configs/hog_experiment.json

```
{
  "status": "Exploratory extension after seeing Option B results; fixed before the new HOG experiment",
  "question": "How does a hand-crafted gradient descriptor compare with frozen CNN features on the identical images and split?",
  "feature": "224 square global-view luminance, unsigned 9-bin HOG, 16x16 cells, 2x2-cell blocks, stride one cell, L2 block and image normalization",
  "split_policy": "Reuse existing manifest, test identities and groups exactly; no split/seed search",
  "C_candidates": [0.1, 1, 10, 100],
  "selection": "Validation macro F1; smallest C wins ties",
  "classifier": "L2-regularized multinomial logistic regression, balanced class weights, lbfgs, max_iter=2000",
  "scaling": "Deterministic per-image normalization only. No learned scaling, PCA or feature selection.",
  "calibration": "None for HOG. Existing CNN temperatures and review threshold remain unchanged.",
  "uncertainty": "2000 paired visual-group bootstrap draws, seed 17, fixed model and split; no independence claim for 945 images",
  "fusion": "No HOG+CNN fusion experiment; keep one focused course comparison"
}
```

configs/image_experiment.json

```
{
  "seed": 604,
  "views": ["full image", "crop x=15..85%, y=5..95%"],
  "modes": ["global", "focus", "concat", "contrast"],
  "deployment_mode": "contrast",
  "duplicate_rule": "Initial RGB/pHash groups strengthened with quarter-turn/mirror pHash and ORB/RANSAC verified frozen-feature nearest-neighbor pairs. See strengthen_image_groups.py for exact criteria. Components stay in one split.",
  "split": "StratifiedGroupKFold(5, shuffle=True, random_state=604): test fold0, val fold1, train other folds",
  "C_candidates": [0.1, 1, 10, 100],
  "selection": "validation macro F1, smallest C wins ties",
  "review": "Lowest threshold 0.50..0.95 step0.05 with agreement of global, focus and fused heads, at least 20 accepted validation images, >=20% validation coverage, >=90% validation accuracy. Fallback threshold=0.80.",
  "status": "The view geometry and head comparison preceded all image scores. Visual QA after the initial hash-grouped score found transformed copies; grouping was strengthened and the comparison rerun. Final revised-grouping scores are exploratory, with initial scores preserved. No source/match IDs are available."
}
```

configs/submission.json

```
{
  "group_name": "EE604 Project Group",
  "github_url": "https://github.com/hrushikeshroop/cricket-shot-recognition-ee604",
  "member_names": [
    "Hrushikesh Roop Avvari",
    "Voora Rakesh",
    "Gokavarapu Manikanta Sri Akshith",
    "Lokesh Kumar Moond",
    "Sharath"
  ],
  "members": [
    {
      "name": "Hrushikesh Roop Avvari",
      "roll_number": "230249"
    },
    {
      "name": "Voora Rakesh",
      "roll_number": "231174"
```

configs/submission.json

```
    },
    {
      "name": "Gokavarapu Manikanta Sri Akshith",
      "roll_number": "230418"
    },
    {
      "name": "Lokesh Kumar Moond",
      "roll_number": "230606"
    },
    {
      "name": "Sharath",
      "roll_number": "230303"
    }
  ]
}
```

pyproject.toml

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "cricketshot-ee604"
version = "1.0.0"
description = "Four-class cricket image recognition with HOG and frozen CNN features"
requires-python = ">=3.11"
dependencies = ["numpy>=1.26", "Pillow>=10", "onnxruntime>=1.17", "opencv-python-headless>=4.9", "Flask>=3.0"]

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
```

requirements-dev.txt

```
-r requirements.txt
scikit-learn>=1.4
scipy>=1.12
threadpoolctl>=3.2
onnx>=1.15
ImageHash>=4.3
matplotlib>=3.8
requests>=2.31
pytest>=8.0
reportlab>=4.0
PyMuPDF>=1.24
```

requirements-lock.txt

```
# Exact versions used on Python 3.14.5 / Windows x64.
blinker==1.9.0
certifi==2026.7.22
charset-normalizer==3.5.2
click==8.5.0
colorama==0.4.6
contourpy==1.4.0
cycler==0.12.1
Flask==3.1.3
```

requirements-lock.txt

```
flatbuffers==25.12.19
fonttools==4.66.1
idna==3.20
ImageHash==4.3.2
iniconfig==2.3.1
itsdangerous==2.2.0
Jinja2==3.1.6
joblib==1.6.0
kiwisolver==1.5.1
MarkupSafe==3.0.4
matplotlib==3.11.2
ml_dtypes==0.6.0
numpy==2.5.3
onnx==1.23.2
onnxruntime==1.31.0
opencv-python-headless==5.0.0.93
packaging==26.3
Pillow==12.3.0
pluggy==1.7.0
protobuf==7.36.2
Pygments==2.21.0
PyMuPDF==1.28.2
pyparsing==3.3.3
pytest==9.1.1
python-dateutil==2.9.0.post0
PyWavelets==1.10.0
reportlab==5.0.1
requests==2.34.2
scikit-learn==1.9.1
scipy==1.18.1
six==1.17.0
threadpoolctl==3.7.0
urllib3==2.8.0
Werkzeug==3.1.9
```

requirements.txt

```
# Demo and inference, Python 3.11 or newer. See requirements-lock.txt for tested versions.
numpy>=1.26
Pillow>=10
onnxruntime>=1.17
opencv-python-headless>=4.9
Flask>=3.0
```

run_demo.bat

```
@echo off
cd /d "%~dp0"
if not exist ".venv\Scripts\python.exe" (
  echo Run setup.bat first.
  pause
  exit /b 1
)
echo CricketShot will be available at http://127.0.0.1:6040
".venv\Scripts\python.exe" app.py
pause
```

scripts/analyze_revision.py

scripts/analyze_revision.py

```
"""Recompute comparisons and paired uncertainty from saved predictions only."""
from pathlib import Path
import sys
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
import csv
import hashlib
import json
import numpy as np
from train_evaluate import metrics


def confusion_statistics(matrix):
    """Accept (..., 4, 4) counts and compute image-weighted accuracy/macro F1."""
    diagonal = np.diagonal(matrix, axis1=-2, axis2=-1)
    accuracy = diagonal.sum(-1) / matrix.sum(axis=(-2, -1))
    denominator = matrix.sum(-1) + matrix.sum(-2)
    f1 = np.divide(2 * diagonal, denominator, out=np.zeros_like(diagonal, dtype=float),
                   where=denominator > 0).mean(-1)
    return np.stack([accuracy, f1], axis=-1)


def main():
    out = ROOT / "results/revision"
    out.mkdir(parents=True, exist_ok=True)
    old = json.loads((ROOT / "results/images/metrics.json").read_text())
    rows = list(csv.DictReader((ROOT / "data/images/processed/manifest.csv").open(encoding="utf-8",
newline="")))
    test_rows = [r for r in rows if r["split"] == "test"]
    with np.load(ROOT / "results/images/test_predictions.npz", allow_pickle=False) as arrays:
        y, ids, groups = [arrays[k].copy() for k in ("y", "ids", "groups")]
        predictions = {mode: arrays["p_" + mode].copy() for mode in ("global", "focus", "concat", "contrast")}
        accepted = arrays["accepted"].copy()
    with np.load(ROOT / "results/hog/test_predictions.npz", allow_pickle=False) as arrays:
        assert np.array_equal(ids, arrays["ids"])
        assert np.array_equal(y, arrays["y"])
        predictions["hog"] = arrays["p"].copy()
    assert ids.tolist() == [r["id"] for r in test_rows]
    assert len(np.unique(groups)) == 70 and len(y) == 945
    classes = old["classes"]
    unique = np.unique(groups)
    modes = list(predictions)
    group_matrices = np.zeros((len(modes), len(unique), 4, 4), dtype=int)
    for mi, mode in enumerate(modes):
        pred = predictions[mode].argmax(1)
        for gi, group in enumerate(unique):
            index = np.flatnonzero(groups == group)
            np.add.at(group_matrices[mi, gi], (y[index], pred[index]), 1)
    rng = np.random.default_rng(17)
    # Resample whole visual groups. Every image of a drawn group is retained.
    # Use the identical draws for all methods to form paired differences.
    draws = np.stack([np.bincount(rng.choice(len(unique), size=len(unique), replace=True),
                                  minlength=len(unique)) for _ in range(2000)])
    boot_matrices = np.einsum("bg,mgij->mbij", draws, group_matrices)
    boot_values = confusion_statistics(boot_matrices)
    comparisons = {}
    for mi, mode in enumerate(modes):
        measured = metrics(y, predictions[mode], classes)
        if mode != "hog":
            assert abs(measured["accuracy"] - old["models"][mode]["test"]["accuracy"]) < 1e-12
            assert abs(measured["macro_f1"] - old["models"][mode]["test"]["macro_f1"]) < 1e-12
```

scripts/analyze_revision.py

```
        interval = np.percentile(boot_values[mi], [2.5, 97.5], axis=0)
        comparisons[mode] = {"test": measured, "correct": int((predictions[mode].argmax(1) == y).sum()),
                             "accuracy_95": interval[:, 0].tolist(),
                             "macro_f1_95": interval[:, 1].tolist()}
    ci, cj = modes.index("contrast"), modes.index("concat")
    difference = boot_values[ci] - boot_values[cj]
    paired_interval = np.percentile(difference, [2.5, 97.5], axis=0)
    paired = {"comparison": "contrast minus concatenation", "correct_difference":
comparisons["contrast"]["correct"] - comparisons["concat"]["correct"],
              "accuracy_difference": comparisons["contrast"]["test"]["accuracy"] -
comparisons["concat"]["test"]["accuracy"],
              "accuracy_difference_95": paired_interval[:, 0].tolist(),
              "macro_f1_difference_95": paired_interval[:, 1].tolist()}
    info = {"status": "Exploratory; prior results already known; no new test-based model choice",
            "models": comparisons, "paired_contrast_concat": paired,
            "uncertainty": {"unit": "visual duplicate group", "groups": len(unique), "draws": 2000, "seed": 17,
                            "method": "percentile cluster bootstrap; image-weighted statistics within
whole-group resamples",
                            "limitations": "Conditions on fixed training, split and imperfect visual groups;
omits training variability and unknown match correlation."},
            "review": {"accepted": int(accepted.sum()),
                       "correct": int(((predictions["contrast"].argmax(1) == y) & accepted).sum()),
                       "coverage": float(accepted.mean()),
                       "accepted_accuracy": float((predictions["contrast"].argmax(1)[accepted] ==
y[accepted]).mean()),
                       "threshold_unchanged": old["review"]["threshold"]},
            "calibration": {"uncalibrated": old["models"]["contrast"]["test_uncalibrated"],
                            "scaled": old["models"]["contrast"]["test"],
                            "conclusion": "Test NLL and ECE worsened. Retain original temperature; no
improvement claim."},
            "manifest_sha256": hashlib.sha256((ROOT /
"data/images/processed/manifest.csv").read_bytes()).hexdigest()}
    (out / "comparison.json").write_text(json.dumps(info, indent=2))
    with (out / "comparison.csv").open("w", newline="", encoding="utf-8") as stream:
        writer = csv.writer(stream)
        writer.writerow(["method", "correct", "n", "accuracy", "macro_f1", "accuracy_ci_low",
"accuracy_ci_high", "f1_ci_low", "f1_ci_high"])
        for mode, result in comparisons.items():
            writer.writerow([mode, result["correct"], len(y), result["test"]["accuracy"],
result["test"]["macro_f1"],
                             *result["accuracy_95"], *result["macro_f1_95"]])
    print(json.dumps({"paired": paired, "review": info["review"], "hog": comparisons["hog"]}, indent=2))


if __name__ == "__main__":
    main()
```

scripts/benchmark.py

```
"""Time end-to-end CPU inference on deterministic held-out clips, without caches."""
from pathlib import Path
import sys
ROOT=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(ROOT/'src'))
import csv
import json
import platform
import numpy as np
from cricketshot.core import Recognizer

def main():
    rows=list(csv.DictReader((ROOT/'data/processed/manifest.csv').open(encoding='utf-8',newline='')))
    rows=[r for r in rows if r['split']=='test']
```

scripts/benchmark.py

```
    model=Recognizer()
    model.predict(ROOT/rows[0]['path'])
    sample=[rows[i] for i in np.linspace(0,len(rows)-1,min(25,len(rows)),dtype=int)]
    times=[]
    for row in sample:
        result,_=model.predict(ROOT/row['path']);times.append(result['seconds'])
    result={'clips':len(times),'warmup_clips':1,'scope':'Decode, sample, nine CNN passes and classifier. Model initialization excluded.',
            'median_seconds':float(np.median(times)),'p95_seconds':float(np.percentile(times,95)),
            'mean_seconds':float(np.mean(times)),'times_seconds':times,'clip_ids':[r['id'] for r in sample],
            'platform':platform.platform(),'processor':platform.processor(),'onnx_intra_op_threads':2}
    (ROOT/'results/latency.json').write_text(json.dumps(result,indent=2),encoding='utf-8')
    print(json.dumps(result,indent=2))

if __name__=='__main__':main()
```

scripts/build_image_report.py

```
"""Compatibility entry point for the course-grounded revised manuscript."""
from build_revision_report import main

if __name__ == "__main__":
    main()
```

scripts/build_report.py

```
"""Shared report utilities and optional supplementary video-pilot manuscript.

Metrics are read from saved experiment evidence, never entered by hand.
Run after benchmarks and tests; use --body-only to compile without code appendix.
"""
from pathlib import Path
import sys
ROOT=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(ROOT/'src'))
import argparse
import csv
import hashlib
import json
import re
import shutil
import subprocess
import textwrap
import pymupdf as fitz
import numpy as np
from reportlab.pdfgen import canvas
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from cricketshot.core import DISPLAY

def pct(x):return f'{x*100:.1f}'
def esc(s):
    return str(s).replace('\\',r'\textbackslash{}').replace('_',r'\_').replace('&',r'\&').replace('%',r'\%').replace('#',r'\#')

def table(headers, rows, caption, label, spec=None):
    spec=spec or ('l'+'r'*(len(headers)-1))
    body='\n'.join(' & '.join(map(str,row))+r' \\' for row in rows)
    return ('\\begin{table}[t]\n\\centering\\small\n\\begin{tabular}{'+spec+'}\n\\toprule\n'+
            ' & '.join(headers)+r' \\'+'\n\\midrule\n'+body+'\n\\bottomrule\n\\end{tabular}\n\\caption{'+caption+'}\n\\label{'+label+'}\n\\end{table}\n')

```

scripts/build_report.py

```
def source_files():
    paths=[ROOT/'app.py',ROOT/'pyproject.toml',ROOT/'requirements.txt',ROOT/'requirements-dev.txt',
           ROOT/'requirements-lock.txt',ROOT/'run_demo.bat',ROOT/'setup.bat']
    for folder in ('src','scripts','tests','templates','static','configs'):
        paths.extend(p for p in (ROOT/folder).rglob('*') if p.is_file() and p.suffix in
{'.py','.ps1','.mjs','.js','.css','.html','.json'})
    return sorted(set(p for p in paths if p.exists()),key=lambda p:p.relative_to(ROOT).as_posix())

def code_appendix(path):
    font='CodeFont';pdfmetrics.registerFont(TTFont(font,'C:/Windows/Fonts/consola.ttf'))
    c=canvas.Canvas(str(path),pagesize=(612,792))
    c.setTitle('Cricket Shot Recognition - Complete Project Source Appendix')
    page=0;yy=0;current='';inventory=[]
    def new_page():
        nonlocal page,yy
        if page:c.showPage()
        page+=1;c.setFillColorRGB(.13,.28,.18);c.setFont('Helvetica-Bold',10)
        c.drawString(36,759,'Appendix B / Complete project source')
        c.setFont('Helvetica',7);c.drawRightString(576,759,f'B-{page}')
        c.setStrokeColorRGB(.8,.85,.78);c.line(36,749,576,749)
        c.setFont(font,7);c.setFillColorRGB(.15,.18,.15);c.drawString(36,735,current[:120]);yy=720
    current='Original code and configuration. Line numbers identify source lines. Wrapped lines continue without
 a number.'
    new_page()
    for p in source_files():
        current=p.relative_to(ROOT).as_posix();data=p.read_bytes();text=data.decode('utf-8-sig')

inventory.append({'path':current,'sha256':hashlib.sha256(data).hexdigest(),'lines':len(text.splitlines())})
        if yy<110:new_page()
        yy-=15;c.setFont('Helvetica-Bold',9);c.setFillColorRGB(.13,.28,.18);c.drawString(36,yy,current);yy-=16
        c.setFont(font,8);c.setFillColorRGB(.15,.18,.15)
        for n,line in enumerate(text.splitlines(),1):
            chunks=textwrap.wrap(line.expandtabs(4),width=112,replace_whitespace=False,drop_whitespace=False,bre
ak_long_words=True,break_on_hyphens=False) or ['']
            for k,chunk in enumerate(chunks):
                if yy<38:
                    new_page();c.setFont(font,8)
                prefix=f'{n:4d}  ' if k==0 else '      '
                c.drawString(36,yy,prefix+chunk);yy-=10.8
        yy-=10
    c.save()
    (ROOT/'results/source_inventory.json').write_text(json.dumps(inventory,indent=2),encoding='utf-8')
    return page

def main():

parser=argparse.ArgumentParser();parser.add_argument('--body-only',action='store_true');args=parser.parse_args()
    report=ROOT/'report';g=report/'generated';g.mkdir(exist_ok=True)
    out=ROOT/'output/pdf';out.mkdir(parents=True,exist_ok=True)
    build=ROOT/'.build/latex/video';build.mkdir(parents=True,exist_ok=True)
    r=json.loads((ROOT/'results/metrics.json').read_text());audit=json.loads((ROOT/'data/processed/audit.json').
read_text())
    latency=json.loads((ROOT/'results/latency.json').read_text());submission=json.loads((ROOT/'configs/submissio
n.json').read_text())
    pilot=json.loads((ROOT/'results/pilot_fullframe/metrics.json').read_text())
    rows=list(csv.DictReader((ROOT/'data/processed/manifest.csv').open(encoding='utf-8',newline='')))
    classes=r['classes'];m=r['models']['phase_delta'];middle=r['models']['middle'];mean=r['models']['mean'];phas
e=r['models']['phase'];b=m['bootstrap']
    counts={s:sum(audit['split_counts'][s].values()) for s in ('train','val','test')}
    data_rows=[[DISPLAY[c]]+[audit['split_counts'][s].get(c,0) for s in
('train','val','test')]+[audit['class_counts'][c]] for c in classes]
    data_rows.append(['Total',counts['train'],counts['val'],counts['test'],audit['decoded_files']])
    (g/'data_table.tex').write_text(table(['Label','Train','Val.','Test','All'],data_rows,'Clip counts from the
prepared manifest. All eight labels occur in each set, but validation has only one unorthodox
```

scripts/build_report.py

```
    clip.','tab:data'),encoding='utf-8')
 89     result_rows=[['Training
    majority','--',pct(r['majority_baseline']['accuracy']),pct(r['majority_baseline']['macro_f1'])]]
 90     names={'color':'Color histogram','middle':'Middle frame','mean':'Temporal mean','phase':'Ordered
    phases','phase_delta':'Phase + diff./std.'}
 91     for mode in names:
 92         info=r['models'][mode];result_rows.append([names[mode],str(info['C']),pct(info['test']['accuracy']),pct(
    info['test']['macro_f1'])])
 93     (g/'results_table.tex').write_text(table(['Representation','$C$','Acc. (\\%)','F1 (\\%)'],result_rows,'All
    learned heads select regularization by validation macro F1. Final visual models use the same fixed ROI and
    split. Color uses the full frame. F1 is macro-averaged.','tab:results'),encoding='utf-8')
 94     cal_rows=[]
 95     for label,key in [('Before calibration','test_uncalibrated'),('After calibration','test')]:
 96
    mm=m[key];cal_rows.append([label,f'{mm["nll"]:.3f}',f'{mm["ece_10_bins"]:.3f}',f'{mm["brier_multiclass"]:.3f}'])
 97     (g/'calibration_table.tex').write_text(table(['Probabilities','NLL','ECE','Brier'],cal_rows,'Proposed model
    on test clips. ECE uses ten equal-width confidence bins. Brier is the unscaled mean sum of squared errors over
    eight classes. Lower is better.','tab:cal'),encoding='utf-8')
 98     robustness=[]
 99     for mode in ('middle','mean','phase_delta'):
100         robustness.append([names[mode],pct(r['models'][mode]['test']['accuracy']),pct(r['robustness']['dark'][mo
    de]['accuracy']),pct(r['robustness']['blur'][mode]['accuracy']),pct(r['models'][mode]['temporal_reversal']['accu
    racy'])])
101
    (g/'robustness_table.tex').write_text(table(['Representation','Clean','Dark','Blur','Reverse'],robustness,'Test
    accuracy in percent. Every corruption uses the complete unchanged test set. Reverse changes temporal order
    only.','tab:robust'),encoding='utf-8')
102     delta=(m['test']['macro_f1']-middle['test']['macro_f1'])*100
103     finding=(f'The proposed representation exceeds the middle-frame macro F1 by {delta:.1f} percentage points.'
    if delta>=0 else f'The middle-frame baseline exceeds the proposed representation in macro F1 by {-delta:.1f}
    percentage points.')
104     best=max(r['models'],key=lambda k:r['models'][k]['test']['macro_f1'])
105     discussion=(f'Table~\\ref{{tab:results}} shows that the proposed representation has
    {pct(m["test"]["accuracy"])}\\% accuracy and {pct(m["test"]["macro_f1"])}\\% macro F1.
    '+finding.replace('percentage points.','percentage points.').replace('%',r'\%')+
106                 f' The highest test macro F1 among these learned heads is
    {pct(r["models"][best]["test"]["macro_f1"])}\\%, from {names[best].lower()}. We report that result even when it
    does not favor the proposed descriptor. '
107                 f'Adding difference and spread to the ordered phase means changes test macro F1 by
    {(m["test"]["macro_f1"]-phase["test"]["macro_f1"])*100:+.1f} points. This is a joint ablation. The full-frame
    pilot and ROI experiment also differ in their input crop, so a crop improvement should not be attributed to
    temporal pooling alone.')
108     recalls=sorted([(c,m['test']['per_class'][c]['recall'],m['test']['per_class'][c]['support']) for c in
    classes],key=lambda x:x[1])
109     low,high=recalls[0],recalls[-1]
110     class_text=(f'Figure~\\ref{{fig:confusion}} shows uneven class behavior. {DISPLAY[high[0]]} recall is
    {pct(high[1])}\\% on {int(high[2])} clips, while {DISPLAY[low[0]].lower()} recall is {pct(low[1])}\\% on
    {int(low[2])}. Defence has only three test clips, so one changed prediction moves its recall by 33.3 points. The
     single unorthodox validation clip also makes class-specific tuning unreliable.')
111     sel=m['test_selective'];th=m['threshold_selection'];target='The validation target was met.' if
    th['target_met'] else 'No candidate met the stated validation target, so the 0.80 fallback was used.'
112     selective_acc='undefined because no clip is accepted' if sel['accuracy'] is None else
    f'{pct(sel["accuracy"])}\\%'
113     calibration=(f'The fitted temperature is {m["temperature"]:.3f} and the review threshold is
    {m["review_threshold"]:.2f}. {target} On test clips, the rule accepts {sel["accepted"]} of {m["test"]["n"]}
    ({pct(sel["coverage"])}\\% coverage), with accepted accuracy {selective_acc}. This operating result is an
    observation, not a promised 90\\% accuracy. Temperature scaling leaves the confusion matrix unchanged. A
    calibrated score is still not evidence that an unknown upload belongs to one of the eight classes.')
114     reverse=m['temporal_reversal']['accuracy']-m['test']['accuracy']
115     reverse_text=('The drop is consistent with some use of temporal order, but it does not identify which
    movements the model recognizes.' if reverse<0 else 'Reversal does not reduce accuracy here, which weakens the
    claim that this model has learned a reliable direction of stroke progression.')
116
    dark=r['robustness']['dark']['phase_delta']['accuracy'];blur=r['robustness']['blur']['phase_delta']['accuracy']
```

scripts/build_report.py

```
    robust_text=f'The proposed model scores {pct(dark)}\\% in the darkened condition and {pct(blur)}\\% under blur. Neither corruption result is used to retrain or select the model.'
    with np.load(ROOT/'results/test_predictions.npz',allow_pickle=False) as a:
        y,p,ids=a['y'],a['p_phase_delta'],a['ids']
    wrong=np.flatnonzero(p.argmax(1)!=y);i=int(wrong[np.argmax(p[wrong].max(1))]);row=next(x for x in rows if x['id']==str(ids[i]))
    failtext=f'Figure~\\ref{{fig:failure}} is clip \\texttt{{{esc(row["id"])}}}, whose upstream label is {DISPLAY[classes[y[i]]].lower()}. The model predicts {DISPLAY[classes[p[i].argmax()]].lower()} with score {pct(p[i].max())}\\%. We deliberately include this confident error in the demo rather than showing only successful clips.'
    url=submission.get('github_url')
    note='Project repository: \\url{'+url+'}.' if url else 'The group will choose and publish its GitHub repository after reviewing the project. The required repository URL is not yet available and must be inserted before submission.'
    values={'N':str(audit['decoded_files']),'ACC':pct(m['test']['accuracy']),'F1':pct(m['test']['macro_f1']),
'MID_ACC':pct(middle['test']['accuracy']),'MID_F1':pct(middle['test']['macro_f1']),'ABSTRACT_FINDING':finding,
            'TEST_N':str(counts['test']),'TRAIN_N':str(counts['train']),'VAL_N':str(counts['val']),
'LARGEST_N':str(max(audit['class_counts'].values())),'SMALLEST_N':str(min(audit['class_counts'].values())),
            'SOURCE_N':str(audit['sources']),'ORIGINAL_SOURCE_N':str(len(set(re.sub(r'_\d+_\d+$','',Path(x['original_name']).stem) for x in rows))),
            'GROUP_N':str(audit['union_groups']),'EXACT_DUP':str(audit['exact_duplicate_files']),'NEAR_DUP':str(len(audit['cross_source_near_duplicates'])),
            'DECODE_ERRORS':str(len(audit['errors'])),'TRAIN_G':str(audit['split_groups']['train']),'VAL_G':str(audit['split_groups']['val']),'TEST_G':str(audit['split_groups']['test']),
            'PILOT_ACC':pct(pilot['models']['phase_delta']['test']['accuracy']),'PILOT_F1':pct(pilot['models']['phase_delta']['test']['macro_f1']),
            'RESULTS_DISCUSSION':discussion,'CLASS_DISCUSSION':class_text,'ACC_LOW':pct(b['accuracy_95'][0]),'ACC_HIGH':pct(b['accuracy_95'][1]),
'F1_LOW':pct(b['macro_f1_95'][0]),'F1_HIGH':pct(b['macro_f1_95'][1]),'CALIBRATION_DISCUSSION':calibration,
            'REVERSE_ACC':pct(m['temporal_reversal']['accuracy']),'REVERSE_DISCUSSION':reverse_text,'ROBUST_DISCUSSION':robust_text,
            'FAILURE_DISCUSSION':failtext,'LATENCY_N':str(latency['clips']),'MEDIAN_MS':f'{latency["median_seconds"]*1000:.0f}',
            'P95_MS':f'{latency["p95_seconds"]*1000:.0f}','SUBMISSION_NOTE':note,
            'CONCLUSION':finding+' The ablations show how much of the result survives changes in temporal representation and input quality. Because the ROI revision followed pilot test feedback, the results should guide a larger independent evaluation rather than serve as a final generalization claim.'}
    body=(report/'content_template.tex').read_text(encoding='utf-8')
    for key,value in values.items():body=body.replace('{{'+key+'}}',value)
    if re.search(r'\{\{[A-Z_]+\}\}',body):raise ValueError('Unresolved report variables')
    (g/'body.tex').write_text(body,encoding='utf-8')
    (g/'authors.tex').write_text('\\author{'+esc(submission['group_name'])+r'\\Indian Institute of Technology Kanpur\\EE604, October 2026}'+'\n',encoding='utf-8')
    repo=note+'\n\nDataset: \\url{https://www.kaggle.com/datasets/amittalmale/cricket-shots}. Pretrained encoder: \\url{https://github.com/onnx/models}. Libraries: \\url{https://github.com/scikit-learn/scikit-learn}, \\url{https://github.com/microsoft/onnxruntime}, \\url{https://github.com/opencv/opencv}, \\url{https://github.com/pallets/flask}. No other dataset or cricket-model repository is used for training. The archived CVPR 2022 style is from \\url{https://cvpr2022.thecvf.com/author-guidelines}.\n'
    (g/'repository.tex').write_text(repo,encoding='utf-8')
    for name in ('cvpr.sty','ieee_fullname.bst'):
        shutil.copy2(report/'template/latex'/name,report/name)
    compiler=ROOT/'tools/tectonic/tectonic.exe'
subprocess.run([str(compiler),'--keep-logs','--outdir',str(build),str(report/'main.tex')],cwd=report,check=True)
    shutil.copy2(build/'main.pdf',out/'Video_Pilot_Report.pdf')
    if not args.body_only:
        pages=code_appendix(out/'CricketShot_Code_Appendix.pdf')
        complete=fitz.open(out/'Video_Pilot_Report.pdf')
        appendix=fitz.open(out/'CricketShot_Code_Appendix.pdf');complete.insert_pdf(appendix)
        complete.set_metadata({'title':'Cricket Shot Recognition - EE604 report with full source appendix','author':submission['group_name']})
```

scripts/build_report.py

```
        complete.save(out/'Video_Pilot_Report_Complete.pdf',garbage=4,deflate=True)
        print('Complete report',len(complete),'pages, source appendix',pages,'pages')
    print('Report generated from measured evidence.')

if __name__=='__main__':main()
```

scripts/build_revision_report.py

```
"""Build the course revision from saved evidence, including its full source."""
from pathlib import Path
import json
import re
import shutil
import subprocess
import zipfile
import pymupdf as fitz
from build_report import ROOT, pct, table, code_appendix, esc


def main():
    report = ROOT / "report"
    generated = report / "generated"
    generated.mkdir(exist_ok=True)
    out = ROOT / "output/pdf"
    out.mkdir(parents=True, exist_ok=True)
    build = ROOT / ".build/latex/revision"
    build.mkdir(parents=True, exist_ok=True)
    original = json.loads((ROOT/"results/images/metrics.json").read_text())
    comparison = json.loads((ROOT/"results/revision/comparison.json").read_text())
    hog = json.loads((ROOT/"results/hog/metrics.json").read_text())
    audit = json.loads((ROOT/"data/images/processed/audit.json").read_text())
    latency = json.loads((ROOT/"results/revision/latency.json").read_text())
    sub = json.loads((ROOT/"configs/submission.json").read_text())
    names = {"drive":"Drive", "legglance-flick":"Glance/flick", "pullshot":"Pull", "sweep":"Sweep"}
    data_rows = [[names[c]]+[audit["split_counts"][s][c] for s in ("train","val","test")]
                 for c in original["classes"]]
    data_rows.append(["Total",2834,944,945])
    (generated/"image_data_table.tex").write_text(table(["Label","Train","Val.","Test"],data_rows,
        "Retained image counts; all visual groups remain in one split.","tab:data"))
    labels={"hog":"HOG", "global":"CNN global", "focus":"CNN crop", "concat":"Concat.", "contrast":"Contrast"}
    rows=[];intervals=[]
    for mode in labels:
        info=hog if mode=="hog" else original["models"][mode]
        result=comparison["models"][mode]
        rows.append([labels[mode],info["dimension"],info["C"],pct(result["test"]["accuracy"]),pct(result["test"]
["macro_f1"])])
        intervals.append([labels[mode],f'{pct(result["accuracy_95"][0])}--{pct(result["accuracy_95"][1])}',
                          f'{pct(result["macro_f1_95"][0])}--{pct(result["macro_f1_95"][1])}'])
    (generated/"image_results_table.tex").write_text(table(["Feature","Dim.","$C$","Acc.","F1"],rows,
        "All-test accuracy and macro F1 in percent. Same images, labels, groups, split and regularization grid;
HOG has no ImageNet pretraining.","tab:results"))
    (generated/"image_uncertainty_table.tex").write_text(table(["Feature","Acc. 95\\%","F1 95\\%"],intervals,
        "Conditional visual-group bootstrap intervals, in percent, from saved individual
predictions.","tab:intervals"))
    contrast=original["models"]["contrast"]
    values={"HOG_ACC":pct(hog["test"]["accuracy"]),"HOG_F1":pct(hog["test"]["macro_f1"]),
            "HOG_VAL_F1":pct(hog["validation"]["macro_f1"]),"HOG_C":str(hog["C"]),
            "GLOBAL_ACC":pct(original["models"]["global"]["test"]["accuracy"]),
            "GLOBAL_F1":pct(original["models"]["global"]["test"]["macro_f1"]),
            "ACC":pct(contrast["test"]["accuracy"]),"F1":pct(contrast["test"]["macro_f1"]),
            "PAIR_LOW":f'{100*comparison["paired_contrast_concat"]["accuracy_difference_95"][0]:+.2f}',
            "PAIR_HIGH":f'{100*comparison["paired_contrast_concat"]["accuracy_difference_95"][1]:+.2f}',
            "ECE_BEFORE":f'{contrast["test_uncalibrated"]["ece_10_bins"]:.3f}',
```

scripts/build_revision_report.py

```
            "ECE_AFTER":f'{contrast["test"]["ece_10_bins"]:.3f}',
            "NLL_BEFORE":f'{contrast["test_uncalibrated"]["nll"]:.3f}',
            "NLL_AFTER":f'{contrast["test"]["nll"]:.3f}',
            "MEDIAN_MS":f'{latency["median_seconds"]*1000:.0f}',
            "P95_MS":f'{latency["p95_seconds"]*1000:.0f}'}
    recalls=contrast["test"]["per_class"]
    values["CLASS_DISCUSSION"]=(f'Glance/flick recall is {pct(recalls["legglance-flick"]["recall"])}\\% on 224 examples, '
        f'while pull recall is {pct(recalls["pullshot"]["recall"])}\\% on 252. ')
    body=(report/"image_content_template.tex").read_text(encoding="utf-8")
    for key,value in values.items():body=body.replace("{{"+key+"}}",value)
    if re.search(r"\{\{[A-Z_]+\}\}",body):raise ValueError("Unresolved report variable")
    (generated/"image_body.tex").write_text(body,encoding="utf-8")
    members = [esc(m["name"])+" ("+esc(m["roll_number"])+")" for m in sub["members"]]
    author_lines = [members[0]+r"\quad "+members[1], members[2], members[3]+r"\quad "+members[4]]
    (generated/"authors.tex").write_text("\\author{"+r"\\".join(author_lines)+r"\\Indian Institute of Technology Kanpur\\EE604, October 2026}"+"\n")
    calibration=[]
    for label,key in [("Unscaled","test_uncalibrated"),("Scaled","test")]:
        m=contrast[key];calibration.append([label,f'{m["nll"]:.3f}',f'{m["ece_10_bins"]:.3f}',f'{m["brier_multiclass"]:.3f}'])
    cal_table=table(["Scores","NLL","ECE","Brier"],calibration,"Unchanged contrast scores on test; lower is better. Scaling worsens NLL and ECE.","tab:calibration").replace("[t]","[h]")
    stress=[]
    for mode in ("global","contrast"):
        stress.append([labels[mode],pct(original["models"][mode]["test"]["accuracy"])]+[
            pct(original["stress"][condition][mode]["accuracy"]) for condition in ("dark","blur","flip")])
    stress_table=table(["Head","Clean","Dark","Blur","Mirror"],stress,"Preserved descriptive stress-test accuracy in percent; no retraining or parameter choice.","tab:stress").replace("[t]","[h]")
    url=sub.get("github_url")
    repository=(r"Project repository: \url{"+url+"}." if url else
                "The real GitHub URL is pending the group's decision and must be inserted before submission. No repository has been published for this handoff.")
    supplement=r"""\section{Supplementary evidence}
\subsection{Visual-copy audit}\label{app:audit}
The initial audit uses decoded RGB hashes, 64-bit perceptual hash Hamming distance at most six, and 16 by 16 RGB thumbnail mean absolute difference at most 18. Exact copies collapse; conflicting-label components are quarantined. The stronger audit adds quarter-turn/mirror hashes (same bit cutoff, aligned thumbnail difference at most 24), then proposes the nearest 15 frozen-feature neighbors within cosine distance 0.20.

ORB binary descriptors and a RANSAC homography verify proposed geometric links. Requirements are a 0.72 match-ratio cutoff, at least 16 filtered matches, 12 inliers, 65\% inlier fraction and 8\% convex-hull coverage in both images. This adds 948 transform and 1,797 geometric edges. There are no final conflicting components. All linked copies and initial hash components stay in one split. These checks reduce leakage but can still miss related photographs or join visually similar ones. Full criteria and preserved edges are in the commented audit script and data directory.

\subsection{Secondary diagnostics}\label{app:secondary}
"""+cal_table+r"""
The original temperature is approximately 0.802. Validation is reused for regularization, temperature and review selection. The review search chooses the lowest threshold from 0.50 to 0.95 in steps of 0.05 attaining 90\% validation accuracy with at least 20 accepted images and 20\% coverage, with 0.80 fallback. Its chosen 0.70 threshold stays unchanged. At that threshold confidence alone covers 62.2\% with 77.6\% accepted accuracy; adding agreement covers 58.9\% with 77.7\%. This modest difference is descriptive, not a guaranteed improvement.

"""+stress_table+r"""
Dark multiplies source brightness by 0.6; blur uses Gaussian radius two at source resolution; mirror reflects horizontally while retaining the four broad labels. Both CNN views are recomputed. These conditions do not represent a new match or camera, and blur severity varies with source resolution. Original predictions, stress evidence and CNN-only timings remain under \texttt{results/images/}; preliminary hash-only evidence remains under \texttt{results/images\_initial\_hash/}.

\subsection{Exploratory video pilot}\label{app:video}
The separate Kaggle CRICKET\_SHOTS archive~\cite{dataset} contains 562 clips in eight labels and 25 normalized
```

scripts/build_revision_report.py

```
source-title groups. Its test set has 116 clips from five groups. Nine uniformly sampled frames feed the frozen encoder. Three ordered phase means, late-minus-early difference and temporal standard deviation form 6,400 features. This sparse temporal summary is motivated by segment-based representations~\cite{wang}, without reproducing TSN training.

The full-frame pilot obtains 18.1\% accuracy. A fixed upper-center crop, introduced after pilot test feedback, reaches 30.2\% accuracy and 21.3\% macro F1. The training-majority baseline also reaches 30.2\% accuracy (5.8\% macro F1). Equal majority accuracy is not a successful general video recognizer. Defence has only three test clips; validation has one unorthodox clip. The reused holdout makes this exploratory. Original and revised predictions remain available; different data and labels prevent a direct image-versus-video comparison.

\subsection{Code, data and repositories}
The complete report appends every current custom executable source and configuration, with a SHA-256 inventory. Public libraries, pretrained weights and the unmodified CVPR style are not original code. The project includes dependency files, manifests, numeric heads, saved predictions, tests and reproduction commands. The separate course mapping and viva guide explain lecture references, advanced audit tools and one measured image trace.

"""+repository+r"""

Data: \href{https://www.kaggle.com/datasets/aneesh10/cricket-shot-dataset}{Cricket Shots Dataset v2} and \href{https://www.kaggle.com/datasets/amittalmale/cricket-shots}{CRICKET\_SHOTS v1}. Pretrained weights: \href{https://github.com/onnx/models}{ONNX Model Zoo}. Libraries: \href{https://github.com/scikit-learn/scikit-learn}{scikit-learn}, \href{https://github.com/microsoft/onnxruntime}{ONNX Runtime}, \href{https://github.com/opencv/opencv}{OpenCV}, \href{https://github.com/pallets/flask}{Flask}, and \href{https://github.com/numpy/numpy}{NumPy}. Complete dependency links and attribution accompany \texttt{docs/sources.md}. Our NumPy HOG follows the cited method and supplied lectures; no external cricket classifier code was imported.
"""
    (generated/"image_supplement.tex").write_text(supplement,encoding="utf-8")
    for name in ("cvpr.sty","ieee_fullname.bst"):
        shutil.copy2(report/"template/latex"/name,report/name)
    subprocess.run([str(ROOT/"tools/tectonic/tectonic.exe"),"--keep-logs","--outdir",str(build),str(report/"image_main.tex")],cwd=report,check=True)
    shutil.copy2(build/"image_main.pdf",out/"CricketShot_Report_Main.pdf")
    count=code_appendix(out/"CricketShot_Code_Appendix.pdf")
    complete=fitz.open(out/"CricketShot_Report_Main.pdf")
    appendix=fitz.open(out/"CricketShot_Code_Appendix.pdf");complete.insert_pdf(appendix)
    complete.set_metadata({"title":"Cricket Shot Recognition - EE604 course revision", "author":";".join(sub["member_names"])})
    complete.save(out/"CricketShot_Report_Complete.pdf",garbage=4,deflate=True)
    with zipfile.ZipFile(ROOT/"output/CricketShot_Overleaf.zip","w",zipfile.ZIP_DEFLATED) as z:
        main=(report/"image_main.tex").read_text()
        main=re.sub(r"\\graphicspath\{.*?\}\n",lambda _:r"\graphicspath{{figures/}}"+"\n",main)
        z.writestr("main.tex",main)
        for name in ("cvpr.sty","ieee_fullname.bst","references.bib"):z.write(report/name,name)
        for path in generated.glob("*.tex"):
            if path.name.startswith("image_") or path.name=="authors.tex":z.write(path,"generated/"+path.name)
        for folder in (ROOT/"results/images/figures",ROOT/"results/revision/figures"):
            for path in folder.iterdir():
                if path.suffix in {".pdf",".png",".jpg"}:z.write(path,"figures/"+path.name)
        z.write(out/"CricketShot_Code_Appendix.pdf","code_appendix.pdf")
        z.write(out/"CricketShot_Report_Complete.pdf","complete_report.pdf")
        z.write(ROOT/"scripts/merge_report.py","merge_report.py")
        z.writestr("README.txt","Compile main.tex for the revised CVPR manuscript and supplementary evidence. The unmodified 2022 style is included.\nThe current complete report and entire source appendix are included. After editing, export main.pdf and run:\npython -m pip install pymupdf\npython merge_report.py main.pdf code_appendix.pdf complete_updated.pdf\nThe PDF merge avoids the 2022 style's embedded eso-pic conflict with pdfpages.\n")
    print(f"Revised manuscript with supplement: {len(complete)-len(appendix)} pages; entire source appendix: {count}; complete: {len(complete)}")


if __name__=="__main__":main()
```

scripts/build_slides.ps1

scripts/build_slides.ps1

```
param([string]$SpecPath = "slides/content.json")
# Create an editable presentation with native PowerPoint text, tables and diagrams.
# Open and export without a presentation window. Run outside the sandbox because
# Office COM cannot attach to a user logon session inside the sandbox.
$ErrorActionPreference = 'Stop'
$workspace = Split-Path -Parent $PSScriptRoot
$spec = Get-Content -LiteralPath (Join-Path $workspace $SpecPath) -Raw -Encoding UTF8 | ConvertFrom-Json
$outDir = Join-Path $workspace 'output/presentation'
$previewDir = Join-Path $workspace 'output/presentation/previews'
New-Item -ItemType Directory -Force -Path $outDir,$previewDir | Out-Null

function RGB([string]$hex) {
    $v = $hex.TrimStart('#')
    return [Convert]::ToInt32($v.Substring(0,2),16) + 256*[Convert]::ToInt32($v.Substring(2,2),16) +
65536*[Convert]::ToInt32($v.Substring(4,2),16)
}
$ink = RGB '20382A'; $green = RGB '426B42'; $muted = RGB '72806B'; $paper = RGB 'F7F8F1'; $orange = RGB 'AF623E'

function Text($slide,[string]$text,[double]$x,[double]$y,[double]$w,[double]$h,[double]$size=18,[bool]$bold=$fal
se,[int]$color=$ink) {
    $shape=$slide.Shapes.AddTextbox(1,[single]$x,[single]$y,[single]$w,[single]$h)
    $shape.TextFrame.MarginLeft=0; $shape.TextFrame.MarginRight=0
    $shape.TextFrame.MarginTop=0; $shape.TextFrame.MarginBottom=0
    $shape.TextFrame.WordWrap=-1; $shape.TextFrame.AutoSize=0
    $range=$shape.TextFrame.TextRange; $range.Text=$text
    $range.Font.Name='Arial'; $range.Font.Size=$size; $range.Font.Color.RGB=$color
    $range.Font.Bold=$(if($bold){-1}else{0})
    $range.ParagraphFormat.SpaceAfter=8
    $shape.TextFrame2.AutoSize=0
    $shape.TextFrame2.WordWrap=-1
    $shape.Height=[single]$h
    return $shape
}

function Picture($slide,[string]$relative,[double]$x,[double]$y,[double]$w,[double]$h) {
    $path=Join-Path $workspace $relative
    if (!(Test-Path -LiteralPath $path)) {throw "Missing slide asset: $relative"}
    Add-Type -AssemblyName System.Drawing
    $image=[System.Drawing.Image]::FromFile($path)
    try {$ratio=[double]$image.Width/$image.Height} finally {$image.Dispose()}
    $width=[Math]::Min($w,$h*$ratio);$height=$width/$ratio
    $slide.Shapes.AddPicture($path,0,-1,[single]($x+($w-$width)/2),[single]($y+($h-$height)/2),[single]$width,[s
ingle]$height) | Out-Null
}

function NativeTable($slide,$tableSpec) {
    $rows=@($tableSpec.rows); $headers=@($tableSpec.headers)
    $shape=$slide.Shapes.AddTable($rows.Count+1,$headers.Count,[single]$tableSpec.x,[single]$tableSpec.y,[single
]$tableSpec.w,[single]$tableSpec.h)
    $table=$shape.Table
    if($tableSpec.widths){for($j=0;$j -lt
$headers.Count;$j++){$table.Columns.Item($j+1).Width=[single]$tableSpec.widths[$j]}}
    for($i=0;$i -le $rows.Count;$i++) {
        for($j=0;$j -lt $headers.Count;$j++) {
            $cell=$table.Cell($i+1,$j+1)
            $val=if($i -eq 0){[string]$headers[$j]}else{[string]$rows[$i-1][$j]}
            $cell.Shape.TextFrame.TextRange.Text=$val
            $cell.Shape.TextFrame.MarginLeft=9; $cell.Shape.TextFrame.MarginRight=6
            $cell.Shape.TextFrame.MarginTop=5; $cell.Shape.TextFrame.MarginBottom=4
            $cell.Shape.TextFrame.VerticalAnchor=3
            $font=$cell.Shape.TextFrame.TextRange.Font
```

scripts/build_slides.ps1

```
            $font.Name='Arial';$font.Size=[double]$tableSpec.fontSize
            if($i -eq 0){$cell.Shape.Fill.ForeColor.RGB=$ink;$font.Color.RGB=RGB 'FFFFFF';$font.Bold=-1}
            else {
                $cell.Shape.Fill.ForeColor.RGB=$(if($i%2 -eq 0){RGB 'EEF2E7'}else{RGB 'FFFFFF'})
                $font.Color.RGB=$ink;$font.Bold=0
                if($tableSpec.heatmap -and $j -gt 0){
                    $den=[double]$tableSpec.totals[$i-1];$fraction=if($den -gt 0){[double]$val/$den}else{0}
                    $rr=[int](246-170*$fraction);$gg=[int](248-135*$fraction);$bb=[int](238-165*$fraction)
                    $cell.Shape.Fill.ForeColor.RGB=$rr+256*$gg+65536*$bb
                    if($fraction -gt .6){$font.Color.RGB=RGB 'FFFFFF'}
                }
            }
            for($b=1;$b -le 4;$b++){$cell.Borders.Item($b).ForeColor.RGB=RGB 'D8DFD0';$cell.Borders.Item($b).Weight=.5}
        }
    }
}

$app=$null;$deck=$null
try {
    $app=New-Object -ComObject PowerPoint.Application
    $deck=$app.Presentations.Add(0)
    $deck.PageSetup.SlideWidth=960;$deck.PageSetup.SlideHeight=540
    foreach($entry in $spec.slides) {
        $slide=$deck.Slides.Add($deck.Slides.Count+1,12)

$slide.FollowMasterBackground=0;$slide.Background.Fill.Solid();$slide.Background.Fill.ForeColor.RGB=$paper
        if($entry.kind -eq 'cover') {
            Text $slide $entry.kicker 48 45 800 24 11 $true $green | Out-Null
            Text $slide $entry.title 48 115 500 185 44 $true $ink | Out-Null
            Text $slide $entry.subtitle 50 323 440 85 20 $false $muted | Out-Null
            Picture $slide $entry.image 565 100 350 315
            if($entry.team_text) {
                $teamShape=Text $slide $entry.team_text 50 423 850 68 12 $false $ink
                $teamShape.TextFrame.TextRange.ParagraphFormat.SpaceAfter=2
                Text $slide $entry.bottom 50 506 850 19 10 $false $muted | Out-Null
            } else {
                Text $slide $entry.bottom 50 473 850 25 11 $false $muted | Out-Null
            }
        } else {
            Text $slide $entry.title 48 32 865 48 32 $true $ink | Out-Null
            if($entry.subtitle){Text $slide $entry.subtitle 49 90 855 44 15 $false $muted | Out-Null}
            if($entry.text){Text $slide ($entry.text -join "`r`r") 50 145 850 310 20 $false $ink | Out-Null}
            foreach($block in $entry.blocks){
                Text $slide $block.text $block.x $block.y $block.w $block.h $block.size ([bool]$block.bold) $(if($block.color){RGB $block.color}else{$ink}) | Out-Null
            }
            foreach($imageSpec in $entry.images){Picture $slide $imageSpec.path $imageSpec.x $imageSpec.y $imageSpec.w $imageSpec.h}
            foreach($tableSpec in $entry.tables){NativeTable $slide $tableSpec}
            if($entry.diagram){
                $boxes=@($entry.diagram)
                for($i=0;$i -lt $boxes.Count;$i++){
                    $xx=50+$i*176
                    $box=$slide.Shapes.AddShape(1,[single]$xx,176,155,115)
                    $box.Fill.ForeColor.RGB=RGB 'E7EEDD';$box.Line.ForeColor.RGB=RGB 'B3C3A6';$box.Line.Weight=1
                    Text $slide $boxes[$i] ($xx+12) 194 132 85 17 $true $ink | Out-Null
                    if($i -lt $boxes.Count-1){$line=$slide.Shapes.AddLine([single]($xx+158),233,[single]($xx+173),233);$line.Line.EndArrowheadStyle=3;$line.Line.ForeColor.RGB=$green}
                }
            }
            if($entry.callout){Text $slide $entry.callout 50 449 855 42 16 $false $orange | Out-Null}
            Text $slide 'CRICKET SHOT RECOGNITION / EE604' 50 511 750 15 8 $false $muted | Out-Null
            Text $slide ([string]$slide.SlideIndex) 883 508 25 18 10 $false $muted | Out-Null
```

scripts/build_slides.ps1

```
        }

if($entry.notes){$slide.NotesPage.Shapes.Placeholders.Item(2).TextFrame.TextRange.Text=[string]$entry.notes}
    }
    $pptx=Join-Path $outDir 'CricketShot_Presentation.pptx'
    $pdf=Join-Path $outDir 'CricketShot_Presentation.pdf'
    $deck.SaveAs($pptx,24)
    $deck.SaveAs($pdf,32)
    $deck.Export($previewDir,'PNG',1600,900)
    # Verify text geometry after the actual Office renderer has laid out all text.
    $issues=@()
    foreach($slide in $deck.Slides){
        foreach($shape in $slide.Shapes){
            if($shape.HasTextFrame -eq -1 -and $shape.TextFrame.HasText -eq -1){
                $range=$shape.TextFrame.TextRange
                if($range.BoundHeight -gt
($shape.Height+2)){$issues+=@{slide=$slide.SlideIndex;shape=$shape.Name;issue='Text height exceeds
shape';height=$shape.Height;boundHeight=$range.BoundHeight;text=$range.Text}}
                if($shape.Left -lt -1 -or $shape.Top -lt -1 -or $shape.Left+$shape.Width -gt 962 -or
$shape.Top+$shape.Height -gt 542){$issues+=@{slide=$slide.SlideIndex;shape=$shape.Name;issue='Shape outside
slide'}}
            }
        }
    }
    $record=@{renderer='Microsoft PowerPoint 16
COM';slide_count=$deck.Slides.Count;issues=@($issues);font='Arial';source='slides/content.json'}
    $record | ConvertTo-Json -Depth 8 | Set-Content -LiteralPath (Join-Path $workspace
'results/presentation_validation.json') -Encoding UTF8
    Write-Output "Saved $($deck.Slides.Count) editable slides. Geometry issues: $($issues.Count)"
} finally {
    if($deck){$deck.Close();[System.Runtime.InteropServices.Marshal]::ReleaseComObject($deck)|Out-Null}
    if($app){$app.Quit();[System.Runtime.InteropServices.Marshal]::ReleaseComObject($app)|Out-Null}
    [GC]::Collect();[GC]::WaitForPendingFinalizers()
}
```

scripts/extract_roi_features.py

```
"""Second, explicitly exploratory, experiment after observing the pilot failure.

The split is unchanged. ROI geometry was fixed before computing ROI validation
and test results. This is development on a reused holdout, not pristine testing.
"""
from pathlib import Path
import sys
ROOT=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(ROOT/'src'))
import csv
import json
import shutil
import numpy as np
from cricketshot.core import Encoder, decode_video

def main():
    d=ROOT/'data/processed'
    # Preserve the full-frame experiment and never overwrite its evidence.
    original=d/'features_fullframe.npz'
    if not original.exists():shutil.copy2(d/'features.npz',original)
    cache=d/'roi_feature_cache';cache.mkdir(exist_ok=True)
    rows=list(csv.DictReader((d/'manifest.csv').open(encoding='utf-8',newline='')))
    encoder=Encoder(view='batter_roi');all_features=[]
    for i,row in enumerate(rows):
        cp=cache/(row['sha256']+'.npy')
        if cp.exists():feat=np.load(cp,allow_pickle=False)
```

scripts/extract_roi_features.py

```
        else:
            frames,_=decode_video(ROOT/row['path']);feat=encoder.encode(frames)
            np.save(cp,feat,allow_pickle=False)
        all_features.append(feat)
        if (i+1)%50==0:print(f'ROI features {i+1}/{len(rows)}',flush=True)
    with np.load(original,allow_pickle=False) as a:
        data={k:a[k] for k in a.files}
    data['features']=np.stack(all_features)
    np.savez_compressed(d/'features.npz',**data)
    record={'view':'batter_roi','normalized_bounds':[.30,.02,.70,.65],
            'motivation':'Full-frame pilot showed poor recognition. Small batters and broadcast context motivated a fixed upper-center crop.',
            'evaluation_status':'Exploratory reused holdout. Original test results were observed before this revision.',
            'split':'Unchanged source-grouped split, seed 17','detector':False}
    (ROOT/'results/roi_protocol.json').write_text(json.dumps(record,indent=2),encoding='utf-8')
    print('ROI features ready.',flush=True)

if __name__=='__main__':main()
```

scripts/fetch_assets.py

```
"""Download public upstream assets, keeping URLs and checksums for provenance."""
from pathlib import Path
import hashlib
import json
import requests
import zipfile

ROOT = Path(__file__).resolve().parents[1]

def fetch(url, target, expected_sha256=None):
    target = ROOT / target
    target.parent.mkdir(parents=True, exist_ok=True)
    if not target.exists():
        with requests.get(url, stream=True, timeout=(30, 180)) as r:
            r.raise_for_status()
            with target.with_suffix(target.suffix + '.part').open('wb') as f:
                for chunk in r.iter_content(1024 * 1024):
                    f.write(chunk)
        target.with_suffix(target.suffix + '.part').replace(target)
    digest = hashlib.sha256(target.read_bytes()).hexdigest()
    if expected_sha256 and digest != expected_sha256:
        raise ValueError(f'Checksum mismatch for {target.name}. Refusing changed or incomplete asset.')
    print(target.relative_to(ROOT), target.stat().st_size, digest, flush=True)
    return target, {'url': url, 'path': str(target.relative_to(ROOT)), 'sha256': digest,
                    'bytes': target.stat().st_size, 'accessed': '2026-10-17'}

def main():
    provenance = []
    ref = 'amittalmale/cricket-shots'
    meta = requests.get('https://www.kaggle.com/api/v1/datasets/view/' + ref, timeout=60)
    meta.raise_for_status()
    (ROOT / 'data').mkdir(exist_ok=True)
    (ROOT / 'data/source_metadata.json').write_text(json.dumps(meta.json(), indent=2), encoding='utf-8')
    archive, item = fetch('https://www.kaggle.com/api/v1/datasets/download/' + ref + '?datasetVersionNumber=1', 'data/downloads/cricket-shots-v1.zip', '1b4ceb40a8c976c2a06b532aa0280b82fdf41cfc1d83cd228e9159cfa5b5053d')
    provenance.append(item)
    out = ROOT / 'data/raw'
    out.mkdir(exist_ok=True)
    with zipfile.ZipFile(archive) as z:
        for member in z.infolist():
            dest = (out / member.filename).resolve()
```

scripts/fetch_assets.py

```
            if not dest.is_relative_to(out.resolve()):
                raise ValueError('Unsafe archive path')
            if not dest.exists():
                z.extract(member, out)
        print('Archive entries', len(z.infolist()), flush=True)
        print('\n'.join(z.namelist()[:10]), flush=True)
    _, item = fetch('https://media.githubusercontent.com/media/onnx/models/main/validated/vision/classification/mobilenet/model/mobilenetv2-12.onnx', 'models/mobilenetv2-12.onnx', 'c0c3f76d93fa3fd6580652a45618618a220fced18babf65774ed169de0432ad5')
    provenance.append(item)
    (ROOT / 'data/provenance.json').write_text(json.dumps(provenance, indent=2), encoding='utf-8')

if __name__ == '__main__':
    main()
```

scripts/fetch_images.py

```
"""Fetch the public four-class still-image dataset and preserve its metadata."""
from pathlib import Path
import json
import zipfile
import requests
from fetch_assets import fetch
ROOT=Path(__file__).resolve().parents[1]

def main():
    ref='aneesh10/cricket-shot-dataset'
    meta=requests.get('https://www.kaggle.com/api/v1/datasets/view/'+ref,timeout=60);meta.raise_for_status()
    out=ROOT/'data/images';out.mkdir(parents=True,exist_ok=True)
    (out/'source_metadata.json').write_text(json.dumps(meta.json(),indent=2),encoding='utf-8')
    version=2
    archive,item=fetch('https://www.kaggle.com/api/v1/datasets/download/'+ref+f'?datasetVersionNumber={version}',f'data/downloads/cricket-images-v{version}.zip','9d497543aca86587c65e167b19f4704e1e639af61da0dbe30a3a7d1485d53e62')
    (out/'provenance.json').write_text(json.dumps(item,indent=2),encoding='utf-8')
    raw=out/'raw';raw.mkdir(exist_ok=True)
    with zipfile.ZipFile(archive) as z:
        for member in z.infolist():
            if not (raw/member.filename).resolve().is_relative_to(raw.resolve()):raise ValueError('Unsafe archive path')
            if not (raw/member.filename).exists():z.extract(member,raw)
        print('Images archive entries',len(z.infolist()),z.namelist()[:5],flush=True)

if __name__=='__main__':main()
```

scripts/image_figures.py

```
"""Figures, selected examples and latency for the four-class image experiment."""
from pathlib import Path
import sys
ROOT=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(ROOT/'src'))
import csv
import json
import numpy as np
from PIL import Image,ImageDraw,ImageFont,ImageOps
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
from cricketshot.images import ImageRecognizer,LABELS,load_image,image_views

```

scripts/image_figures.py

```
def main():
    out=ROOT/'results/images/figures';out.mkdir(parents=True,exist_ok=True)
    assets=ROOT/'slides/assets';assets.mkdir(parents=True,exist_ok=True)
    examples=ROOT/'demo/images';examples.mkdir(parents=True,exist_ok=True)
    r=json.loads((ROOT/'results/images/metrics.json').read_text());classes=r['classes']
    audit=json.loads((ROOT/'data/images/processed/audit.json').read_text())
    rows=list(csv.DictReader((ROOT/'data/images/processed/manifest.csv').open(encoding='utf-8',newline='')));by_id={x['id']:x for x in rows}
    with np.load(ROOT/'results/images/test_predictions.npz',allow_pickle=False) as a:
        y,p,ids,agreement,accepted=a['y'],a['p_contrast'],a['ids'],a['agreement'],a['accepted']
    font=ImageFont.truetype('C:/Windows/Fonts/arial.ttf',21)
    chosen=[]
    for ci,c in enumerate(classes):
        candidates=np.flatnonzero((y==ci)&(p.argmax(1)==ci));candidates=candidates if len(candidates) else np.flatnonzero(y==ci)
        jpeg_candidates=[int(i) for i in candidates if Path(by_id[str(ids[i])]['path']).suffix.lower() in ('.jpg','.jpeg')]
        pool=np.array(jpeg_candidates,dtype=int) if jpeg_candidates else candidates
        chosen.append(int(pool[np.argmax(p[pool,ci])]))
    wrong=np.flatnonzero(p.argmax(1)!=y);fail=int(wrong[np.argmax(p[wrong].max(1))]) if len(wrong) else None
    low=int(np.argmin(p.max(1)));disagree=np.flatnonzero(~agreement);dis=int(disagree[0]) if len(disagree) else low
    entries=[]
    for i,tag in [(chosen[0],'Selected success'),(chosen[2],'Selected success'),(fail,'Confident failure'),(dis,'View disagreement')]:
        if i is None or str(ids[i]) in [e['id'] for e in entries]:continue
        row=by_id[str(ids[i])];dest=examples/(row['id']+'.png');load_image(ROOT/row['path']).save(dest)
        entries.append({'id':row['id'],'label':row['label'],'path':dest.relative_to(ROOT).as_posix(),'split':'test',
                        'short_label':LABELS[row['label']],'note':tag+'. Chosen after evaluation for the demo.'})
    (ROOT/'demo/image_examples.json').write_text(json.dumps(entries,indent=2),encoding='utf-8')
    used={Path(e['path']).name for e in entries}
    for old in examples.glob('image_*'):
        if old.is_file() and old.name not in used:old.unlink()
    # A four-image sheet of dataset labels. Each image is an actual held-out example.
    sheet=Image.new('RGB',(1200,720),'#f7f8f1');draw=ImageDraw.Draw(sheet)
    for j,i in enumerate(chosen):
        row=by_id[str(ids[i])];im=load_image(ROOT/row['path']);tile=ImageOps.contain(im,(570,300));x=(j%2)*600+15;yy=(j//2)*360+40
        sheet.paste(tile,(int(x+(570-tile.width)/2),yy));draw.text((x,yy-30),LABELS[row['label']],fill='#254832',font=font)
        im.save(assets/f'class_{j}.jpg',quality=95)
    sheet.save(out/'class_examples.jpg',quality=95)
    if fail is not None:
        im=load_image(ROOT/by_id[str(ids[fail])]['path']);views=image_views(im)
        sheet=Image.new('RGB',(1200,520),'white');d=ImageDraw.Draw(sheet)
        title=f'Dataset: {LABELS[classes[y[fail]]]} / Predicted: {LABELS[classes[p[fail].argmax()]]} / Score {p[fail].max()*100:.1f}%'
        d.text((15,10),title,fill='#254832',font=font)
        for j,v in enumerate(views):
            tile=ImageOps.contain(v,(570,445));sheet.paste(tile,(15+j*600,60))
        sheet.save(out/'failure.jpg',quality=95)
        im.save(assets/'failure_original.jpg',quality=95)
    cm=np.array(r['models']['contrast']['test']['confusion']);cmn=cm/cm.sum(1,keepdims=True)
    plt.rcParams.update({'font.family':'DejaVu Sans','font.size':9,'axes.spines.top':False,'axes.spines.right':False,'savefig.dpi':200})
    labels=['Drive','Glance/flick','Pull','Sweep']
    fig,ax=plt.subplots(figsize=(4.8,4.2));img=ax.imshow(cmn,vmin=0,vmax=1,cmap='Greens')
    for i in range(4):
        for j in range(4):ax.text(j,i,str(cm[i,j]),ha='center',va='center',color='white' if cmn[i,j]>.6 else '#233b2a',fontsize=11)
    ax.set_xticks(range(4),labels);ax.set_yticks(range(4),labels);ax.set_xlabel('Predicted label');ax.set_ylabel('Dataset label');fig.colorbar(img,ax=ax,shrink=.8,label='Within-class
```

scripts/image_figures.py

```
    fraction');fig.tight_layout();fig.savefig(out/'confusion.png');plt.close(fig)
    fig,ax=plt.subplots(figsize=(6,2.7));x=np.arange(4);bottom=np.zeros(4)
    for split,color in [('train','#3c6b46'),('val','#9eae8c'),('test','#b8754b')]:
        vals=[audit['split_counts'][split][c] for c in
    classes];ax.bar(x,vals,bottom=bottom,label=split,color=color);bottom+=vals
    ax.set_xticks(x,labels);ax.set_ylabel('Retained
    images');ax.legend(frameon=False);fig.tight_layout();fig.savefig(out/'distribution.png');plt.close(fig)
    fig,ax=plt.subplots(figsize=(4.8,3.0));conf=p.max(1);correct=p.argmax(1)==y
    for use_agreement,name,color in [(False,'Confidence only','#9eae8c'),(True,'Confidence +
    agreement','#3c6b46')]:
        xx=[];yy=[]
        for th in np.linspace(0,1,101):
            keep=(conf>=th)&(agreement if use_agreement else True)
            if keep.any():xx.append(keep.mean()*100);yy.append(correct[keep].mean()*100)
        ax.plot(xx,yy,label=name,color=color)
    ax.set_xlim(0,100);ax.set_ylim(0,100);ax.set_xlabel('Test coverage (%)');ax.set_ylabel('Accepted accuracy
    (%)');ax.legend(frameon=False,fontsize=8);fig.tight_layout();fig.savefig(out/'review_curve.png');plt.close(fig)
    fig,ax=plt.subplots(figsize=(7,2));ax.axis('off');ax.set_xlim(0,10);ax.set_ylim(0,2)
    items=[(.1,'Image + central crop\n224 x 224 inputs'),(2.55,'Shared frozen encoder\n2 x 1280
    features'),(5.0,'Concatenate vectors\n+ absolute difference'),(7.45,'Linear head + temperature\nagreement review
     gate')]
    for x,text in items:ax.add_patch(FancyBboxPatch((x,.55),2.1,.9,boxstyle='round,pad=.05',edgecolor='#3c6b46',
    facecolor='#edf2e5'));ax.text(x+1.05,1,text,ha='center',va='center',fontsize=8)
    for x in
    (2.25,4.7,7.15):ax.annotate('',xy=(x+.22,1),xytext=(x,1),arrowprops={'arrowstyle':'->','color':'#3c6b46'})
    fig.tight_layout(pad=.1);fig.savefig(out/'method.pdf',bbox_inches='tight');fig.savefig(out/'method.png',bbox
    _inches='tight');plt.close(fig)
    # End-to-end timing, one warmup, 50 uniformly indexed held-out images.
    model=ImageRecognizer();test=[by_id[str(i)] for i in ids];model.predict(ROOT/test[0]['path']);times=[]
    for i in np.linspace(0,len(test)-1,min(50,len(test)),dtype=int):
        result,_=model.predict(ROOT/test[i]['path']);times.append(result['seconds'])

    latency={'n':len(times),'median_seconds':float(np.median(times)),'p95_seconds':float(np.percentile(times,95)),
             'scope':'Image decoding, two CNN views and all three inference heads. One warmup, model
    initialization excluded.',
             'times_seconds':times,'onnx_threads':2}
    (ROOT/'results/images/latency.json').write_text(json.dumps(latency,indent=2),encoding='utf-8')
    print('Image figures, examples and latency saved',flush=True)

if __name__=='__main__':main()
```

scripts/inspect_environment.py

```
from pathlib import Path
from collections import Counter
import re
import json
import requests
import zipfile
from html.parser import HTMLParser

ROOT = Path(__file__).resolve().parents[1]
p = list((ROOT/'data/raw').rglob('*.avi'))
print('Classes', Counter(x.parent.name for x in p))
groups = Counter(re.sub(r'_\d+_\d+$', '', x.stem) for x in p)
print('Source groups', len(groups), json.dumps(groups, indent=2))

class Links(HTMLParser):
    def handle_starttag(self, tag, attrs):
        for k, v in attrs:
            if k == 'href' and v and ('zip' in v.lower() or 'author_kit' in v.lower()):
                print('Author kit URL', v)

```

scripts/inspect_environment.py

```
Links().feed(requests.get('https://cvpr2022.thecvf.com/author-guidelines', timeout=30).text)
d = requests.get('https://api.github.com/repos/cvpr-org/author-kit/tags', timeout=30).json()
print('CVPR tags', [x['name'] for x in d])
for path in ['C:/Program Files/Microsoft Office/root/Office16/POWERPNT.EXE', 'C:/Program Files/LibreOffice/program/soffice.exe']:
    print(path, Path(path).exists())

url = 'https://github.com/tectonic-typesetting/tectonic/releases/download/tectonic%400.17.0/tectonic-0.17.0-x86_64-pc-windows-msvc.zip'
out = ROOT/'tools/tectonic'
out.mkdir(parents=True, exist_ok=True)
if not (out/'tectonic.exe').exists():
    r = requests.get(url, timeout=120)
    r.raise_for_status()
    import io
    with zipfile.ZipFile(io.BytesIO(r.content)) as z:
        z.extractall(out)
print('Tectonic ready', flush=True)
```

scripts/make_figures.py

```
"""Create report figures and honest demo examples directly from saved predictions."""
from pathlib import Path
import sys
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / 'src'))
import csv
import json
import shutil
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
import numpy as np
from PIL import Image, ImageDraw, ImageFont, ImageOps
from cricketshot.core import decode_video, DISPLAY

GREEN, ORANGE, GRAY = '#326847', '#bc7045', '#98a58e'

def main():
    out = ROOT / 'results/figures'
    out.mkdir(parents=True, exist_ok=True)
    (ROOT / 'demo/clips').mkdir(parents=True, exist_ok=True)
    (ROOT / 'demo/thumbnails').mkdir(parents=True, exist_ok=True)
    r = json.loads((ROOT / 'results/metrics.json').read_text())
    audit = json.loads((ROOT / 'data/processed/audit.json').read_text())
    rows = list(csv.DictReader((ROOT / 'data/processed/manifest.csv').open(encoding='utf-8', newline='')))
    by_id = {row['id']: row for row in rows}
    with np.load(ROOT / 'results/test_predictions.npz', allow_pickle=False) as a:
        y, p, ids = a['y'], a['p_phase_delta'], a['ids']
    classes = r['classes']
    labels = [DISPLAY[c] for c in classes]
    plt.rcParams.update({'font.family': 'DejaVu Sans', 'font.size': 9, 'axes.spines.top': False,
                         'axes.spines.right': False, 'figure.facecolor': 'white', 'savefig.dpi': 200})
    fig, ax = plt.subplots(figsize=(7, 2.9))
    x = np.arange(len(classes))
    bottom = np.zeros(len(classes))
    for s, color in [('train',GREEN),('val',GRAY),('test',ORANGE)]:
        values = [audit['split_counts'][s].get(c, 0) for c in classes]
        ax.bar(x, values, bottom=bottom, label={'train':'Train','val':'Validation','test':'Test'}[s], color=color)
        bottom += values
    ax.set_xticks(x, labels, rotation=20, ha='right'); ax.set_ylabel('Clips'); ax.legend(frameon=False)
```

scripts/make_figures.py

```
    fig.tight_layout(); fig.savefig(out/'class_distribution.png'); plt.close(fig)
    modes = ['middle','mean','phase','phase_delta']
    fig, ax = plt.subplots(figsize=(7, 3.0))
    x = np.arange(4); width=.32
    ax.bar(x-width/2, [r['models'][m]['test']['accuracy']*100 for m in modes],width,color=GREEN,label='Accuracy')
    ax.bar(x+width/2, [r['models'][m]['test']['macro_f1']*100 for m in modes],width,color=ORANGE,label='Macro F1')
    ax.set_xticks(x,['Middle frame','Mean','Phase','Phase + delta']);ax.set_ylim(0,100);ax.set_ylabel('Test score (%)');ax.legend(frameon=False)
    fig.tight_layout();fig.savefig(out/'ablations.png');plt.close(fig)
    cm = np.array(r['models']['phase_delta']['test']['confusion'])
    norm = cm / cm.sum(1, keepdims=True)
    fig, ax = plt.subplots(figsize=(5.4,4.6))
    im=ax.imshow(norm,cmap='Greens',vmin=0,vmax=1)
    for i in range(len(classes)):
        for j in range(len(classes)):
            ax.text(j,i,str(cm[i,j]),ha='center',va='center',fontsize=8,color='white' if norm[i,j]>.6 else '#24382a')
    ax.set_xticks(range(8),labels,rotation=45,ha='right');ax.set_yticks(range(8),labels)
    ax.set_xlabel('Predicted label');ax.set_ylabel('Dataset label')
    fig.colorbar(im,ax=ax,shrink=.75,label='Within-class fraction')
    fig.tight_layout();fig.savefig(out/'confusion_matrix.png');plt.close(fig)
    fig, ax = plt.subplots(figsize=(4,3))
    confidence=p.max(1);correct=p.argmax(1)==y
    thresholds=np.linspace(0,1,101);cov=[];acc=[]
    for th in thresholds:
        mask=confidence>=th
        if mask.any():cov.append(mask.mean()*100);acc.append(correct[mask].mean()*100)
    ax.plot(cov,acc,color=GREEN,linewidth=2)
    model=r['models']['phase_delta'];sel=model['test_selective']
    if sel['accepted']:
        ax.scatter(sel['coverage']*100,sel['accuracy']*100,color=ORANGE,zorder=3,label='Validation-chosen threshold')
    ax.set_xlim(0,100);ax.set_ylim(0,100);ax.set_xlabel('Test coverage (%)');ax.set_ylabel('Accuracy on accepted clips (%)')
    if sel['accepted']:ax.legend(frameon=False,fontsize=7)
    fig.tight_layout();fig.savefig(out/'selective_curve.png');plt.close(fig)
    # Actual early/middle/late samples, with labels only from the upstream dataset.
    font_path='C:/Windows/Fonts/arial.ttf'
    font=ImageFont.truetype(font_path,18) if Path(font_path).exists() else ImageFont.load_default()
    small=ImageFont.truetype(font_path,13) if Path(font_path).exists() else ImageFont.load_default()
    chosen=[]
    for ci, c in enumerate(classes):
        candidates=np.flatnonzero(y==ci)
        # Deterministic high confidence per class, documented as selected examples.
        correct_candidates=candidates[p[candidates].argmax(1)==ci]
        use=correct_candidates if len(correct_candidates) else candidates
        i=int(use[np.argmax(p[use,ci])]); chosen.append(i)
    failures=np.flatnonzero(p.argmax(1)!=y)
    fail=int(failures[np.argmax(confidence[failures])]) if len(failures) else None
    uncertain=int(np.argmin(confidence))
    demo_choices=[]
    # Four visible examples include a failure and low confidence case when distinct.
    for i,tag in [(chosen[3],'Selected correct example'),(chosen[5],'Selected correct example'),
                  (fail,'Confident failure example'),(uncertain,'Low-confidence example')]:
        if i is not None and i not in [x[0] for x in demo_choices]:
            demo_choices.append((i,tag))
    entries=[]
    for n,(i,tag) in enumerate(demo_choices):
        row=by_id[str(ids[i])];src=ROOT/row['path'];suffix=src.suffix
        dest=ROOT/f'demo/clips/{row["id"]}{suffix}';shutil.copy2(src,dest)
        frames,_=decode_video(src)
        thumb=f'demo/thumbnails/{row["id"]}.jpg'
```

scripts/make_figures.py

```
        Image.fromarray(frames[4]).save(ROOT/thumb,quality=90)
        entries.append({'id':row['id'],'label':row['label'],'path':dest.relative_to(ROOT).as_posix(),
                        'thumbnail':thumb,'note':tag+'. Chosen after evaluation for demonstration.',
                        'source':row['source'],'split':'test'})
    (ROOT/'demo/examples.json').write_text(json.dumps(entries,indent=2),encoding='utf-8')
    for page in range(2):
        sheet=Image.new('RGB',(1200,820),'white');draw=ImageDraw.Draw(sheet)
        for k,i in enumerate(chosen[page*4:page*4+4]):
            row=by_id[str(ids[i])];frames,_=decode_video(ROOT/row['path']);yy=k*205
            draw.text((12,yy+7),f'{DISPLAY[row["label"]]} / {row["id"]} / test',fill='#254932',font=font)
            for phase,t in enumerate((1,4,7)):
                tile=ImageOps.contain(Image.fromarray(frames[t]),(370,155))
                sheet.paste(tile,(12+phase*395,yy+37))
        sheet.save(out/f'dataset_examples_{page+1}.jpg',quality=94)
    if fail is not None:
        row=by_id[str(ids[fail])];frames,_=decode_video(ROOT/row['path'])
        sheet=Image.new('RGB',(1200,285),'white');draw=ImageDraw.Draw(sheet)
        draw.text((12,8),f'Dataset: {DISPLAY[row["label"]]} / Predicted: {DISPLAY[classes[p[fail].argmax()]]} /
Score {confidence[fail]*100:.1f}%',fill='#254932',font=font)
        for j,t in enumerate((1,4,7)):
            tile=ImageOps.contain(Image.fromarray(frames[t]),(380,220));sheet.paste(tile,(12+j*395,45))
        sheet.save(out/'failure_example.jpg',quality=94)
    # Original editable method figure for the report, using measured dimensions.
    fig, ax=plt.subplots(figsize=(7.2,1.9));ax.axis('off');ax.set_xlim(0,10);ax.set_ylim(0,2)
    blocks=[(.1,'Trimmed clip\n9 uniform frames'),(2.55,'Frozen MobileNetV2\n9 x 1280 features'),(5.0,'3 phase
means\nlate - early + std'),(7.45,'Linear classifier\ncalibration + review')]
    for xx,label in blocks:
        ax.add_patch(FancyBboxPatch((xx,.55),2.1,.9,boxstyle='round,pad=0.07',edgecolor=GREEN,facecolor='#f2f6eb
',linewidth=1))
        ax.text(xx+1.05,1,label,ha='center',va='center',fontsize=8)
    for xx in (2.25,4.7,7.15):
        ax.annotate('',xy=(xx+.22,1),xytext=(xx,1),arrowprops={'arrowstyle':'->','color':GREEN})
    fig.tight_layout(pad=.1);fig.savefig(out/'method_overview.pdf',bbox_inches='tight');fig.savefig(out/'method_
overview.png',bbox_inches='tight');plt.close(fig)
    print('Figures and demo examples generated from measured predictions.')

if __name__=='__main__':
    main()
```

scripts/merge_report.py

```
"""Append the complete code PDF to a freshly exported Overleaf manuscript."""
import argparse
from pathlib import Path
import pymupdf

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('manuscript',type=Path)
    parser.add_argument('appendix',type=Path)
    parser.add_argument('output',type=Path)
    args=parser.parse_args()
    if args.output.resolve() in {args.manuscript.resolve(),args.appendix.resolve()}:
        raise ValueError('Choose a new output path; keep the input PDFs intact.')
    with pymupdf.open(args.manuscript) as paper,pymupdf.open(args.appendix) as code:
        paper.insert_pdf(code)
        paper.save(args.output,garbage=4,deflate=True)
        print('Saved',len(paper),'pages to',args.output)

if __name__=='__main__':main()
```

scripts/package_project.py

scripts/package_project.py

```
"""Compatibility entry point; full is the default revised handoff."""
from package_revised import main

if __name__ == "__main__":
    main()
```

scripts/package_revised.py

```
"""Archive the full comparison or a compact demo; verify every payload hash."""
from pathlib import Path
import argparse
import hashlib
import json
import time
import zipfile

ROOT=Path(__file__).resolve().parents[1]
EXCLUDED={".venv",".build",".git",".pytest_cache","__pycache__",".codex",".agents",".aws"}
STORED={".zip",".jpg",".jpeg",".png",".webp",".avi",".mp4",".pdf",".pptx",".docx"}


def digest(path):
    h=hashlib.sha256()
    with path.open("rb") as stream:
        for block in iter(lambda:stream.read(1024*1024),b""):h.update(block)
    return h.hexdigest()


def select_files(kind):
    for path in sorted(ROOT.rglob("*")):
        if not path.is_file():continue
        relative=path.relative_to(ROOT)
        if any(p in EXCLUDED for p in relative.parts):continue
        name=relative.as_posix()
        if path.suffix in {".pyc",".part",".log",".aux",".blg",".bbl"}:continue
        if name.startswith("output/"):
            if path.name!="CricketShot_Overleaf.zip" and path.suffix not in {".pdf",".pptx",".PNG"}:continue
        if kind=="demo":
            if
name.startswith(("data/raw/","data/downloads/","data/images/raw/","tools/","course_notes/")):continue
            if name.startswith("data/") and path.suffix in {".npy",".npz"}:continue
            if "/cache/" in name or "/feature_cache/" in name or "/roi_feature_cache/" in name:continue
            if name=="models/mobilenetv2-12.onnx":continue
        yield path


def main():
    parser=argparse.ArgumentParser();parser.add_argument("--kind",choices=["full","demo"],default="full")
    args=parser.parse_args();output=ROOT/"output";output.mkdir(exist_ok=True)
    archive=output/("CricketShot_Revised_"+("Full" if args.kind=="full" else "Demo")+".zip")
    partial=archive.with_suffix(".zip.part")
    files=list(select_files(args.kind));entries=[];started=time.monotonic();last=started
    print(f"Packaging {len(files)} {args.kind} files",flush=True)
    prefix="CricketShot_EE604_Revised/"
    guide=("# Start here\n\nRead README.md and docs/demo_guide.md. Open the report and slides under output/. "
           "See docs/course_mapping.md and docs/viva_guide.md. On a new Windows machine with Python 3.11+, "
           "run setup.bat then run_demo.bat and open http://127.0.0.1:6040. No machine environment is
included.\n\n"
           "The full package includes raw data, original dataset ZIPs, computed research features, models,
evidence and lecture PDFs. "
```

scripts/package_revised.py

```
            "The smaller demo omits training assets, lecture PDFs and build tools; full-data tests require the full package. "
            "Machine environments, temporary build/Python caches, account metadata and duplicate handoff archives are excluded.\n\n"
            "The original CNN scores remain 65.5% accuracy and 64.9% macro F1. HOG scores 35.3% and 34.5%. "
            "Novelty and generalization remain limited. Team identities and the public repository URL are in README.md and the report. "
            "The public code repository is linked in README.md. ARCHIVE_MANIFEST.json lists all payload sizes and SHA-256 values.\n")
        with zipfile.ZipFile(partial,"w",zipfile.ZIP_DEFLATED,compresslevel=6,allowZip64=True) as z:
            for i,path in enumerate(files,1):
                name=path.relative_to(ROOT).as_posix()
                entries.append({"path":name,"bytes":path.stat().st_size,"sha256":digest(path)})
                z.write(path,prefix+name,compress_type=zipfile.ZIP_STORED if path.suffix.lower() in STORED else zipfile.ZIP_DEFLATED)
                if time.monotonic()-last>=10 or i==len(files):
                    print(f"Packed {i}/{len(files)} ({time.monotonic()-started:.0f}s)",flush=True);last=time.monotonic()
            content=guide.encode("utf-8");z.writestr(prefix+"COMPARE_FIRST.md",content)

            entries.append({"path":"COMPARE_FIRST.md","bytes":len(content),"sha256":hashlib.sha256(content).hexdigest()})
            manifest={"kind":args.kind,"workspace_file_count":len(files),"files":entries,"excluded_directories":sorted(EXCLUDED)}
            manifest_bytes=(json.dumps(manifest,indent=2)+"\n").encode("utf-8")
            z.writestr(prefix+"ARCHIVE_MANIFEST.json",manifest_bytes)
        with zipfile.ZipFile(partial) as z:
            names=z.namelist();expected={prefix+entry["path"] for entry in entries}|{prefix+"ARCHIVE_MANIFEST.json"}
            assert set(names)==expected and len(names)==len(expected)
            assert z.read(prefix+"ARCHIVE_MANIFEST.json")==manifest_bytes
            for entry in entries:
                h=hashlib.sha256();size=0
                with z.open(prefix+entry["path"]) as stream:
                    for block in iter(lambda:stream.read(1024*1024),b""):h.update(block);size+=len(block)
                assert size==entry["bytes"] and h.hexdigest()==entry["sha256"],entry["path"]
        assert [p.relative_to(ROOT).as_posix() for p in select_files(args.kind)]==[p.relative_to(ROOT).as_posix() for p in files]
        partial.replace(archive);archive_hash=digest(archive)
        result={"archive":archive.name,"bytes":archive.stat().st_size,"sha256":archive_hash,
                "workspace_files":len(files),"entries":len(entries)+1,"all_payload_hashes_sizes_and_crcs_pass":True,
                "workspace_membership_rechecked":True,"seconds":round(time.monotonic()-started,1)}
        (output/(archive.stem+"_verification.json")).write_text(json.dumps(result,indent=2))
        archive.with_suffix(".zip.sha256").write_text(archive_hash+"  "+archive.name+"\n",encoding="ascii")
        print(json.dumps(result,indent=2),flush=True)


if __name__=="__main__":main()
```

scripts/prepare_data.py

```
"""Audit real videos, establish source-disjoint splits, cache frozen CNN features."""
from pathlib import Path
import sys
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / 'src'))
import csv
import hashlib
import json
import re
from collections import Counter
import cv2
import imagehash
import numpy as np
import onnx
```

scripts/prepare_data.py

```
from PIL import Image
from sklearn.model_selection import StratifiedGroupKFold
from cricketshot.core import Encoder, decode_video, color_descriptor

def source_name(stem):
    name = re.sub(r'_\d+_\d+$', '', stem)
    name = re.sub(r'[^a-z0-9]+', ' ', name.lower()).strip()
    name = re.sub(r'\b(icc|wt20|match|highlights|cricket|semi|final)\b', '', name)
    name = re.sub(r'\bv\b', 'vs', name)
    name = re.sub(r'\s+', ' ', name).strip()
    # Two Ireland/Oman highlight titles differ only in v/vs and the word match.
    # Sorting team names also conservatively merges a reversed title pairing.
    return ' vs '.join(sorted(s.strip() for s in name.split(' vs ')))

def build_encoder():
    source = ROOT / 'models/mobilenetv2-12.onnx'
    out = ROOT / 'models/mobilenetv2_features.onnx'
    if out.exists():
        return
    model = onnx.load(source)
    # Expose the global pooled 1280-dimensional tensor before ImageNet's head.
    pools = [n for n in model.graph.node if n.op_type == 'GlobalAveragePool']
    if len(pools) != 1:
        raise ValueError('Unexpected upstream encoder graph')
    tensor = pools[0].output[0]
    onnx.utils.extract_model(str(source), str(out), [model.graph.input[0].name], [tensor])
    onnx.checker.check_model(onnx.load(out))

def main():
    out = ROOT / 'data/processed'
    out.mkdir(parents=True, exist_ok=True)
    build_encoder()
    encoder = Encoder(view='full')
    paths = sorted((ROOT / 'data/raw').rglob('*.avi'))
    if not paths:
        raise RuntimeError('No videos. Run python scripts/fetch_assets.py first.')
    rows, features, colors, hashes, errors = [], [], [], [], []
    cache = out / 'feature_cache'
    cache.mkdir(exist_ok=True)
    for n, path in enumerate(paths):
        sha = hashlib.sha256(path.read_bytes()).hexdigest()
        cp = cache / (sha + '.npz')
        try:
            if cp.exists():
                with np.load(cp, allow_pickle=False) as a:
                    feat, col, hs = a['features'], a['color'], a['phash']
                    meta = json.loads(str(a['metadata']))
            else:
                frames, meta = decode_video(path)
                feat, col = encoder.encode(frames), color_descriptor(frames)
                hs = np.array([str(imagehash.phash(Image.fromarray(frames[i]))) for i in (0, 4, 8)])
                np.savez_compressed(cp, features=feat, color=col, phash=hs, metadata=json.dumps(meta))
            rows.append({'id': f'clip_{n:04d}', 'path': path.relative_to(ROOT).as_posix(),
                         'label': path.parent.name, 'source': source_name(path.stem),
                         'original_name': path.name, 'sha256': sha,
                         'frames': meta['decoded_frames'], 'duration_seconds': meta['duration_seconds']})
            features.append(feat)
            colors.append(col)
            hashes.append([int(h, 16) for h in hs])
        except Exception as exc:
            errors.append({'path': path.relative_to(ROOT).as_posix(), 'error': str(exc)})
        if (n+1) % 25 == 0:
            print(f'Encoded {n+1}/{len(paths)} clips, errors={len(errors)}', flush=True)
    # Union source groups when exact or strongly similar clips cross source names.
```

scripts/prepare_data.py

```
    parent = list(range(len(rows)))
    def find(i):
        while parent[i] != i:
            parent[i] = parent[parent[i]]
            i = parent[i]
        return i
    def union(i, j):
        parent[find(j)] = find(i)
    sources, shas = {}, {}
    for i, row in enumerate(rows):
        for key, mapping in ((row['source'], sources), (row['sha256'], shas)):
            if key in mapping:
                union(i, mapping[key])
            else:
                mapping[key] = i
    near = []
    for i in range(len(rows)):
        for j in range(i):
            if rows[i]['source'] == rows[j]['source']:
                continue
            distances = [(a ^ b).bit_count() for a, b in zip(hashes[i], hashes[j])]
            if max(distances) <= 6 and np.mean(distances) <= 4:
                union(i, j)
                near.append({'a': rows[j]['id'], 'b': rows[i]['id'], 'distances': distances,
                             'label_conflict': rows[j]['label'] != rows[i]['label']})
    labels = np.array([r['label'] for r in rows])
    groups = np.array([find(i) for i in range(len(rows))])
    folds = list(StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=17).split(np.zeros(len(rows)),
labels, groups))
    split = np.full(len(rows), 'train', dtype='<U5')
    split[folds[0][1]] = 'test'
    split[folds[1][1]] = 'val'
    for i, row in enumerate(rows):
        row['group'] = f'source_{groups[i]:04d}'
        row['split'] = str(split[i])
    for label in sorted(set(labels)):
        for s in ('train', 'val', 'test'):
            if not any((labels == label) & (split == s)):
                raise ValueError(f'Class {label} absent from {s}')
    with (out / 'manifest.csv').open('w', newline='', encoding='utf-8') as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0]))
        w.writeheader()
        w.writerows(rows)
    np.savez_compressed(out / 'features.npz', features=np.stack(features), color=np.stack(colors),
                        labels=labels, split=split, groups=groups, ids=np.array([r['id'] for r in rows]))
    audit = {'seed': 17, 'split_policy': 'StratifiedGroupKFold(5), test fold 0, validation fold 1, other folds
train',
             'upstream_files': len(paths), 'decoded_files': len(rows), 'errors': errors,
             'class_counts': dict(Counter(labels)), 'sources': len(sources), 'union_groups': len(set(groups)),
             'exact_duplicate_files': len(rows) - len(shas), 'cross_source_near_duplicates': near,
             'near_duplicate_rule': '64-bit pHash at first/middle/last sampled frame: max Hamming <=6 and mean
<=4',
             'split_counts': {s: dict(Counter(labels[split == s])) for s in ('train', 'val', 'test')},
             'split_groups': {s: len(set(groups[split == s])) for s in ('train', 'val', 'test')},
             'feature_shape': list(np.stack(features).shape), 'frame_sampling': '9 uniform samples, endpoints
included'}
    (out / 'audit.json').write_text(json.dumps(audit, indent=2), encoding='utf-8')
    print(json.dumps(audit, indent=2), flush=True)

if __name__ == '__main__':
    main()
```

scripts/prepare_demo_examples.py

scripts/prepare_demo_examples.py

```
"""Recreate the selected local examples without changing experiment evidence.

The public Git repository omits raw photographs and clips. Fetch the pinned
archives with --download, or reuse an existing local data directory. Download
requires requests (included in requirements-dev.txt). No training is performed.
"""
from pathlib import Path
import argparse
import csv
import json
import shutil
import subprocess
import sys

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
from cricketshot.images import load_image


def restore_images():
    with (ROOT / "data/images/processed/manifest.csv").open(encoding="utf-8", newline="") as stream:
        rows = {row["id"]: row for row in csv.DictReader(stream)}
    examples = json.loads((ROOT / "demo/image_examples.json").read_text())
    for entry in examples:
        row = rows[entry["id"]]
        if row["label"] != entry["label"] or row["split"] != entry["split"]:
            raise ValueError("Example metadata does not match the recorded manifest")
        source = ROOT / row["path"]
        if not source.is_file():
            raise FileNotFoundError("Raw image missing. Run this script with --download first.")
        target = ROOT / entry["path"]
        target.parent.mkdir(parents=True, exist_ok=True)
        load_image(source).save(target)
    return len(examples)


def restore_videos():
    from PIL import Image
    from cricketshot.core import decode_video
    with (ROOT / "data/processed/manifest.csv").open(encoding="utf-8", newline="") as stream:
        rows = {row["id"]: row for row in csv.DictReader(stream)}
    examples = json.loads((ROOT / "demo/examples.json").read_text())
    for entry in examples:
        source = ROOT / rows[entry["id"]]["path"]
        if not source.is_file():
            raise FileNotFoundError("Raw clip missing. Use --download --include-video.")
        target = ROOT / entry["path"]
        target.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(source, target)
        thumbnail = ROOT / entry["thumbnail"]
        thumbnail.parent.mkdir(parents=True, exist_ok=True)
        if not thumbnail.exists():
            frames, _ = decode_video(target)
            im = Image.fromarray(frames[len(frames)//2])
            im.thumbnail((320, 180))
            im.save(thumbnail, quality=85)
    return len(examples)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
```

scripts/prepare_demo_examples.py

```
    parser.add_argument("--download", action="store_true")
    parser.add_argument("--include-video", action="store_true")
    args = parser.parse_args()
    if args.download:
        subprocess.run([sys.executable, str(ROOT / "scripts/fetch_images.py")], cwd=ROOT, check=True)
        if args.include_video:
            subprocess.run([sys.executable, str(ROOT / "scripts/fetch_assets.py")], cwd=ROOT, check=True)
    print("Restored image examples:", restore_images())
    if args.include_video:
        print("Restored video examples:", restore_videos())


if __name__ == "__main__":
    main()
```

scripts/prepare_images.py

```
"""Audit, deduplicate, split and cache two-view features for real cricket images."""

from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
import csv
import hashlib
import json
from collections import Counter
import imagehash
import numpy as np
from sklearn.model_selection import StratifiedGroupKFold
from cricketshot.core import Encoder
from cricketshot.images import load_image, image_views


def main():
    out = ROOT / "data/images/processed"
    out.mkdir(parents=True, exist_ok=True)
    cache = out / "cache"
    cache.mkdir(exist_ok=True)
    encoder = Encoder(view="full")
    paths = sorted(
        p
        for p in (ROOT / "data/images/raw").rglob("*")
        if p.suffix.lower() in (".jpg", ".jpeg", ".png", ".webp", ".bmp")
    )
    rows = []
    features = []
    phashes = []
    thumbs = []
    errors = []
    aliases = {
        "drives": "drive",
        "drive": "drive",
        "legglance-flick": "legglance-flick",
        "pullshot": "pullshot",
        "sweep": "sweep",
    }
    seen = {}
    for n, path in enumerate(paths):
        label = aliases.get(path.parent.name.lower())
        if not label:
            continue
```

scripts/prepare_images.py

```
        try:
            image = load_image(path)
            sha = hashlib.sha256(
                image.size[0].to_bytes(4, "big")
                + image.size[1].to_bytes(4, "big")
                + image.tobytes()
            ).hexdigest()
            views = image_views(image)
            cp = cache / (sha + ".npy")
            if cp.exists():
                feat = np.load(cp, allow_pickle=False)
            else:
                feat = encoder.encode([np.asarray(v) for v in views])
                np.save(cp, feat, allow_pickle=False)
            rows.append(
                {
                    "id": f"image_{n:05d}",
                    "path": path.relative_to(ROOT).as_posix(),
                    "label": label,
                    "rgb_sha256": sha,
                    "width": image.width,
                    "height": image.height,
                }
            )
            features.append(feat)
            phashes.append(int(str(imagehash.phash(image)), 16))
            thumbs.append(np.asarray(image.resize((16, 16)), dtype=np.int16))
        except Exception as exc:
            errors.append({"path": str(path.relative_to(ROOT)), "error": str(exc)})
        if (n + 1) % 200 == 0:
            print(f"Image features {n+1}/{len(paths)}, valid {len(rows)}", flush=True)
    if not rows:
        raise ValueError("No labeled images found. Run fetch_images.py first.")
    parent = list(range(len(rows)))

    def find(i):
        while parent[i] != i:
            parent[i] = parent[parent[i]]
            i = parent[i]
        return i

    def union(i, j):
        parent[find(j)] = find(i)

    exact = 0
    near = []
    for i, row in enumerate(rows):
        if row["rgb_sha256"] in seen:
            union(i, seen[row["rgb_sha256"]])
            exact += 1
        else:
            seen[row["rgb_sha256"]] = i
    for i in range(len(rows)):
        for j in range(i):
            if (phashes[i] ^ phashes[j]).bit_count() <= 6:
                mad = float(np.abs(thumbs[i] - thumbs[j]).mean())
                if mad <= 18:
                    union(i, j)
                    if rows[i]["rgb_sha256"] != rows[j]["rgb_sha256"]:
                        near.append([i, j, mad])
    components = {}
    for i in range(len(rows)):
        components.setdefault(find(i), []).append(i)
    conflicts = [
```

scripts/prepare_images.py

```
        ii for ii in components.values() if len({rows[i]["label"] for i in ii}) > 1
    ]
    excluded = {i for ii in conflicts for i in ii}
    # Collapse decoded exact duplicates to one representative within each label.
    representatives = []
    hash_seen = set()
    for i, row in enumerate(rows):
        if i not in excluded and row["rgb_sha256"] not in hash_seen:
            hash_seen.add(row["rgb_sha256"])
            representatives.append(i)
    kept = [rows[i] for i in representatives]
    feats = np.stack([features[i] for i in representatives])
    labels = np.array([r["label"] for r in kept])
    groups = np.array([find(i) for i in representatives])
    folds = list(
        StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=604).split(
            np.zeros(len(kept)), labels, groups
        )
    )
    split = np.full(len(kept), "train", dtype="<U5")
    split[folds[0][1]] = "test"
    split[folds[1][1]] = "val"
    for i, row in enumerate(kept):
        row["group"] = f"visual_{groups[i]:05d}"
        row["split"] = str(split[i])
    with (out / "manifest.csv").open("w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=list(kept[0]))
        w.writeheader()
        w.writerows(kept)
    np.savez_compressed(
        out / "features.npz",
        features=feats,
        labels=labels,
        split=split,
        groups=groups,
        ids=np.array([r["id"] for r in kept]),
    )
    audit = {
        "files_found": len(paths),
        "decoded_labeled": len(rows),
        "errors": errors,
        "exact_duplicate_excess": exact,
        "near_duplicate_pairs": len(near),
        "conflicting_components": len(conflicts),
        "conflicting_images_excluded": len(excluded),
        "exact_representatives_kept": len(kept),
        "visual_groups": len(set(groups)),
        "class_counts": dict(Counter(labels)),
        "split_counts": {
            s: dict(Counter(labels[split == s])) for s in ("train", "val", "test")
        },
        "split_groups": {
            s: len(set(groups[split == s])) for s in ("train", "val", "test")
        },
        "split_seed": 604,
        "source_ids_available": False,
        "rule": "Exact decoded RGB hash or pHash <=6 and thumbnail RGB MAD <=18",
        "warning": "Visual groups do not establish match/player independence. Related frames may remain across
splits.",
    }
    (out / "audit.json").write_text(json.dumps(audit, indent=2), encoding="utf-8")
    print(json.dumps(audit, indent=2), flush=True)


```

scripts/prepare_images.py

```
if __name__ == "__main__":
    main()
```

scripts/prepare_revision_slides.py

```
"""Keep the 16-slide native deck, with a single course-grounded image narrative."""
from pathlib import Path
import json
ROOT=Path(__file__).resolve().parents[1]


def pct(x):return f"{100*x:.1f}%"
def block(text,x,y,w,h,size=20,bold=False):
    return dict(text=text,x=x,y=y,w=w,h=h,size=size,bold=bold)
def table(headers,rows,x=50,y=150,w=860,h=270,fontSize=17,widths=None,**extra):
    return dict(headers=headers,rows=rows,x=x,y=y,w=w,h=h,fontSize=fontSize,widths=widths,**extra)


def main():
    r=json.loads((ROOT/"results/images/metrics.json").read_text())
    c=json.loads((ROOT/"results/revision/comparison.json").read_text())
    h=json.loads((ROOT/"results/hog/metrics.json").read_text())
    a=json.loads((ROOT/"data/images/processed/audit.json").read_text())
    v=json.loads((ROOT/"results/metrics.json").read_text())
    old=json.loads((ROOT/"results/pilot_fullframe/metrics.json").read_text())
    sub=json.loads((ROOT/"configs/submission.json").read_text())
    team=[m["name"]+" ("+m["roll_number"]+")" for m in sub["members"]]
    team_text=team[0]+"  /  "+team[1]+"\n"+team[2]+"\n"+team[3]+"  /  "+team[4]
    names={"hog":"HOG","global":"CNN global","focus":"CNN crop","concat":"Concatenation","contrast":"Contrast"}
    labels={"drive":"Drive","legglance-flick":"Glance / flick","pullshot":"Pull","sweep":"Sweep"}
    slides=[]
    def add(title,**kwargs):slides.append(dict(title=title,**kwargs))
    add("Cricket Shot\nRecognition",kind="cover",kicker="EE604 COURSE PROJECT / 2026",
        subtitle="Gradient features and a frozen CNN\nA controlled four-class image study",
        image="slides/assets/class_0.jpg",team_text=team_text,bottom=sub["github_url"],
        notes="Introduce one-image classification. HOG and frozen features connect directly to lectures 10 and 15. The two-view difference is a small extension to test, not a new backbone. Cover image: selected held-out source image, https://www.kaggle.com/datasets/aneesh10/cricket-shot-dataset v2.")
    add("Problem, labels and constraints",subtitle="Input: one selected cricket image. Output: one of four dataset labels.",
        blocks=[block("Drive\nLeg glance / flick\nPull\nSweep",50,150,370,170,26,True),
                block("CPU inference, offline after setup.\nNo batter detection, contact localization or technique grading.\nUnknown shots can still receive confident scores.",50,350,850,105,20)],
        images=[dict(path="slides/assets/class_1.jpg",x=490,y=145,w=410,h=190)],
        notes="The user supplies the image. Drive and leg glance/flick are combined upstream classes. A full match is outside scope. Classification is multiclass with one target, not multilabel sigmoid prediction. Source: dataset v2; lectures 15 pages 16-19.")
    add("Course foundation and prior work",subtitle="Use established feature extraction, then test one modest extension",
        tables=[table(["Foundation","Source","Our use"],[
            ["Gradient histograms","Lecture 10; Dalal / Triggs (2005)","Explicit HOG baseline"],
            ["Frozen pretrained features","Lecture 15, pp. 11-14","MobileNetV2 + linear classifier"],
            ["Cricket recognition","KUCricShot (2023); Kang (2025)","Prior task studies; different data"],
            ["Two-view difference","Course implementation","Compare with simple concatenation"]],h=255,fontSize=16,widths=[230,330,300])],
        callout="No claim of a new backbone or an unprecedented combination. Novelty remains limited.",
        notes="Lecture 8 part 2 page 9 distinguishes local/global features; our crop is a regional descriptor, not a keypoint. Lecture 14 part 2 pp 2-3 and 27 cover extractor/classifier and softmax. MobileNetV2 is our architecture choice, not claimed taught by name. Sources: https://lear.inrialpes.fr/people/triggs/pubs/Dalal-cvpr05.pdf ; https://arxiv.org/abs/1801.04381 ; https://github.com/AtanuCSE/CricketShotDetection ; https://arxiv.org/abs/2510.09187 . Do not compare published percentages across datasets.")
    add("Images and fixed visual-group split",subtitle="4,724 original files; 4,723 exact representatives in 363
```

scripts/prepare_revision_slides.py

```
 visual groups",
        tables=[table(["Class","Train","Validation","Test"],[[labels[k]]+[a["split_counts"][s][k] for s in ("train","val","test")] for k in r["classes"]],h=245,widths=[320,180,180,180])],
        blocks=[block("221 training groups / 72 validation groups / 70 test groups\nNo shared group or decoded hash. Match and player identities remain unknown.",50,420,850,58,17)],
        notes="Data: https://www.kaggle.com/datasets/aneesh10/cricket-shot-dataset v2. Unknown license in downloaded metadata; no original annotations claimed. Fixed seed 604 folds; this is one holdout, not five-fold cross-validation. Test images are clustered copies and must not be described as 945 independent photographs.")
    add("Evaluation choices and their limits",subtitle="All five methods share images, labels, visual groups and split identities",
        tables=[table(["Choice","Data used","What remains fixed"],[
            ["Fit classifier coefficients","Training only","Frozen CNN; deterministic HOG"],
            ["Choose C: 0.1, 1, 10, 100","Validation macro F1","Smallest C wins ties"],
            ["Temperature and review","Original validation reuse","Keep original settings"],
            ["Final comparison","Same 945 test identities","No new split or seed search"]],h=265,fontSize=16,widths=[300,260,300])],
        callout="Exploratory: grouping changed after preliminary evaluation; HOG was added after old results were known.",
        notes="One validation set was reused for regularization, calibration and threshold selection. This can overfit validation. We disclose it rather than create a new split and pretend the original test was untouched. Initial hash-only contrast result was 96.3%; stronger transformed-copy grouping yields 65.5%. Both results and original individual predictions remain available.")
    add("Implemented pipeline",subtitle="Only the small cricket classifiers learn from the training images",
        images=[dict(path="results/revision/figures/pipeline.png",x=40,y=135,w=880,h=280)],
        callout="The fixed crop is a regional view. It does not detect or segment the batter.",
        notes="Explain the two branches. HOG and CNN global inputs have identical aspect-preserving 224 square geometry. CNN has two view passes; its global, crop and fused heads share features. HOG uses luminance, unsigned gradients, cells, blocks and per-image normalization. No StandardScaler or PCA learns from test. Source: actual implemented pipeline; lecture mapping in docs/course_mapping.md.")
    add("HOG: gradients, cells and blocks",subtitle="Lecture 10: accumulate gradient magnitude in orientation bins",
        tables=[table(["Step","Fixed setting"],[
            ["Input","224 x 224 luminance"],["Orientation votes","9 unsigned bins; linear bin weights"],
            ["Cells","16 x 16 pixels; 14 x 14 cells"],["Blocks","2 x 2 cells; stride 1 cell"],
            ["Normalize / concatenate","L2 blocks, then 6,084-value L2 vector"]],x=50,y=150,w=455,h=260,fontSize=15,widths=[165,290])],
        images=[dict(path="results/revision/figures/hog_steps.png",x=525,y=160,w=385,h=230)],
        callout="Votes use magnitude, not frequency. No learned feature scaling; no test-based HOG tuning.",
        notes="Trace gx and gy central differences, sqrt(gx²+gy²), atan2 modulo 180. A 50-degree gradient of magnitude 90 votes 45 to the 40-degree bin and 45 to 60. No spatial vote interpolation. Each 36-value block divides by sqrt(sum(h²)+1e-10). 13*13*36=6084. This is a documented HOG variant, not the exact signed-angle lecture example or original pedestrian detector. Lecture 10 pp 7-23; original method https://lear.inrialpes.fr/people/triggs/pubs/Dalal-cvpr05.pdf .")
    add("CNN views and the contrast feature",subtitle="Frozen ImageNet MobileNetV2 gives two 1,280-value pooled embeddings",
        tables=[table(["Representation","Normalized classifier input","Dimension"],[
            ["Global","u","1,280"],["Fixed regional crop","v","1,280"],["Concatenation","[u; v]","2,560"],["Contrast","[u; v; abs(u-v)]","3,840"]],h=235,fontSize=18,widths=[300,360,200])],
        blocks=[block("Absolute difference is nonlinear; signed difference would be redundant\nfor a linear head on concatenation. This motivates a test, not a success claim.",50,407,850,65,18)],
        notes="Both vectors are normalized before joining; the joined descriptor is normalized again. Contrast W has (4,3840) shape plus four biases. It learns weighted multiclass cross-entropy with L2 regularization. No bat-motion interpretation is assigned to feature coordinates. Actual inputs are displayed in the demo. MobileNetV2: https://arxiv.org/abs/1801.04381 ; ONNX weights: https://github.com/onnx/models .")
    result_rows=[]
    for mode in names:
        info=h if mode=="hog" else r["models"][mode]
        result_rows.append([names[mode],str(info["C"]),pct(info["validation"]["macro_f1"]),pct(info["test"]["accuracy"]),pct(info["test"]["macro_f1"])])
    add("Recognition results",subtitle="Same corrected split; every test image contributes, including review cases",
        tables=[table(["Feature","C","Val. F1","Test acc.","Test F1"],result_rows,h=265,fontSize=17,widths=[300,80,160,160,160])],
        callout="HOG is a controlled baseline. Pretrained features help here; contrast adds only three correct
```

scripts/prepare_revision_slides.py

```
images.",
        notes="HOG fixed settings, validation-selected C and predictions are saved under results/hog. Do not try a new descriptor after seeing the 35.3% test score to improve a headline. CNN heads/predictions are retained unchanged. This comparison does not isolate color versus pretraining versus architecture; no general statement that all HOG methods are inferior. Evidence: results/revision/comparison.csv and saved individual probabilities.")

interval_rows=[[names[k],pct(c["models"][k]["test"]["accuracy"]),f'{100*c["models"][k]["accuracy_95"][0]:.1f} to {100*c["models"][k]["accuracy_95"][1]:.1f}%'] for k in names]
    pair=c["paired_contrast_concat"]["accuracy_difference_95"]
    add("Uncertainty comes from 70 visual groups",subtitle="Whole-group bootstrap, 2,000 paired draws, fixed model and split",
        tables=[table(["Feature","Accuracy","Conditional 95% interval"],interval_rows,h=245,fontSize=17,widths=[300,180,380])],
        callout=f'Contrast minus concatenation: +0.32 points; interval {100*pair[0]:+.2f} to {100*pair[1]:+.2f}. No clear advantage.',
        notes="Draw groups with replacement and keep all images within each drawn group. Reuse draws between methods for paired differences. Statistics are image-weighted in each resample. Intervals omit training/split variability and unknown match correlation. They are not a hypothesis-test certificate of equality. Contrast 619/945, concatenation 616/945. Data and calculations: scripts/analyze_revision.py.")
    cm=r["models"]["contrast"]["test"]["confusion"]
    add("Class errors",subtitle="Contrast confusion counts; rows are dataset labels",
        tables=[table(["True / predicted","Drive","Glance/flick","Pull","Sweep"],[[labels[k]]+cm[i] for i,k in enumerate(r["classes"])],h=270,fontSize=19,widths=[260,150,150,150,150],heatmap=True,totals=[sum(row) for row in cm])],
        callout="Macro F1 averages the four class F1 scores equally. Review does not remove mistakes from this table.",
        notes="Read an off-diagonal entry and explain precision versus recall. Glance/flick is difficult under this protocol. HOG predicts pull for 110 of 224 sweep images. Fixed geometry, similar stances and background are plausible issues, not proven causes for each error. Evidence: saved prediction arithmetic in comparison.json.")
    model=r["models"]["contrast"]
    add("Scores, calibration and secondary review",subtitle="Temperature scaling worsened the recorded test calibration",
        tables=[table(["Contrast scores","NLL","ECE"],[[label,f'{model[key]["nll"]:.3f}',f'{model[key]["ece_10_bins"]:.3f}'] for label,key in [("Unscaled","test_uncalibrated"),("Scaled","test")]],x=50,y=160,w=420,h=165,fontSize=18,widths=[200,110,110])],
        blocks=[block("557 / 945 accepted\n433 accepted predictions correct\n58.9% coverage / 77.7% accuracy",510,165,400,150,25,True),block("Keep T = 0.802 and threshold = 0.70 unchanged.\nThe 90% validation target did not transfer to test.",50,370,850,65,20)],
        callout="Softmax is a relative class score. Agreement is not a guarantee or an unknown-shot detector.",
        notes="NLL and ECE worsen; do not say calibration improved. ECE uses ten equal-width confidence bins. The same validation set selected C, temperature and threshold. The gate needs global/crop/contrast agreement; the heads are correlated. HOG scores are unscaled and do not affect review. Sources: https://proceedings.mlr.press/v70/guo17a.html ; https://arxiv.org/abs/1705.08500 .")
    add("A confident error stays in the demo",subtitle="Selected after evaluation, alongside successful and disagreement examples",
        images=[dict(path="results/images/figures/failure.jpg",x=50,y=145,w=860,h=280)],
        callout="Inspect the upstream label, predicted label and actual input views. Confidence can be wrong.",
        notes="Use image_04199, the preserved confident failure. Do not invent its exact cause or silently relabel it. The selected picture is an illustration, not an estimate of failure frequency. Real data source: aneesh10/cricket-shot-dataset v2. Follow with the view-disagreement image_00014.")
    add("Local image demonstration",subtitle="Upload an image, compare CNN and HOG, inspect the review reason, export JSON",
        images=[dict(path="results/ui_image_prediction.png",x=50,y=135,w=860,h=325)],
        notes="Run setup.bat on a fresh machine, then run_demo.bat. Default URL http://127.0.0.1:6040. Revision verification may use 6060 to avoid another local service. Show success image_00210, confident error image_04199 and disagreement image_00014. Selected examples are honestly labeled. JSON preserves temperature-scaled score semantics. Screenshot must come from the revised running app, not a mockup.")
    add("What we can defend",subtitle="An implemented comparison with useful negative findings",
        blocks=[block("Course foundation",50,145,850,35,26,True),block("HOG and frozen features, each followed by a trained multiclass head.",50,193,850,50,21),
                block("Limited extension",50,275,850,35,26,True),block("Contrast gains three test images; uncertainty spans no gain.\nThe audit corrects optimism, and calibration fails to transfer.",50,323,850,85,21)],
        callout="Next useful step: independently labeled data with known match and player identities.",
```

scripts/prepare_revision_slides.py

```
        notes="Novelty remains a weakness, not a promise of marks. Do not claim a new architecture, detector or state-of-the-art result. Mention unknown upstream license, possible missed copies and unverified labels. Course mapping and viva guide contain actual page references and a tensor-shape trace. Team identities and the real repository URL are on the cover.")
    vm=v["models"]["phase_delta"];pm=old["models"]["phase_delta"]
    add("Backup: video pilot and duplicate audit",subtitle="Separate eight-class data and reused pilot holdout; not the main recognition claim",
        tables=[table(["Video model","Accuracy","Macro F1"],[["Full frame",pct(pm["test"]["accuracy"]),pct(pm["test"]["macro_f1"])],["Fixed crop",pct(vm["test"]["accuracy"]),pct(vm["test"]["macro_f1"])],["Majority baseline",pct(v["majority_baseline"]["accuracy"]),pct(v["majority_baseline"]["macro_f1"])]],y=150,h=205,fontSize=18,widths=[450,205,205])],
        blocks=[block("562 clips / 25 source groups / 116 test clips from five groups\nImage audit: hashes, transforms, ORB/RANSAC links; all connected copies stay together.",50,387,850,65,17)],
        notes="Use only for questions. Equal 30.2% majority accuracy means weak video recognition, despite higher macro F1. Fixed crop followed pilot test feedback. Source: https://www.kaggle.com/datasets/amittalmale/cricket-shots . Image audit: pHash <=6 bits with thumbnail MAD, rotations/mirrors, then 15 feature neighbors within cosine .20. ORB binary matching, .72 ratio, >=16 matches, >=12 RANSAC inliers, >=65% inlier fraction, >=8% hull coverage. These are evaluation tools, not a novel shot classifier. Other sources: https://github.com/onnx/models ; https://github.com/scikit-learn/scikit-learn ; https://github.com/microsoft/onnxruntime ; https://github.com/opencv/opencv ; https://arxiv.org/abs/1608.00859 . Full URLs, saved evidence and complete code appendix accompany the report.")
    assert len(slides)==16
    (ROOT/"slides/content.json").write_text(json.dumps({"slides":slides},indent=2),encoding="utf-8")
    text="# Revised presentation notes\n\nSlides 1-15 form the main talk; slide 16 is backup. Adjust timing to the assigned slot.\n\n"
    for i,slide in enumerate(slides,1):text+=f'## {i}. {slide["title"].replace(chr(10)," ")}\n\n{slide["notes"]}\n\n'
    (ROOT/"slides/speaker_notes.md").write_text(text,encoding="utf-8")
    print("Prepared 16 revised slides and speaker notes")


if __name__=="__main__":main()
```

scripts/prepare_slides.py

```
"""Compatibility entry point for the revised 16-slide course presentation."""
from prepare_revision_slides import main

if __name__ == "__main__":
    main()
```

scripts/record_environment.py

```
"""Record only the installed packages needed by this project, not unrelated tools."""
from pathlib import Path
import importlib.metadata
import json
import platform
import subprocess
ROOT=Path(__file__).resolve().parents[1]
packages=['numpy','Pillow','onnxruntime','opencv-python-headless','Flask','scikit-learn','scipy','threadpoolctl',
'onnx','ImageHash','matplotlib','requests','pytest','reportlab','PyMuPDF','blinker','click','itsdangerous',
          'Jinja2','MarkupSafe','Werkzeug','flatbuffers','protobuf','ml_dtypes','PyWavelets','joblib',
          'packaging','python-dateutil','six','contourpy','cycler','fonttools','kiwisolver','pyparsing',
          'certifi','charset-normalizer','idna','urllib3','Pygments','pluggy','iniconfig','colorama']
versions={p:importlib.metadata.version(p) for p in packages}
(ROOT/'requirements-lock.txt').write_text('# Exact versions used on Python '+platform.python_version()+' / Windows x64.\n'+
                                          '\n'.join(f'{p}=={v}' for p,v in sorted(versions.items(),key=lambda x:x[0].lower()))+'\n',encoding='utf-8')
```

scripts/record_environment.py

```
(ROOT/'results/environment.json').write_text(json.dumps({'python':platform.python_version(),'platform':platform.platform(),'packages':versions},indent=2),encoding='utf-8')
print('Recorded',len(versions),'package versions')
```

scripts/render_demo.ps1

```
param([int]$Port = 6040)
# Render the local project in isolated headless profiles for visual QA.
# No user browser profile, account or extension is accessed.
$ErrorActionPreference='Stop'
$workspace=Split-Path -Parent $PSScriptRoot
$chrome='C:/Program Files/Google/Chrome/Application/chrome.exe'
if(!(Test-Path -LiteralPath $chrome)){throw 'Install Chrome or edit this renderer path. The app itself does not require Chrome.'}
$examples=Get-Content -LiteralPath (Join-Path $workspace 'demo/image_examples.json') -Raw -Encoding UTF8 | ConvertFrom-Json
$first=$examples[0].id
$failure=($examples | Where-Object {$_.note -like 'Confident failure*'} | Select-Object -First 1).id
$runToken=[guid]::NewGuid().ToString('N').Substring(0,8)
$cases=@(
    @{name='ui_home';size='1440,1250';url="http://127.0.0.1:$Port/"},
    @{name='ui_image_prediction';size='1440,680';url="http://127.0.0.1:$Port/?example=$first&focus=1"},
    @{name='ui_failure';size='1440,680';url="http://127.0.0.1:$Port/?example=$failure&focus=1"},
    @{name='ui_narrow';size='600,1200';url="http://127.0.0.1:$Port/?example=$first&focus=1"}
)
foreach($case in $cases){
    $profile=Join-Path $workspace ('.build/headless-revised-'+$runToken+'-'+$case.name)
    $target=Join-Path $workspace ('results/'+$case.name+'.png')
    $log=Join-Path $workspace ('.build/'+$case.name+'.stderr.log')

    $arguments=@('--headless=new','--disable-gpu','--no-first-run','--no-default-browser-check','--hide-scrollbars',
                 ('--window-size='+$case.size),'--virtual-time-budget=12000',
                 ('--user-data-dir="'+$profile+'"'),('--screenshot="'+$target+'"'),('"'+$case.url+'"'))
    $started=Get-Date
    Start-Process -FilePath $chrome -ArgumentList $arguments -WindowStyle Hidden -RedirectStandardError $log
    # Office/Chrome process handles can be inaccessible from this host. Require
    # an actual newly written screenshot instead of treating process exit as proof.
    $fresh=$false
    $deadline=(Get-Date).AddSeconds(45)
    while((Get-Date) -lt $deadline){
        if((Test-Path -LiteralPath $target) -and ((Get-Item -LiteralPath $target).LastWriteTime -ge $started)){$fresh=$true;break}
        Start-Sleep -Milliseconds 250
    }
    if(!$fresh){throw ('No fresh screenshot was generated: '+$case.name)}
    if(!(Test-Path -LiteralPath $target)){throw ('Screenshot missing: '+$case.name)}
    Write-Output ('Rendered '+$case.name)
}
```

scripts/revision_figures.py

```
"""Create explanatory figures, a measured image trace and current demo timing."""
from pathlib import Path
import sys
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
import csv
import json
import time
import numpy as np
import matplotlib
matplotlib.use("Agg")
```

scripts/revision_figures.py

```
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
from cricketshot.core import prepare_frame
from cricketshot.images import ImageRecognizer, image_descriptor, image_views, load_image
from cricketshot.hog import HOGRecognizer, hog_descriptor


def main():
    out = ROOT / "results/revision"
    figures = out / "figures"
    figures.mkdir(parents=True, exist_ok=True)
    plt.rcParams.update({"font.size": 10, "font.family": "DejaVu Sans", "savefig.dpi": 180})
    fig, ax = plt.subplots(figsize=(12, 3.7))
    ax.set(xlim=(0, 12), ylim=(0, 3.7)); ax.axis("off")
    for y, labels, color in [(2.2, ["RGB image\n+ fixed regional crop", "Frozen MobileNetV2\ntwo 1,280-value vectors", "Normalize and combine\n[u; v; |u-v|]", "Trained linear head\nfour shot scores"], "#dfeadb"),
                             (.55, ["Same global input\n224 x 224 grayscale", "Gradients, cells\nand overlapping blocks", "Normalize HOG\n6,084 values", "Trained linear head\nfour shot scores"], "#f0e5d8")]:
        for i, label in enumerate(labels):
            x = .05 + i*3
            ax.add_patch(Rectangle((x,y),2.7,.9,facecolor=color,edgecolor="#485b48"))
            ax.text(x+1.35,y+.45,label,ha="center",va="center",fontsize=14)
            if i < 3: ax.annotate("",(x+2.98,y+.45),(x+2.72,y+.45),arrowprops={"arrowstyle":"->"})
    ax.text(.05,3.35,"CNN branch: one shared encoder, four controlled descriptor ablations",fontsize=13)
    ax.text(.05,.1,"HOG branch: deterministic features; identical labels, visual groups and train/validation/test split",fontsize=13)
    fig.tight_layout(pad=.4)
    for ext in ("pdf","png"):fig.savefig(figures/("pipeline."+ext),bbox_inches="tight")
    plt.close(fig)

    example = json.loads((ROOT/"demo/image_examples.json").read_text())[0]
    image = load_image(ROOT/example["path"])
    feature, gray, magnitude, cells = hog_descriptor(image, return_maps=True)
    fig, axes = plt.subplots(1,3,figsize=(6,2.2))
    axes[0].imshow(gray,cmap="gray",vmin=0,vmax=1);axes[0].set_title("Luminance",fontsize=14)
    axes[1].imshow(magnitude,cmap="gray");axes[1].set_title("Magnitude",fontsize=14)
    axes[2].imshow(gray,cmap="gray",alpha=.35,vmin=0,vmax=1)
    for yy in range(14):
        for xx in range(14):
            for b in range(9):
                angle=np.deg2rad(b*20);length=6*cells[yy,xx,b]/max(cells[yy,xx].max(),1e-12)
                cx,cy=xx*16+8,yy*16+8
                axes[2].plot([cx-length*np.cos(angle),cx+length*np.cos(angle)],
                             [cy-length*np.sin(angle),cy+length*np.sin(angle)],color="#284b30",lw=.7)
    axes[2].set_title("Cell bins",fontsize=14)
    for ax in axes:ax.axis("off")
    fig.tight_layout()
    fig.savefig(figures/"hog_steps.png",bbox_inches="tight");plt.close(fig)

    recognizer=ImageRecognizer();hog=HOGRecognizer()
    views=image_views(image)
    features=recognizer.encoder.encode([np.asarray(v) for v in views])
    z=image_descriptor(features,"contrast")
    logits=z@recognizer.heads["contrast_coef"].T+recognizer.heads["contrast_intercept"]
    result,_=recognizer.predict(ROOT/example["path"])
    trace={"id":example["id"],"dataset_label":example["label"],"selected_after_evaluation":True,
           "functions":["load_image","image_views","prepare_frame","Encoder.encode","image_descriptor","ImageRecognizer.predict_features"],
           "loaded_size_wh":list(image.size),"view_sizes_wh":[list(v.size) for v in views],
           "encoder_input_shape":list(prepare_frame(np.asarray(views[0]),"full").shape),
           "encoder_output_shapes":[o.shape for o in recognizer.encoder.session.get_outputs()],
           "two_view_features_shape":list(features.shape),"contrast_shape":list(z.shape),
           "head_weight_shape":list(recognizer.heads["contrast_coef"].shape),
           "contrast_l2_norm":float(np.linalg.norm(z)),"first_eight_contrast_values":z[:8].tolist(),
```

scripts/revision_figures.py

```
            "logits":logits.tolist(),"temperature":recognizer.config["temperatures"]["contrast"],
            "result":result,"hog_shape":list(feature.shape),"hog_l2_norm":float(np.linalg.norm(feature)),
            "hog_result":hog.predict(ROOT/example["path"])}
    (out/"worked_trace.json").write_text(json.dumps(trace,indent=2))
    rows=list(csv.DictReader((ROOT/"data/images/processed/manifest.csv").open(encoding="utf-8",newline="")))
    tests=[r for r in rows if r["split"]=="test"]
    timings=[]
    for index in np.linspace(0,len(tests)-1,50,dtype=int):
        path=ROOT/tests[index]["path"];start=time.perf_counter()
        recognizer.predict(path);hog.predict(path)
        timings.append(time.perf_counter()-start)
    timing={"n":50,"median_seconds":float(np.median(timings)),"p95_seconds":float(np.percentile(timings,95)),
            "times_seconds":timings,"scope":"CNN prediction plus HOG comparison, including separate image decodes; excludes model initialization, preview encoding and HTTP transport; warmed models"}
    (out/"latency.json").write_text(json.dumps(timing,indent=2))
    print("Created pipeline, HOG explanation, measured trace and current inference timing")


if __name__ == "__main__":main()
```

scripts/strengthen_image_groups.py

```
"""Audit transformed copies before the final image comparison.

Initial hash-only scores are preserved. The final grouping adds dihedral pHash
and geometric ORB verification of frozen-feature nearest-neighbor candidates.
The protocol revision follows visual QA, so final scores are exploratory.
"""

from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
import csv
import json
import shutil
import cv2
import imagehash
import numpy as np
from PIL import Image, ImageOps
from sklearn.model_selection import StratifiedGroupKFold
from sklearn.neighbors import NearestNeighbors
from threadpoolctl import threadpool_limits
from cricketshot.images import load_image
from cricketshot.core import normalize


def main():
    out = ROOT / "data/images/processed"
    initial = out / "initial_hash_audit"
    initial.mkdir(exist_ok=True)
    for name in ("manifest.csv", "audit.json", "features.npz"):
        if not (initial / name).exists():
            shutil.copy2(out / name, initial / name)
    evidence = ROOT / "results/images_initial_hash"
    evidence.mkdir(exist_ok=True)
    for name in (
        "metrics.json",
        "test_predictions.npz",
        "test_predictions.csv",
        "latency.json",
    ):
```

scripts/strengthen_image_groups.py

```
        p = ROOT / "results/images" / name
        if p.exists() and not (evidence / name).exists():
            shutil.copy2(p, evidence / name)
    rows = list(
        csv.DictReader((initial / "manifest.csv").open(encoding="utf-8", newline=""))
    )
    with np.load(initial / "features.npz", allow_pickle=False) as a:
        data = {k: a[k] for k in a.files}
    audit = json.loads((initial / "audit.json").read_text())
    parent = list(range(len(rows)))

    def find(i):
        while parent[i] != i:
            parent[i] = parent[parent[i]]
            i = parent[i]
        return i

    def union(i, j):
        parent[find(j)] = find(i)

    old_groups = {}
    for i, row in enumerate(rows):
        if row["group"] in old_groups:
            union(i, old_groups[row["group"]])
        else:
            old_groups[row["group"]] = i
    # ORB descriptors are matched only as an audit tool, never as label features.
    orb = cv2.ORB_create(nfeatures=400, edgeThreshold=5, patchSize=15, fastThreshold=7)
    hashes = []
    thumbs = []
    keypoints = []
    descriptors = []
    areas = []
    for i, row in enumerate(rows):
        im = load_image(ROOT / row["path"])
        im.thumbnail((384, 384))
        variants = []
        for rot in (0, 90, 180, 270):
            variant = im.rotate(rot, expand=True)
            variants.extend([variant, ImageOps.mirror(variant)])
        hashes.append([int(str(imagehash.phash(v)), 16) for v in variants])
        thumbs.append(
            [np.asarray(v.resize((16, 16)), dtype=np.int16) for v in variants]
        )
        gray = cv2.cvtColor(np.asarray(im), cv2.COLOR_RGB2GRAY)
        kp, desc = orb.detectAndCompute(gray, None)
        keypoints.append(np.array([k.pt for k in kp], dtype=np.float32))
        descriptors.append(desc)
        areas.append(im.width * im.height)
        if (i + 1) % 1000 == 0:
            print("Transform audit descriptors", i + 1, flush=True)
    rotation_edges = []
    for i in range(len(rows)):
        for j in range(i):
            if find(i) == find(j):
                continue
            # Transform i into the orientation of j. Eight variants cover quarter
            # turns and mirrors; arbitrary rotations rely on local-feature checks.
            for k, h in enumerate(hashes[i]):
                if (h ^ hashes[j][0]).bit_count() <= 6 and np.abs(
                    thumbs[i][k] - thumbs[j][0]
                ).mean() <= 24:
                    union(i, j)
                    rotation_edges.append((i, j))
```

scripts/strengthen_image_groups.py

```
                    break
    print("New transform hash edges", len(rotation_edges), flush=True)
    x = normalize(data["features"][:, 0, :])
    candidates = set()
    with threadpool_limits(limits=2):
        nn = NearestNeighbors(
            n_neighbors=16, metric="cosine", algorithm="brute", n_jobs=2
        ).fit(x)
        distances, neighbors = nn.kneighbors(x)
    for i, (dd, ii) in enumerate(zip(distances, neighbors)):
        for distance, j in zip(dd[1:], ii[1:]):
            if distance <= 0.20 and find(i) != find(int(j)):
                candidates.add(tuple(sorted((i, int(j)))))
    matcher = cv2.BFMatcher(cv2.NORM_HAMMING)
    orb_edges = []
    for n, (i, j) in enumerate(sorted(candidates)):
        if find(i) == find(j):
            continue
        d1, d2 = descriptors[i], descriptors[j]
        if d1 is None or d2 is None or min(len(d1), len(d2)) < 12:
            continue
        pairs = matcher.knnMatch(d1, d2, k=2)
        good = [
            pair[0]
            for pair in pairs
            if len(pair) == 2 and pair[0].distance < 0.72 * pair[1].distance
        ]
        if len(good) < 16:
            continue
        p1 = np.float32([keypoints[i][m.queryIdx] for m in good])
        p2 = np.float32([keypoints[j][m.trainIdx] for m in good])
        h, mask = cv2.findHomography(
            p1, p2, cv2.RANSAC, 3.0, maxIters=500, confidence=0.99
        )
        if h is None or mask is None:
            continue
        mask = mask.ravel().astype(bool)
        count = int(mask.sum())
        if count < 12 or mask.mean() < 0.65:
            continue
        cov1 = cv2.contourArea(cv2.convexHull(p1[mask])) / areas[i]
        cov2 = cv2.contourArea(cv2.convexHull(p2[mask])) / areas[j]
        if min(cov1, cov2) < 0.08:
            continue
        union(i, j)
        orb_edges.append(
            {
                "a": rows[i]["id"],
                "b": rows[j]["id"],
                "inliers": count,
                "ratio": float(mask.mean()),
                "coverage": [float(cov1), float(cov2)],
            }
        )
        if len(orb_edges) % 100 == 0:
            print("Verified geometric edges", len(orb_edges), flush=True)
    components = {}
    for i in range(len(rows)):
        components.setdefault(find(i), []).append(i)
    conflicts = [
        ii for ii in components.values() if len({rows[i]["label"] for i in ii}) > 1
    ]
    excluded = {i for ii in conflicts for i in ii}
    kept = [i for i in range(len(rows)) if i not in excluded]
```

scripts/strengthen_image_groups.py

```
    final_rows = [rows[i] for i in kept]
    labels = data["labels"][kept]
    groups = np.array([find(i) for i in kept])
    folds = list(
        StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=604).split(
            np.zeros(len(kept)), labels, groups
        )
    )
    split = np.full(len(kept), "train", dtype="<U5")
    split[folds[0][1]] = "test"
    split[folds[1][1]] = "val"
    from collections import Counter

    for i, row in enumerate(final_rows):
        row["group"] = f"photo_{groups[i]:05d}"
        row["split"] = str(split[i])
    with (out / "manifest.csv").open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=list(final_rows[0]))
        writer.writeheader()
        writer.writerows(final_rows)
    final_data = {k: v[kept] for k, v in data.items()}
    final_data["groups"] = groups
    final_data["split"] = split
    np.savez_compressed(out / "features.npz", **final_data)
    audit.update(
        {
            "initial_visual_groups": audit["visual_groups"],
            "visual_groups": len(set(groups)),
            "exact_representatives_kept": len(kept),
            "class_counts": dict(Counter(labels)),
            "transform_hash_edges": len(rotation_edges),
            "orb_verified_edges": len(orb_edges),
            "orb_candidates": len(candidates),
            "final_conflicting_components": len(conflicts),
            "final_conflicting_images_excluded": len(excluded),
            "split_counts": {
                s: dict(Counter(labels[split == s])) for s in ("train", "val", "test")
            },
            "split_groups": {
                s: len(set(groups[split == s])) for s in ("train", "val", "test")
            },
            "rule": "Original pHash/RGB components + quarter-turn/mirror pHash <=6 and MAD <=24 + ORB/RANSAC verified frozen-feature neighbors",
            "protocol_status": "Strengthened after initial image score and visual QA. Final comparison is exploratory under revised grouping.",
            "warning": "Transform and geometric grouping reduce further leakage but do not establish match or player independence.",
        }
    )
    (out / "audit.json").write_text(json.dumps(audit, indent=2), encoding="utf-8")
    edge_record = {
        "transform_edges": [[rows[i]["id"], rows[j]["id"]] for i, j in rotation_edges],
        "orb_edges": orb_edges,
        "excluded_ids": [rows[i]["id"] for i in sorted(excluded)],
    }
    (out / "transformed_duplicate_edges.json").write_text(
        json.dumps(edge_record, indent=2), encoding="utf-8"
    )
    print(json.dumps(audit, indent=2), flush=True)


if __name__ == "__main__":
    main()
```

scripts/train_evaluate.py

scripts/train_evaluate.py

```python
"""Fit classifier heads, freeze validation choices and save measured test evidence."""

from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
import csv
import hashlib
import json
import platform
import time
import warnings
import numpy as np
import sklearn
from PIL import Image, ImageEnhance, ImageFilter
from scipy.optimize import minimize_scalar
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    log_loss,
)
from threadpoolctl import threadpool_limits
from cricketshot.core import Encoder, decode_video, descriptor, softmax


def ece(y, p):
    confidence, correct = p.max(axis=1), p.argmax(axis=1) == y
    total = 0.0
    for lo, hi in zip(np.linspace(0, 1, 11)[:-1], np.linspace(0, 1, 11)[1:]):
        mask = (confidence > lo) & (confidence <= hi)
        if mask.any():
            total += mask.mean() * abs(correct[mask].mean() - confidence[mask].mean())
    return float(total)


def metrics(y, p, classes):
    pred = p.argmax(axis=1)
    return {
        "n": len(y),
        "accuracy": float(accuracy_score(y, pred)),
        "balanced_accuracy": float(balanced_accuracy_score(y, pred)),
        "macro_f1": float(
            f1_score(
                y,
                pred,
                labels=np.arange(len(classes)),
                average="macro",
                zero_division=0,
            )
        ),
        "nll": float(log_loss(y, p, labels=np.arange(len(classes)))),
        "ece_10_bins": ece(y, p),
        "brier_multiclass": float(
            np.mean(np.sum((p - np.eye(len(classes))[y]) ** 2, axis=1))
        ),
        "confusion": confusion_matrix(y, pred, labels=np.arange(len(classes))).tolist(),
```

scripts/train_evaluate.py

```
        "per_class": classification_report(
            y,
            pred,
            labels=np.arange(len(classes)),
            target_names=classes,
            output_dict=True,
            zero_division=0,
        ),
    }


def source_bootstrap(y, p, groups, classes, repeats=2000):
    rng = np.random.default_rng(17)
    unique = np.unique(groups)
    values = []
    for _ in range(repeats):
        drawn = rng.choice(unique, size=len(unique), replace=True)
        idx = np.concatenate([np.flatnonzero(groups == g) for g in drawn])
        values.append(
            [
                np.mean(p[idx].argmax(1) == y[idx]),
                f1_score(
                    y[idx],
                    p[idx].argmax(1),
                    labels=np.arange(len(classes)),
                    average="macro",
                    zero_division=0,
                ),
            ]
        )
    low, high = np.percentile(values, [2.5, 97.5], axis=0)
    return {
        "method": "Percentile cluster bootstrap over held-out source groups, 2000 draws, seed 17",
        "groups": len(unique),
        "accuracy_95": [float(low[0]), float(high[0])],
        "macro_f1_95": [float(low[1]), float(high[1])],
        "caution": "Few source groups. Intervals describe this test set and omit training variability.",
    }


def select_threshold(y, p):
    choices = []
    for threshold in np.arange(0.30, 0.951, 0.05):
        keep = p.max(axis=1) >= threshold
        if keep.sum() >= max(10, int(np.ceil(0.20 * len(y)))):
            acc = float(np.mean(p[keep].argmax(axis=1) == y[keep]))
            if acc >= 0.90:
                choices.append(
                    (float(threshold), float(keep.mean()), acc, int(keep.sum()))
                )
    if choices:
        th, coverage, acc, n = choices[0]
        return th, {
            "target_met": True,
            "validation_coverage": coverage,
            "validation_accuracy": acc,
            "validation_accepted": n,
        }
    th = 0.80
    keep = p.max(axis=1) >= th
    return th, {
        "target_met": False,
        "reason": "No candidate met 90% accuracy with at least 20% coverage and 10 clips. Fallback 0.80.",
        "validation_coverage": float(keep.mean()),
```

scripts/train_evaluate.py

```
        "validation_accepted": int(keep.sum()),
        "validation_accuracy": (
            float(np.mean(p[keep].argmax(1) == y[keep])) if keep.any() else None
        ),
    }


def main():
    import argparse

    parser = argparse.ArgumentParser()
    parser.add_argument("--view", choices=["full", "batter_roi"], default="batter_roi")
    parser.add_argument("--result-dir", default="results")
    parser.add_argument("--model-dir", default="models")
    args = parser.parse_args()
    cfg = json.loads((ROOT / "configs/experiment.json").read_text())
    cfg["view"] = args.view
    result_dir = ROOT / args.result_dir
    result_dir.mkdir(exist_ok=True, parents=True)
    model_dir = ROOT / args.model_dir
    model_dir.mkdir(exist_ok=True, parents=True)
    data_path = ROOT / (
        "data/processed/features_fullframe.npz"
        if args.view == "full"
        and (ROOT / "data/processed/features_fullframe.npz").exists()
        else "data/processed/features.npz"
    )
    with np.load(data_path, allow_pickle=False) as a:
        features, colors = a["features"], a["color"]
        labels, split, groups, ids = a["labels"], a["split"], a["groups"], a["ids"]
    classes = sorted(set(labels.tolist()))
    y = np.array([classes.index(s) for s in labels])
    tr, va, te = [split == s for s in ("train", "val", "test")]
    test_y = y[te]
    results = {
        "classes": classes,
        "seed": 17,
        "configuration": cfg,
        "models": {},
        "environment": {
            "python": platform.python_version(),
            "sklearn": sklearn.__version__,
            "platform": platform.platform(),
            "processor": platform.processor(),
        },
        "feature_file_sha256": hashlib.sha256(data_path.read_bytes()).hexdigest(),
    }
    predictions = {}
    coefficients = {}
    with threadpool_limits(limits=2):
        for mode in ["color"] + cfg["descriptors"]:
            t = time.perf_counter()
            z = colors if mode == "color" else descriptor(features, mode)
            trials, heads = [], []
            for c in cfg["C_candidates"]:
                head = LogisticRegression(
                    C=c, solver="lbfgs", class_weight="balanced", max_iter=2000
                )
                with warnings.catch_warnings(record=True) as caught:
                    warnings.simplefilter("always")
                    head.fit(z[tr], y[tr])
                val_f1 = f1_score(
                    y[va],
                    head.predict(z[va]),
```

scripts/train_evaluate.py

```
                    labels=np.arange(len(classes)),
                    average="macro",
                    zero_division=0,
                )
                trials.append(
                    {
                        "C": c,
                        "validation_macro_f1": float(val_f1),
                        "iterations": int(head.n_iter_.max()),
                        "warnings": [str(w.message) for w in caught],
                    }
                )
                heads.append(head)
            best = int(np.argmax([r["validation_macro_f1"] for r in trials]))
            head = heads[best]
            val_logits = head.decision_function(z[va])
            fit = minimize_scalar(
                lambda lt: log_loss(
                    y[va],
                    softmax(val_logits, np.exp(lt)),
                    labels=np.arange(len(classes)),
                ),
                bounds=np.log(cfg["temperature_bounds"]),
                method="bounded",
            )
            temp = float(np.exp(fit.x))
            val_p = softmax(val_logits, temp)
            test_logits = head.decision_function(z[te])
            uncalibrated, calibrated = softmax(test_logits), softmax(test_logits, temp)
            threshold, threshold_info = select_threshold(y[va], val_p)
            accepted = calibrated.max(1) >= threshold
            info = {
                "C": cfg["C_candidates"][best],
                "dimension": z.shape[1],
                "temperature": temp,
                "trials": trials,
                "validation": metrics(y[va], val_p, classes),
                "test": metrics(test_y, calibrated, classes),
                "test_uncalibrated": metrics(test_y, uncalibrated, classes),
                "review_threshold": threshold,
                "threshold_selection": threshold_info,
                "test_selective": {
                    "coverage": float(accepted.mean()),
                    "accepted": int(accepted.sum()),
                    "accuracy": (
                        float(
                            np.mean(calibrated[accepted].argmax(1) == test_y[accepted])
                        )
                        if accepted.any()
                        else None
                    ),
                },
                "fit_and_evaluate_seconds": time.perf_counter() - t,
            }
            if mode != "color":
                reversed_p = softmax(
                    descriptor(features[te, ::-1], mode) @ head.coef_.T
                    + head.intercept_,
                    temp,
                )
                info["temporal_reversal"] = metrics(test_y, reversed_p, classes)
                info["reversal_max_probability_change"] = float(
                    np.max(np.abs(reversed_p - calibrated))
                )
```

scripts/train_evaluate.py

```
254             results["models"][mode] = info
255             predictions[mode] = calibrated
256             coefficients[mode] = (head.coef_, head.intercept_)
257             print(
258                 mode,
259                 "C",
260                 info["C"],
261                 "val F1",
262                 round(info["validation"]["macro_f1"], 4),
263                 "test accuracy",
264                 round(info["test"]["accuracy"], 4),
265                 "test F1",
266                 round(info["test"]["macro_f1"], 4),
267                 flush=True,
268             )
269     majority = np.bincount(y[tr]).argmax()
270     results["majority_baseline"] = {
271         "class": classes[majority],
272         "accuracy": float(np.mean(test_y == majority)),
273         "macro_f1": float(
274             f1_score(
275                 test_y,
276                 np.full(len(test_y), majority),
277                 labels=np.arange(len(classes)),
278                 average="macro",
279                 zero_division=0,
280             )
281         ),
282     }
283     mode = cfg["deployment_descriptor"]
284     main_model = results["models"][mode]
285     main_model["bootstrap"] = source_bootstrap(
286         test_y, predictions[mode], groups[te], classes
287     )
288     coef, intercept = coefficients[mode]
289     np.savez_compressed(model_dir / "classifier.npz", coef=coef, intercept=intercept)
290     model_config = {
291         "mode": mode,
292         "view": cfg.get("view", "batter_roi"),
293         "classes": classes,
294         "temperature": main_model["temperature"],
295         "review_threshold": main_model["review_threshold"],
296         "C": main_model["C"],
297         "frames": 9,
298         "train_clips": int(tr.sum()),
299         "validation_clips": int(va.sum()),
300         "test_clips": int(te.sum()),
301         "threshold_selection": main_model["threshold_selection"],
302         "notes": "Review threshold is a validation heuristic, not a certified error bound or open-set
    detector.",
303     }
304     (model_dir / "classifier.json").write_text(
305         json.dumps(model_config, indent=2), encoding="utf-8"
306     )
307     # Corrupt test frames without fitting any parameter on the test set.
308     rows = list(
309         csv.DictReader(
310             (ROOT / "data/processed/manifest.csv").open(encoding="utf-8", newline="")
311         )
312     )
313     test_rows = [r for r in rows if r["split"] == "test"]
314     encoder = Encoder(view=args.view)
315     corrupt_features = {"dark": [], "blur": []}
316     for i, row in enumerate(test_rows):
```

scripts/train_evaluate.py

```
        frames, _ = decode_video(ROOT / row["path"])
        for name in corrupt_features:
            transformed = [
                (
                    np.asarray(ImageEnhance.Brightness(Image.fromarray(f)).enhance(0.6))
                    if name == "dark"
                    else np.asarray(
                        Image.fromarray(f).filter(ImageFilter.GaussianBlur(radius=2))
                    )
                )
                for f in frames
            ]
            corrupt_features[name].append(encoder.encode(transformed))
        if (i + 1) % 25 == 0:
            print(f"Robustness {i+1}/{len(test_rows)}", flush=True)
    results["robustness"] = {}
    for corruption, feats in corrupt_features.items():
        results["robustness"][corruption] = {}
        for name in ("middle", "mean", "phase_delta"):
            c, b = coefficients[name]
            pp = softmax(
                descriptor(np.stack(feats), name) @ c.T + b,
                results["models"][name]["temperature"],
            )
            results["robustness"][corruption][name] = metrics(test_y, pp, classes)
    np.savez_compressed(
        result_dir / "test_predictions.npz",
        y=test_y,
        ids=ids[te],
        groups=groups[te],
        **{f"p_{k}": v for k, v in predictions.items()},
    )
    with (result_dir / "test_predictions.csv").open(
        "w", encoding="utf-8", newline=""
    ) as f:
        w = csv.writer(f)
        w.writerow(
            ["id", "true", "predicted", "confidence", "accepted"]
            + [f"p_{c}" for c in classes]
        )
        for i, row in enumerate(test_rows):
            pp = predictions[mode][i]
            w.writerow(
                [
                    row["id"],
                    row["label"],
                    classes[pp.argmax()],
                    float(pp.max()),
                    bool(pp.max() >= main_model["review_threshold"]),
                ]
                + pp.tolist()
            )
    (result_dir / "metrics.json").write_text(
        json.dumps(results, indent=2), encoding="utf-8"
    )
    print("Saved measured evidence to results/metrics.json", flush=True)


if __name__ == "__main__":
    main()
```

scripts/train_hog.py

scripts/train_hog.py

```
"""One exploratory HOG comparison on the unchanged Option B image split."""
from pathlib import Path
import sys
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
import csv
import hashlib
import json
import time
import warnings
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from threadpoolctl import threadpool_limits
from cricketshot.hog import PARAMETERS, hog_descriptor
from cricketshot.images import load_image
from cricketshot.core import softmax
from train_evaluate import metrics


def file_hash(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def main():
    start = time.perf_counter()
    out = ROOT / "results/hog"
    out.mkdir(parents=True, exist_ok=True)
    folder = ROOT / "models/hog"
    folder.mkdir(parents=True, exist_ok=True)
    config = json.loads((ROOT / "configs/hog_experiment.json").read_text())
    manifest = ROOT / "data/images/processed/manifest.csv"
    rows = list(csv.DictReader(manifest.open(encoding="utf-8", newline="")))
    metadata = {"parameters": PARAMETERS, "manifest_sha256": file_hash(manifest),
                "implementation_sha256": file_hash(ROOT / "src/cricketshot/hog.py"),
                "protocol_sha256": file_hash(ROOT / "configs/hog_experiment.json")}
    # Preserve the intended protocol before extracting features or fitting heads.
    (out / "protocol_before_fit.json").write_text(json.dumps({**config, **metadata}, indent=2))
    cache = ROOT / "data/images/processed/hog_features.npz"
    cache_meta = cache.with_suffix(".json")
    if cache.exists() and cache_meta.exists() and json.loads(cache_meta.read_text()) == metadata:
        with np.load(cache, allow_pickle=False) as arrays:
            x = arrays["features"].copy()
            assert arrays["ids"].tolist() == [r["id"] for r in rows]
    else:
        features = []
        for i, row in enumerate(rows, 1):
            features.append(hog_descriptor(load_image(ROOT / row["path"])))
            if i % 500 == 0:
                print(f"HOG features {i}/{len(rows)}", flush=True)
        x = np.stack(features)
        np.savez_compressed(cache, features=x, ids=np.array([r["id"] for r in rows]))
        cache_meta.write_text(json.dumps(metadata, indent=2))
    classes = sorted({r["label"] for r in rows})
    y = np.array([classes.index(r["label"]) for r in rows])
    masks = {s: np.array([r["split"] == s for r in rows]) for s in ("train", "val", "test")}
    trials = []
    heads = []
    with threadpool_limits(limits=2):
        for c in config["C_candidates"]:
            head = LogisticRegression(C=c, class_weight="balanced", solver="lbfgs", max_iter=2000)
```

scripts/train_hog.py

```
            with warnings.catch_warnings(record=True) as caught:
                warnings.simplefilter("always")
                head.fit(x[masks["train"]], y[masks["train"]])
            score = f1_score(y[masks["val"]], head.predict(x[masks["val"]]), average="macro")
            trials.append({"C": c, "validation_macro_f1": float(score),
                           "iterations": int(head.n_iter_.max()),
                           "warnings": [str(w.message) for w in caught]})
            heads.append(head)
            print(f"HOG C={c}, validation macro F1={score:.6f}", flush=True)
        best = int(np.argmax([t["validation_macro_f1"] for t in trials]))
        head = heads[best]
        # Test is scored only after the validation choice is frozen.
        (out / "selection_before_test.json").write_text(json.dumps({"trials": trials, "selected_C":
trials[best]["C"]}, indent=2))
        probabilities = {s: softmax(head.decision_function(x[masks[s]])) for s in ("val", "test")}
    np.savez_compressed(folder / "head.npz", coef=head.coef_, intercept=head.intercept_)
    model_config = {"classes": classes, "parameters": PARAMETERS, "C": trials[best]["C"],
                    "metadata": metadata, "temperature": 1.0, "exploratory": True}
    (folder / "classifier.json").write_text(json.dumps(model_config, indent=2))
    result = {"classes": classes, "config": config, "metadata": metadata, "trials": trials,
              "C": trials[best]["C"], "dimension": x.shape[1],
              "validation": metrics(y[masks["val"]], probabilities["val"], classes),
              "test": metrics(y[masks["test"]], probabilities["test"], classes),
              "elapsed_seconds": time.perf_counter() - start}
    (out / "metrics.json").write_text(json.dumps(result, indent=2))
    test_rows = [r for r in rows if r["split"] == "test"]
    p = probabilities["test"]
    np.savez_compressed(out / "test_predictions.npz", y=y[masks["test"]], p=p,
                        ids=np.array([r["id"] for r in test_rows]),
                        groups=np.array([r["group"] for r in test_rows]))
    with (out / "test_predictions.csv").open("w", encoding="utf-8", newline="") as stream:
        writer = csv.writer(stream)
        writer.writerow(["id", "group", "true", "predicted"] + ["p_" + c for c in classes])
        for row, probability in zip(test_rows, p):
            writer.writerow([row["id"], row["group"], row["label"], classes[probability.argmax()]] +
probability.tolist())
    print(json.dumps({"C": result["C"], "accuracy": result["test"]["accuracy"],
                      "macro_f1": result["test"]["macro_f1"]}), flush=True)


if __name__ == "__main__":
    main()
```

scripts/train_images.py

```
"""Fit four image heads under the documented, revised visual-grouping protocol."""

from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
import csv
import json
import time
import warnings
import numpy as np
from PIL import Image, ImageEnhance, ImageFilter, ImageOps
from scipy.optimize import minimize_scalar
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, log_loss
from threadpoolctl import threadpool_limits
from cricketshot.core import Encoder, softmax
```

scripts/train_images.py

```
from cricketshot.images import load_image, image_views, image_descriptor
from train_evaluate import metrics, source_bootstrap


def main():
    cfg = json.loads((ROOT / "configs/image_experiment.json").read_text())
    out = ROOT / "results/images"
    out.mkdir(parents=True, exist_ok=True)
    model_dir = ROOT / "models/image"
    model_dir.mkdir(exist_ok=True)
    with np.load(ROOT / "data/images/processed/features.npz", allow_pickle=False) as a:
        features, labels, split, groups, ids = [
            a[k] for k in ("features", "labels", "split", "groups", "ids")
        ]
    classes = sorted(set(labels.tolist()))
    y = np.array([classes.index(c) for c in labels])
    tr, va, te = [split == s for s in ("train", "val", "test")]
    for s in ("train", "val", "test"):
        if set(labels[split == s]) != set(classes):
            raise ValueError("Class absent from " + s)
    results = {
        "classes": classes,
        "config": cfg,
        "models": {},
        "n": {s: int((split == s).sum()) for s in ("train", "val", "test")},
    }
    heads = {}
    preds = {}
    val_predictions = {}
    temperatures = {}
    with threadpool_limits(limits=2):
        for mode in cfg["modes"]:
            z = image_descriptor(features, mode)
            trials = []
            fitted = []
            for c in cfg["C_candidates"]:
                head = LogisticRegression(
                    C=c, class_weight="balanced", solver="lbfgs", max_iter=2000
                )
                with warnings.catch_warnings(record=True) as caught:
                    warnings.simplefilter("always")
                    head.fit(z[tr], y[tr])
                score = f1_score(y[va], head.predict(z[va]), average="macro")
                trials.append(
                    {
                        "C": c,
                        "validation_macro_f1": float(score),
                        "iterations": int(head.n_iter_.max()),
                        "warnings": [str(w.message) for w in caught],
                    }
                )
                fitted.append(head)
            best = int(np.argmax([t["validation_macro_f1"] for t in trials]))
            head = fitted[best]
            vl = head.decision_function(z[va])
            tl = head.decision_function(z[te])
            fit = minimize_scalar(
                lambda t: log_loss(y[va], softmax(vl, np.exp(t)), labels=np.arange(4)),
                bounds=np.log([0.1, 10]),
                method="bounded",
            )
            temp = float(np.exp(fit.x))
            p = softmax(tl, temp)
            v = softmax(vl, temp)
```

scripts/train_images.py

```
            heads[mode + "_coef"] = head.coef_
            heads[mode + "_intercept"] = head.intercept_
            preds[mode] = p
            val_predictions[mode] = v
            temperatures[mode] = temp
            results["models"][mode] = {
                "C": cfg["C_candidates"][best],
                "dimension": z.shape[1],
                "temperature": temp,
                "trials": trials,
                "validation": metrics(y[va], v, classes),
                "test": metrics(y[te], p, classes),
                "test_uncalibrated": metrics(y[te], softmax(tl), classes),
            }
            print(
                mode,
                "val F1",
                round(results["models"][mode]["validation"]["macro_f1"], 4),
                "test accuracy",
                round(results["models"][mode]["test"]["accuracy"], 4),
                "test F1",
                round(results["models"][mode]["test"]["macro_f1"], 4),
                flush=True,
            )
    vp = val_predictions["contrast"]
    agree_val = (
        val_predictions["global"].argmax(1) == val_predictions["focus"].argmax(1)
    ) & (val_predictions["global"].argmax(1) == vp.argmax(1))
    options = []
    for threshold in np.arange(0.5, 0.951, 0.05):
        keep = (vp.max(1) >= threshold) & agree_val
        if (
            keep.sum() >= max(20, np.ceil(0.2 * len(vp)))
            and np.mean(vp[keep].argmax(1) == y[va][keep]) >= 0.9
        ):
            options.append(float(threshold))
    threshold = options[0] if options else 0.8
    p = preds["contrast"]
    agreement = (preds["global"].argmax(1) == preds["focus"].argmax(1)) & (
        preds["global"].argmax(1) == p.argmax(1)
    )
    keep = (p.max(1) >= threshold) & agreement
    conf_only = p.max(1) >= threshold
    results["review"] = {
        "threshold": threshold,
        "target_met": bool(options),
        "validation_accepted": int(((vp.max(1) >= threshold) & agree_val).sum()),
        "test_accepted": int(keep.sum()),
        "test_coverage": float(keep.mean()),
        "test_accuracy": (
            float(np.mean(p[keep].argmax(1) == y[te][keep])) if keep.any() else None
        ),
        "agreement_fraction": float(agreement.mean()),
        "confidence_only_coverage": float(conf_only.mean()),
        "confidence_only_accuracy": (
            float(np.mean(p[conf_only].argmax(1) == y[te][conf_only]))
            if conf_only.any()
            else None
        ),
    }
    results["models"]["contrast"]["bootstrap"] = source_bootstrap(
        y[te], p, groups[te], classes
    )
    results["models"]["contrast"]["bootstrap"][
```

scripts/train_images.py

```
        "method"
    ] = "Percentile cluster bootstrap over visual duplicate groups, 2000 draws, seed 17"
    results["models"]["contrast"]["bootstrap"][
        "caution"
    ] = "Visual groups are not matches. Intervals omit training/split variability and may be optimistic if related frames remain across splits."
    majority = np.bincount(y[tr]).argmax()
    results["majority"] = {
        "class": classes[majority],
        "accuracy": float(np.mean(y[te] == majority)),
        "macro_f1": float(
            f1_score(
                y[te],
                np.full(te.sum(), majority),
                labels=np.arange(4),
                average="macro",
                zero_division=0,
            )
        ),
    }
    np.savez_compressed(model_dir / "heads.npz", **heads)
    config = {
        "classes": classes,
        "mode": "contrast",
        "temperatures": temperatures,
        "review_threshold": threshold,
        "review_rule": "confidence >= threshold AND global/focus/fused head top labels all agree",
        "review_target_met": bool(options),
        "crop_bounds": [0.15, 0.05, 0.85, 0.95],
        "note": "No guarantee of unknown-class rejection or match-independent accuracy.",
    }
    (model_dir / "classifier.json").write_text(
        json.dumps(config, indent=2), encoding="utf-8"
    )
    np.savez_compressed(
        out / "test_predictions.npz",
        y=y[te],
        ids=ids[te],
        groups=groups[te],
        agreement=agreement,
        accepted=keep,
        **{"p_" + k: v for k, v in preds.items()}
    )
    rows = list(
        csv.DictReader(
            (ROOT / "data/images/processed/manifest.csv").open(
                encoding="utf-8", newline=""
            )
        )
    )
    test_rows = [r for r in rows if r["split"] == "test"]
    encoder = Encoder(view="full")
    stress = {"dark": [], "blur": [], "flip": []}
    for i, row in enumerate(test_rows):
        im = load_image(ROOT / row["path"])
        for name in stress:
            changed = (
                ImageEnhance.Brightness(im).enhance(0.6)
                if name == "dark"
                else (
                    im.filter(ImageFilter.GaussianBlur(2))
                    if name == "blur"
                    else ImageOps.mirror(im)
                )
```

scripts/train_images.py

```
            )
            stress[name].append(
                encoder.encode([np.asarray(v) for v in image_views(changed)])
            )
        if (i + 1) % 200 == 0:
            print("Image stress tests", i + 1, "/", len(test_rows), flush=True)
    results["stress"] = {}
    for condition, feats in stress.items():
        results["stress"][condition] = {}
        for mode in ("global", "contrast"):
            pp = softmax(
                image_descriptor(np.stack(feats), mode) @ heads[mode + "_coef"].T
                + heads[mode + "_intercept"],
                temperatures[mode],
            )
            results["stress"][condition][mode] = metrics(y[te], pp, classes)
    (out / "metrics.json").write_text(json.dumps(results, indent=2), encoding="utf-8")
    with (out / "test_predictions.csv").open("w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(
            ["id", "true", "predicted", "confidence", "views_agree", "accepted"]
            + ["p_" + c for c in classes]
        )
        for i, row in enumerate(test_rows):
            writer.writerow(
                [
                    row["id"],
                    row["label"],
                    classes[p[i].argmax()],
                    float(p[i].max()),
                    bool(agreement[i]),
                    bool(keep[i]),
                ]
                + p[i].tolist()
            )
    print("Image evidence saved", flush=True)


if __name__ == "__main__":
    main()
```

scripts/verify_revision.py

```
"""Current-revision checks; keep earlier verification records explicitly separate."""
from pathlib import Path
import sys
ROOT=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(ROOT/"src"))
import csv
import hashlib
import json
import numpy as np
from cricketshot.core import Encoder
from cricketshot.images import load_image,image_views
from cricketshot.hog import PARAMETERS,HOGRecognizer
from analyze_revision import confusion_statistics


def digest(path):return hashlib.sha256(path.read_bytes()).hexdigest()


def main():
    out=ROOT/"results/revision";out.mkdir(exist_ok=True)
```

scripts/verify_revision.py

```
    rows=list(csv.DictReader((ROOT/"data/images/processed/manifest.csv").open(encoding="utf-8",newline="")))
    comparison=json.loads((out/"comparison.json").read_text())
    original=json.loads((ROOT/"results/images/metrics.json").read_text())
    checks=[]
    def check(name,condition,detail=""):
        checks.append({"name":name,"pass":bool(condition),"detail":detail})
        if not condition:raise AssertionError(name+": "+detail)
    for name in ("hog","global","focus","concat","contrast"):
        path=ROOT/("results/hog/test_predictions.npz" if name=="hog" else "results/images/test_predictions.npz")
        with np.load(path,allow_pickle=False) as arrays:
            p=arrays["p"] if name=="hog" else arrays["p_"+name]
            y=arrays["y"];cm=np.zeros((4,4),dtype=int)
            np.add.at(cm,(y,p.argmax(1)),1)
        computed=confusion_statistics(cm)
        recorded=comparison["models"][name]["test"]
        check(name+" independent confusion arithmetic",np.allclose(computed,[recorded["accuracy"],recorded["macro_f1"]],atol=1e-12))
    with np.load(ROOT/"results/images/test_predictions.npz",allow_pickle=False) as arrays:
        accepted=arrays["accepted"];correct=arrays["p_contrast"].argmax(1)==arrays["y"]
    check("Review exact counts preserved",int(accepted.sum())==557 and int((accepted&correct).sum())==433)
    pair=comparison["paired_contrast_concat"]
    check("Paired contrast gain is uncertain",pair["correct_difference"]==3 and pair["accuracy_difference_95"][0]<0<pair["accuracy_difference_95"][1])
    cal=original["models"]["contrast"]
    check("Calibration failure recorded",cal["test"]["ece_10_bins"]>cal["test_uncalibrated"]["ece_10_bins"] and cal["test"]["nll"]>cal["test_uncalibrated"]["nll"])
    check("Original threshold retained",abs(original["review"]["threshold"]-.70)<1e-12)
    model=HOGRecognizer()
    check("HOG preprocessing matches model",model.config["parameters"]==PARAMETERS)
    meta=json.loads((ROOT/"data/images/processed/hog_features.json").read_text())
    check("HOG cache binds manifest and implementation",meta["manifest_sha256"]==digest(ROOT/"data/images/processed/manifest.csv") and meta["implementation_sha256"]==digest(ROOT/"src/cricketshot/hog.py"))
    with np.load(ROOT/"data/images/processed/features.npz",allow_pickle=False) as arrays:
        features=arrays["features"].copy()
        check("CNN cache IDs match manifest",arrays["ids"].tolist()==[r["id"] for r in rows])
    encoder=Encoder(view="full");maximum_error=0.0
    for index in np.linspace(0,len(rows)-1,64,dtype=int):
        row=rows[index];image=load_image(ROOT/row["path"])
        pixel_hash=hashlib.sha256(image.width.to_bytes(4,"big")+image.height.to_bytes(4,"big")+image.tobytes()).hexdigest()
        check("Raw pixel hash "+row["id"],pixel_hash==row["rgb_sha256"])
        fresh=encoder.encode([np.asarray(v) for v in image_views(image)])
        error=float(np.max(np.abs(fresh-features[index])))
        maximum_error=max(maximum_error,error)
        check("Fresh two-view features "+row["id"],np.allclose(fresh,features[index],atol=1e-6))
    cache_proof={"manifest_sha256":digest(ROOT/"data/images/processed/manifest.csv"),
                 "feature_cache_sha256":digest(ROOT/"data/images/processed/features.npz"),
                 "encoder_sha256":digest(ROOT/"models/mobilenetv2_features.onnx"),
                 "images_module_sha256":digest(ROOT/"src/cricketshot/images.py"),
                 "core_module_sha256":digest(ROOT/"src/cricketshot/core.py"),
                 "fresh_images_checked":64,"two_views_per_image":True,"maximum_absolute_error":maximum_error,
                 "scope":"64 deterministic samples across all splits; not every cached image independently recomputed"}
    (out/"cache_validation.json").write_text(json.dumps(cache_proof,indent=2))
    # If the original sibling workspace exists, verify it was not modified.
    sibling=ROOT.parents[1]
    baseline_path=ROOT/".build/local_notes/docs/original_files_baseline.json"
    if baseline_path.exists() and (sibling/"app.py").exists() and (sibling/"revision").exists():
        baseline=json.loads(baseline_path.read_text())
        changed=[item["path"] for item in baseline if not (sibling/item["path"]).exists() or digest(sibling/item["path"])!=item["sha256"]]
        check("Original project files preserved",not changed,str(changed))
        preserved=["data/images/processed/manifest.csv","models/image/heads.npz","models/image/classifier.json",
```

scripts/verify_revision.py

```
77
    "results/images/metrics.json","results/images/test_predictions.npz","results/images_initial_hash/metrics.json",
78
    "results/images_initial_hash/test_predictions.npz","results/metrics.json","results/test_predictions.npz"]
79          for name in preserved:check("Preserved evidence "+name,digest(ROOT/name)==digest(sibling/name))
80      result={"all_checks_pass":True,"checks":checks,"cache_validation":cache_proof,
81              "scope":"Current revision: arithmetic, fixed evidence, preprocessing parity and original
    preservation. Pytest, HTTP, render and document checks have separate records."}
82      (out/"verification.json").write_text(json.dumps(result,indent=2))
83      print(f"{len(checks)} current revision checks passed; maximum CNN feature discrepancy {maximum_error:g}")
84
85
86  if __name__=="__main__":main()
```

scripts/verify_runtime.py

```
1   """Check repository-only inference without training data or development packages.
2
3   Uses Flask's request client and an actual multipart upload. With --image, the
4   input may be a local photograph outside the repository. Otherwise a generated
5   pattern checks wiring only; no recognition-quality claim follows from it.
6   """
7   from pathlib import Path
8   import argparse
9   import hashlib
10  import io
11  import json
12  import sys
13  import importlib.metadata
14  from PIL import Image, ImageDraw
15
16  ROOT = Path(__file__).resolve().parents[1]
17  sys.path.insert(0, str(ROOT))
18  from app import create_app
19
20
21  def main():
22      parser = argparse.ArgumentParser(description=__doc__)
23      parser.add_argument("--image", type=Path)
24      parser.add_argument("--output", type=Path)
25      args = parser.parse_args()
26      client = create_app().test_client()
27      home = client.get("/")
28      assert home.status_code == 200
29      sub = json.loads((ROOT/"configs/submission.json").read_text())
30      html = home.get_data(as_text=True)
31      assert all(m["name"] in html and m["roll_number"] in html for m in sub["members"])
32      assert sub["github_url"] in html
33      assert client.get("/video").status_code == 200
34      assert client.get("/api/status").json["ready"]
35      if args.image:
36          payload = args.image.read_bytes()
37          filename = args.image.name
38      else:
39          im = Image.new("RGB", (320, 240), "#448344")
40          ImageDraw.Draw(im).line((40, 210, 240, 30), fill="white", width=12)
41          stream = io.BytesIO()
42          im.save(stream, format="PNG")
43          payload, filename = stream.getvalue(), "generated_wiring_check.png"
44      response = client.post("/image-api/predict", data={"image": (io.BytesIO(payload), filename)})
45      assert response.status_code == 200, response.get_data(as_text=True)
46      result = response.json
47      for p in (result["probabilities"], result["hog_baseline"]["probabilities"]):
```

scripts/verify_runtime.py

```
        assert len(p) == 4 and abs(sum(p.values())-1) < 1e-8
    assert len(result["previews"]) == 2
    assert client.post("/image-api/predict", data={"image": (io.BytesIO(b"bad"), "bad.png")}).status_code == 400
    examples = client.get("/image-api/examples").json
    for entry in examples:
        assert client.get("/image-api/example/"+entry["id"]+"/thumbnail").status_code == 200
    paths = [ROOT/"app.py", ROOT/"configs/submission.json", Path(__file__).resolve()]
    for folder in ("src", "templates", "static", "models"):
        paths.extend(p for p in (ROOT/folder).rglob("*") if p.is_file() and p.suffix in {".py", ".html", ".js", ".css", ".json", ".npz", ".onnx"})
    proof = {"all_pass": True, "scope": "Repository runtime; Flask request client, actual multipart upload, CNN/HOG probabilities, previews, malformed-upload rejection, team and URL rendering. No new accuracy measurement.",
             "input": "external photograph" if args.image else "generated wiring pattern",
             "input_sha256": hashlib.sha256(payload).hexdigest(), "label": result["label"],
             "available_image_examples": len(examples), "python": sys.version,
             "packages": {p: importlib.metadata.version(p) for p in ["numpy", "Pillow", "onnxruntime", "opencv-python-headless", "Flask"]},
             "source_model_hashes": {p.relative_to(ROOT).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest() for p in sorted(set(paths))}}
    if args.output:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(json.dumps(proof, indent=2)+"\n", encoding="utf-8")
    print(json.dumps({k: v for k, v in proof.items() if k != "source_model_hashes"}, indent=2))


if __name__ == "__main__":
    main()
```

scripts/verify_submission.py

```
"""Independent arithmetic, source, package and submission-coverage audit."""
from pathlib import Path
import csv
import hashlib
import json
import subprocess
import sys
import zipfile
import xml.etree.ElementTree as ET
import numpy as np
import pymupdf as fitz
from sklearn.metrics import f1_score,confusion_matrix
ROOT=Path(__file__).resolve().parents[1]

def main():
    checks=[]
    def check(name,condition,detail=''):
        checks.append({'name':name,'pass':bool(condition),'detail':detail})
        if not condition:print('FAIL:',name,detail,flush=True)
    run=subprocess.run([sys.executable,'-m','pytest','-q'],cwd=ROOT,capture_output=True,text=True)
    check('Behavioral and integration tests',run.returncode==0,run.stdout.strip()+run.stderr.strip())
    r=json.loads((ROOT/'results/images/metrics.json').read_text())
    with np.load(ROOT/'results/images/test_predictions.npz',allow_pickle=False) as a:
        for mode,info in r['models'].items():
            p=a['p_'+mode];pred=p.argmax(1);y=a['y']
            check(mode+' metric arithmetic',abs(np.mean(pred==y)-info['test']['accuracy'])<1e-12 and
                  abs(f1_score(y,pred,average='macro')-info['test']['macro_f1'])<1e-12 and
                  np.array_equal(confusion_matrix(y,pred),info['test']['confusion']))
        check('Review arithmetic',int(a['accepted'].sum())==r['review']['test_accepted'])
    manifest=list(csv.DictReader((ROOT/'data/images/processed/manifest.csv').open(encoding='utf-8',newline='')))
    csv_predictions=list(csv.DictReader((ROOT/'results/images/test_predictions.csv').open(encoding='utf-8',newline='')))
```

scripts/verify_submission.py

```
    check('CSV prediction IDs and test manifest agree',{x['id'] for x in csv_predictions}=={x['id'] for x in manifest if x['split']=='test'})
    edges_path=ROOT/'data/images/processed/transformed_duplicate_edges.json'
    if edges_path.exists():
        edges=json.loads(edges_path.read_text());by_id={x['id']:x for x in manifest};bad=[]
        pairs=edges['transform_edges']+[[e['a'],e['b']] for e in edges['orb_edges']]
        for a,b in pairs:
            if a in by_id and b in by_id and by_id[a]['split']!=by_id[b]['split']:bad.append([a,b])
        check('Verified transformed copies stay in one split',not bad,f'{len(pairs)} recorded edges checked; {len(bad)} violations')
        original=list(csv.DictReader((ROOT/'data/images/processed/initial_hash_audit/manifest.csv').open(encoding='utf-8',newline='')))
        retained={row['id']:row for row in manifest};original_splits={}
        for row in original:
            if row['id'] in retained:original_splits.setdefault(row['group'],set()).add(retained[row['id']]['split'])
        check('Initial hash components remain unsplit',all(len(s)==1 for s in original_splits.values()),f'{len(original_splits)} original components checked')
    from PIL import Image,ImageOps
    mismatches=[]
    for i in np.linspace(0,len(manifest)-1,min(64,len(manifest)),dtype=int):
        row=manifest[i]
        with Image.open(ROOT/row['path']) as source:im=ImageOps.exif_transpose(source).convert('RGB')
        digest=hashlib.sha256(im.width.to_bytes(4,'big')+im.height.to_bytes(4,'big')+im.tobytes()).hexdigest()
        if digest!=row['rgb_sha256']:mismatches.append(row['id'])
    check('Raw pixel hashes independently sampled',not mismatches,f'64 deterministic source images re-read; {len(mismatches)} mismatches')
    inventory=json.loads((ROOT/'results/source_inventory.json').read_text())
    from build_report import source_files
    expected_sources={p.relative_to(ROOT).as_posix() for p in source_files()}
    check('Source inventory covers every current custom file',expected_sources=={item['path'] for item in inventory})
    stale=[]
    for item in inventory:
        p=ROOT/item['path']
        if not p.exists() or hashlib.sha256(p.read_bytes()).hexdigest()!=item['sha256']:stale.append(item['path'])
    check('Full code appendix matches current source',not stale,', '.join(stale) or f'{len(inventory)} source/config files match SHA-256 inventory')
    main=fitz.open(ROOT/'output/pdf/CricketShot_Report_Main.pdf');complete=fitz.open(ROOT/'output/pdf/CricketShot_Report_Complete.pdf');appendix=fitz.open(ROOT/'output/pdf/CricketShot_Code_Appendix.pdf')
    check('Complete report includes entire appendix',len(complete)==len(main)+len(appendix),f'{len(main)} main + {len(appendix)} appendix = {len(complete)} pages')
    text='\n'.join(p.get_text() for p in main)
    check('Report contains current primary score',f'{r["models"]["contrast"]["test"]["accuracy"]*100:.1f}' in text)
    plain=' '.join(text.split())
    required_sections=['Abstract','Introduction','Proposed method','Experiments','Code, data and repositories']
    check('Report covers required sections',all(s in plain for s in required_sections) and str(r['n']['test']) in plain,', '.join(required_sections))
    # Every authored file heading must appear in the printable appendix.
    appendix_text='\n'.join(p.get_text() for p in appendix)
    missing=[i['path'] for i in inventory if i['path'] not in appendix_text]
    check('Every inventoried file appears in source appendix',not missing,', '.join(missing))
    outside=[]
    for n,page in enumerate(complete):
        for word in page.get_text('words'):
            if word[0]<-1 or word[1]<-1 or word[2]>page.rect.width+1 or word[3]>page.rect.height+1:outside.append(n+1)
    check('PDF text stays inside page boundaries',not outside,str(sorted(set(outside))))
    pptx=ROOT/'output/presentation/CricketShot_Presentation.pptx'
    ns={'a':'http://schemas.openxmlformats.org/drawingml/2006/main'}
    with zipfile.ZipFile(pptx) as z:
        check('PowerPoint ZIP integrity',z.testzip() is None)
```

scripts/verify_submission.py

```
        slides=[n for n in z.namelist() if n.startswith('ppt/slides/slide') and n.endswith('.xml') and '/_rels/' not in n]
        check('Presentation slide count',len(slides)==16,str(len(slides)))
        tables=sum(len(ET.fromstring(z.read(n)).findall('.//a:tbl',ns)) for n in slides)
        check('Presentation evidence tables remain native',tables>=6,f'{tables} editable tables')
        slide_text=' '.join(' '.join(ET.fromstring(z.read(n)).itertext()) for n in slides)
        check('Presentation contains current primary score',f'{r["models"]["contrast"]["test"]["accuracy"]*100:.1f}%' in slide_text)
        notes=[n for n in z.namelist() if n.startswith('ppt/notesSlides/notesSlide') and n.endswith('.xml')]
        check('Speaker notes present',len(notes)==16,str(len(notes)))
    office=json.loads((ROOT/'results/presentation_validation.json').read_text(encoding='utf-8-sig'))
    check('PowerPoint renderer geometry',not office['issues'],str(office['issues']))
    with zipfile.ZipFile(ROOT/'output/CricketShot_Overleaf.zip') as z:
        check('Overleaf package contains actual 2022 style and source appendix',all(n in z.namelist() for n in ('main.tex','cvpr.sty','references.bib','code_appendix.pdf')))
    revision=json.loads((ROOT/'results/revision/verification.json').read_text())
    check('Current revision arithmetic/cache audit',revision['all_checks_pass'],f"{len(revision['checks'])} recorded checks; scope in results/revision/verification.json")
    proof=json.loads((ROOT/'results/revision/cache_validation.json').read_text())
    check('Current preprocessing binding',proof['images_module_sha256']==hashlib.sha256((ROOT/'src/cricketshot/images.py').read_bytes()).hexdigest() and proof['core_module_sha256']==hashlib.sha256((ROOT/'src/cricketshot/core.py').read_bytes()).hexdigest())
    http=json.loads((ROOT/'results/revision/http_validation.json').read_text())
    check('Actual upload HTTP flow',http['all_pass'] and all(hashlib.sha256((ROOT/path).read_bytes()).hexdigest()==sha for path,sha in http['source_model_hashes'].items()),'Four real examples and four actual multipart uploads on the revised running app; malformed upload rejected')
    hog=json.loads((ROOT/'results/hog/metrics.json').read_text())
    check('Report includes new HOG evidence',f"{hog['test']['accuracy']*100:.1f}" in text and f"{hog['test']['macro_f1']*100:.1f}" in text and 'HOG' in text)
    check('Slides include new HOG evidence',f"{hog['test']['accuracy']*100:.1f}%" in slide_text and 'HOG' in slide_text)
    check('Report discloses confidence failure',all(value in plain for value in ['0.043','0.094','557','433']))
    check('Course mapping and viva trace present',all((ROOT/path).is_file() for path in ['docs/course_mapping.md','docs/viva_guide.md','results/revision/worked_trace.json']))
    sub=json.loads((ROOT/'configs/submission.json').read_text())
    pending=[]
    if not sub.get('github_url'):pending.append('Project repository URL missing')
    if len(sub.get('members',[]))!=5:pending.append('Five member identities')
    check('Report contains all supplied names and roll numbers',all(m['name'] in plain and m['roll_number'] in plain for m in sub['members']))
    check('Slides contain all supplied names and roll numbers',all(m['name'] in slide_text and m['roll_number'] in slide_text for m in sub['members']))
    report_links={link.get('uri') for page in main for link in page.get_links()}
    check('Current repository URL in report and slides',sub['github_url'] in report_links and sub['github_url'] in slide_text)
    result={'checks':checks,'all_checks_pass':all(x['pass'] for x in checks),'pending_user_submission_items':pending,
            'revision_scope':'Current source, prediction arithmetic and document checks. Behavioral tests are rerun by this verifier.',
            'manual_review':'See docs/visual_review.md for actual render inspection. Automated checks alone do not prove visual quality or data-label correctness.',
            'limits':['No independent label audit of every raw image/clip','No image match/player identities','Exploratory revised grouping and reused video holdout','Unknown upstream image license']}
    (ROOT/'results/verification.json').write_text(json.dumps(result,indent=2),encoding='utf-8')
    md='# Final verification record\n\n'
    md+='| Check | Result | Evidence |\n|---|---|---|\n'
    for c in checks:md+=f'| {c["name"]} | {"PASS" if c["pass"] else "FAIL"} | {c["detail"].replace(chr(10)," ").replace("|","/")[:250]} |\n'
    md+='\nSubmission items deliberately pending:\n\n'+'\n'.join('- '+x for x in pending)+'\n\n'+result['manual_review']+'\n'
    (ROOT/'docs/verification.md').write_text(md,encoding='utf-8')
    print(f'{sum(c["pass"] for c in checks)}/{len(checks)} verification checks pass. Pending submission items: {len(pending)}')
```

scripts/verify_submission.py

```
    if not result['all_checks_pass']:raise SystemExit(1)

if __name__=='__main__':main()
```

setup.bat

```
@echo off
cd /d "%~dp0"
python -m venv .venv
if errorlevel 1 exit /b 1
".venv\Scripts\python.exe" -m pip install -r requirements.txt
if errorlevel 1 exit /b 1
echo Setup finished. Run run_demo.bat to open the local demo.
pause
```

src/cricketshot/__init__.py

```
"""Cricket shot recognition with frozen visual features and temporal pooling."""
__version__ = '1.0.0'
```

src/cricketshot/core.py

```
"""Shared decoding, feature extraction, temporal descriptors and safe inference.

Training and the demo import the same functions to avoid preprocessing drift.
The saved classifier is numeric NPZ/JSON, so inference never unpickles objects.
"""

from pathlib import Path
import json
import time
import cv2
import numpy as np
import onnxruntime as ort
from PIL import Image, ImageOps

ROOT = Path(__file__).resolve().parents[2]
N_FRAMES = 9
DISPLAY = {
    "cut": "Cut",
    "defence": "Defence",
    "glance": "Glance",
    "offdrive": "Off drive",
    "ondrive": "On drive",
    "pull": "Pull",
    "sweep": "Sweep",
    "unorthodox": "Unorthodox",
}


def decode_video(path, max_frames=1500):
    """Decode a trimmed clip in order, then sample nine evenly spaced frames.

    Sequential decoding avoids inaccurate frame seeking on short AVI files.
    Bounds protect the local demo from accidentally processing full matches.
    """
    cap = cv2.VideoCapture(str(path))
    if not cap.isOpened():
        raise ValueError("Cannot open this video. Use an AVI, MP4 or MOV clip.")
    fps = float(cap.get(cv2.CAP_PROP_FPS))
    frames = []
```

src/cricketshot/core.py

```
    try:
        while True:
            ok, bgr = cap.read()
            if not ok:
                break
            if len(frames) >= max_frames:
                raise ValueError(
                    "Clip is too long. Please trim it to at most 30 seconds."
                )
            if bgr.shape[0] * bgr.shape[1] > 3840 * 2160:
                raise ValueError("Video exceeds the 4K input limit.")
            rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
            # Retain bounded previews rather than full-resolution video frames.
            image = Image.fromarray(rgb)
            image.thumbnail((640, 480))
            frames.append(np.asarray(image))
    finally:
        cap.release()
    if not frames:
        raise ValueError("The file contains no decodable video frames.")
    duration = len(frames) / fps if fps > 0 else None
    if duration and duration > 30:
        raise ValueError("Clip is too long. Please trim it to at most 30 seconds.")
    indices = np.rint(np.linspace(0, len(frames) - 1, N_FRAMES)).astype(int)
    return [frames[i] for i in indices], {
        "decoded_frames": len(frames),
        "fps": fps,
        "duration_seconds": duration,
        "sample_indices": indices.tolist(),
    }


def crop_frame(rgb, view="batter_roi"):
    """Fixed broadcast-layout crop, not a learned person detector."""
    im = Image.fromarray(np.asarray(rgb, dtype=np.uint8)).convert("RGB")
    if view == "batter_roi":
        w, h = im.size
        im = im.crop(
            (round(0.30 * w), round(0.02 * h), round(0.70 * w), round(0.65 * h))
        )
    elif view != "full":
        raise ValueError("Unknown encoder view")
    return im


def prepare_frame(rgb, view="batter_roi"):
    """Resize with preserved aspect ratio, center crop and ImageNet normalize."""
    im = crop_frame(rgb, view)
    im = ImageOps.fit(im, (224, 224), method=Image.Resampling.BILINEAR)
    a = np.asarray(im, dtype=np.float32) / 255.0
    a = (a - np.array([0.485, 0.456, 0.406], np.float32)) / np.array(
        [0.229, 0.224, 0.225], np.float32
    )
    return np.ascontiguousarray(a.transpose(2, 0, 1)[None])


def normalize(x):
    """Per-vector L2 normalization; this learns no statistics from any split."""
    return x / np.maximum(np.linalg.norm(x, axis=-1, keepdims=True), 1e-12)


class Encoder:
    def __init__(self, model=None, view="batter_roi"):
        self.view = view
```

src/cricketshot/core.py

```
        model = Path(model or ROOT / "models/mobilenetv2_features.onnx")
        if not model.exists():
            raise FileNotFoundError(
                "Missing visual encoder. Run python scripts/prepare_data.py first."
            )
        options = ort.SessionOptions()
        options.intra_op_num_threads = 2
        options.inter_op_num_threads = 1
        self.session = ort.InferenceSession(
            str(model), sess_options=options, providers=["CPUExecutionProvider"]
        )
        self.input_name = self.session.get_inputs()[0].name

    def encode(self, frames):
        """Run frozen ImageNet parameters; no gradient or weight update occurs."""
        embeddings = []
        for frame in frames:
            tensor = prepare_frame(frame, self.view)  # (1, 3, 224, 224)
            output = self.session.run(None, {self.input_name: tensor})[0]
            embeddings.append(output.reshape(-1))  # pooled 1,280-value vector
        return np.stack(embeddings).astype(np.float32)


def descriptor(features, mode):
    """Order-sensitive compact representation. Accepts (..., 9, 1280)."""
    x = normalize(np.asarray(features, np.float32))
    if mode == "middle":
        z = x[..., 4, :]
    elif mode == "mean":
        z = x.mean(axis=-2)
    elif mode in ("phase", "phase_delta"):
        early, middle, late = [x[..., i : i + 3, :].mean(axis=-2) for i in (0, 3, 6)]
        parts = [early, middle, late]
        if mode == "phase_delta":
            parts += [late - early, x.std(axis=-2)]
        z = np.concatenate(parts, axis=-1)
    else:
        raise ValueError(f"Unknown descriptor: {mode}")
    return normalize(z).astype(np.float32)


def color_descriptor(frames):
    """A deliberately shallow baseline for background and color shortcuts."""
    hs = []
    for frame in frames:
        hsv = cv2.cvtColor(frame, cv2.COLOR_RGB2HSV)
        hist = cv2.calcHist([hsv], [0, 1], None, [12, 8], [0, 180, 0, 256]).reshape(-1)
        hs.append(hist / max(hist.sum(), 1))
    return normalize(np.mean(hs, axis=0)).astype(np.float32)


def softmax(logits, temperature=1.0):
    """Convert logits to relative class scores, stably by subtracting the max."""
    if temperature <= 0:
        raise ValueError("Temperature must be positive")
    z = np.asarray(logits, dtype=np.float64) / temperature
    z -= z.max(axis=-1, keepdims=True)
    p = np.exp(z)
    return p / p.sum(axis=-1, keepdims=True)


class Recognizer:
    def __init__(self, model_dir=None):
        self.model_dir = Path(model_dir or ROOT / "models")
```

src/cricketshot/core.py

```
        self.config = json.loads(
            (self.model_dir / "classifier.json").read_text(encoding="utf-8")
        )
        with np.load(self.model_dir / "classifier.npz", allow_pickle=False) as a:
            self.coef, self.intercept = a["coef"].copy(), a["intercept"].copy()
        self.encoder = Encoder(
            self.model_dir / "mobilenetv2_features.onnx",
            view=self.config.get("view", "batter_roi"),
        )

    def predict_features(self, features):
        z = descriptor(features, self.config["mode"])
        logits = z @ self.coef.T + self.intercept
        p = softmax(logits, self.config["temperature"])
        return p

    def predict(self, path):
        t = time.perf_counter()
        frames, meta = decode_video(path)
        features = self.encoder.encode(frames)
        p = self.predict_features(features)
        i = int(p.argmax())
        # This is a confidence-based review rule, not proof that an input is cricket.
        accepted = bool(p[i] >= self.config["review_threshold"])
        return {
            "label": self.config["classes"][i],
            "display_label": DISPLAY.get(
                self.config["classes"][i], self.config["classes"][i]
            ),
            "confidence": float(p[i]),
            "accepted": accepted,
            "decision": "Prediction" if accepted else "Needs review",
            "probabilities": {c: float(v) for c, v in zip(self.config["classes"], p)},
            "threshold": self.config["review_threshold"],
            "mode": self.config["mode"],
            "seconds": time.perf_counter() - t,
            "video": meta,
        }, frames
```

src/cricketshot/hog.py

```
"""A small, explicit HOG baseline for the Lecture 10 comparison.

This is a documented HOG variant, not a reimplementation claimed to reproduce
Dalal and Triggs exactly. Cells do not interpolate votes across cell boundaries.
Angles interpolate between two unsigned orientation bins. Blocks use L2 norm.
No parameter here is fitted to the dataset.
"""
from pathlib import Path
import json
import numpy as np
from PIL import Image, ImageOps
from .core import ROOT, normalize, softmax
from .images import image_views, load_image

PARAMETERS = {
    "image_size": [224, 224],
    "grayscale": "PIL RGB to L (8-bit luminance)",
    "gradient": "central difference, zero derivative at image boundaries",
    "orientations": 9,
    "angle_range_degrees": [0, 180],
    "pixels_per_cell": [16, 16],
    "cells_per_block": [2, 2],
```

src/cricketshot/hog.py

```
    "block_stride_cells": 1,
    "orientation_interpolation": "linear between circular neighboring bins",
    "spatial_interpolation": False,
    "block_normalization": "L2, epsilon=1e-5",
    "final_normalization": "L2 per image, epsilon floor=1e-12",
    "dimension": 6084,
}


def prepare_hog(image):
    """Use the same global-view geometry as the CNN; then discard color."""
    global_view, _ = image_views(image)
    square = ImageOps.fit(global_view, (224, 224), method=Image.Resampling.BILINEAR)
    return np.asarray(square.convert("L"), dtype=np.float32) / 255.0


def gradients(gray):
    """Return magnitude and unsigned angle for each grayscale pixel."""
    gx = np.zeros_like(gray)
    gy = np.zeros_like(gray)
    gx[:, 1:-1] = gray[:, 2:] - gray[:, :-2]
    gy[1:-1, :] = gray[2:, :] - gray[:-2, :]
    magnitude = np.hypot(gx, gy)
    angle = np.mod(np.degrees(np.arctan2(gy, gx)), 180.0)
    return magnitude, angle


def cell_histograms(magnitude, angle, cell_size=16, bins=9):
    """Accumulate magnitude, not pixel frequency, into orientation bins.

    For a direction between two bin centers, split its magnitude in proportion
    to distance. The final bin wraps around to zero because angles are unsigned.
    """
    height, width = magnitude.shape
    if height % cell_size or width % cell_size:
        raise ValueError("Image dimensions must be multiples of cell size")
    yy, xx = np.indices((height, width))
    cell_id = (yy // cell_size) * (width // cell_size) + xx // cell_size
    position = angle * (bins / 180.0)
    lower = np.floor(position).astype(int) % bins
    fraction = position - np.floor(position)
    upper = (lower + 1) % bins
    count = (height // cell_size) * (width // cell_size) * bins
    low_votes = np.bincount((cell_id * bins + lower).ravel(),
                            weights=(magnitude * (1 - fraction)).ravel(), minlength=count)
    high_votes = np.bincount((cell_id * bins + upper).ravel(),
                             weights=(magnitude * fraction).ravel(), minlength=count)
    return (low_votes + high_votes).reshape(height // cell_size, width // cell_size, bins)


def hog_descriptor(image, return_maps=False):
    """224² -> 14x14 cells -> 13x13 overlapping blocks -> 6,084 values."""
    gray = prepare_hog(image)
    magnitude, angle = gradients(gray)
    cells = cell_histograms(magnitude, angle)
    windows = np.lib.stride_tricks.sliding_window_view(cells, (2, 2), axis=(0, 1))
    # Arrange each block as four neighboring cells, with nine bins per cell.
    blocks = windows.transpose(0, 1, 3, 4, 2).reshape(13, 13, 36)
    blocks = blocks / np.sqrt(np.sum(blocks ** 2, axis=-1, keepdims=True) + 1e-10)
    feature = normalize(blocks.reshape(-1)).astype(np.float32)
    if return_maps:
        return feature, gray, magnitude, cells
    return feature

```

src/cricketshot/hog.py

```
class HOGRecognizer:
    """Read numeric weights only; share extraction with the training script."""
    def __init__(self, model_dir=None):
        folder = Path(model_dir or ROOT / "models/hog")
        self.config = json.loads((folder / "classifier.json").read_text(encoding="utf-8"))
        if self.config["parameters"] != PARAMETERS:
            raise ValueError("HOG model preprocessing does not match this implementation")
        with np.load(folder / "head.npz", allow_pickle=False) as arrays:
            self.coef = arrays["coef"].copy()
            self.intercept = arrays["intercept"].copy()

    def predict_features(self, feature):
        return softmax(np.asarray(feature) @ self.coef.T + self.intercept)

    def predict(self, path):
        feature = hog_descriptor(load_image(path))
        probabilities = self.predict_features(feature)
        index = int(probabilities.argmax())
        return {"label": self.config["classes"][index],
                "score": float(probabilities[index]),
                "probabilities": dict(zip(self.config["classes"], map(float, probabilities))),
                "note": "Exploratory HOG baseline; uncalibrated softmax scores."}
```

src/cricketshot/images.py

```
"""Four-class still-image recognition with dual-view contrast and review gating."""

from pathlib import Path
import json
import time
import numpy as np
from PIL import Image, ImageOps, UnidentifiedImageError
from .core import Encoder, normalize, softmax, ROOT

LABELS = {
    "drive": "Drive",
    "legglance-flick": "Leg glance / flick",
    "pullshot": "Pull",
    "sweep": "Sweep",
}


def load_image(path):
    try:
        with Image.open(path) as source:
            if source.width * source.height > 24_000_000:
                raise ValueError("Use an image below 24 megapixels.")
            source.load()
            image = ImageOps.exif_transpose(source).convert("RGB")
    except (UnidentifiedImageError, OSError) as exc:
        raise ValueError(
            "Cannot read this image. Use a valid JPEG, PNG or WebP file."
        ) from exc
    if min(image.size) < 32:
        raise ValueError("Image is too small. Use at least 32 pixels on each side.")
    return image


def image_views(image):
    """Two fixed views. Central crop is a context test, not an actor detector."""
    image = image.copy()
    image.thumbnail((1024, 1024))
```

src/cricketshot/images.py

```
    w, h = image.size
    focus = image.crop(
        (round(0.15 * w), round(0.05 * h), round(0.85 * w), round(0.95 * h))
    )
    return image, focus


def image_descriptor(features, mode):
    """Compare the two frozen 1,280-value embeddings without training the CNN.

    Normalize each view first so its vector length cannot dominate fusion.
    The absolute difference adds a non-linear relationship to a linear head.
    It is a regional-view comparison, not a keypoint or bat-motion descriptor.
    """
    x = normalize(np.asarray(features, np.float32))
    if mode == "global":
        z = x[..., 0, :]
    elif mode == "focus":
        z = x[..., 1, :]
    elif mode == "concat":
        z = np.concatenate([x[..., 0, :], x[..., 1, :]], axis=-1)
    elif mode == "contrast":
        z = np.concatenate(
            [x[..., 0, :], x[..., 1, :], np.abs(x[..., 0, :] - x[..., 1, :])], axis=-1
        )
    else:
        raise ValueError("Unknown image descriptor")
    return normalize(z).astype(np.float32)


class ImageRecognizer:
    def __init__(self, model_dir=None):
        self.dir = Path(model_dir or ROOT / "models/image")
        self.config = json.loads(
            (self.dir / "classifier.json").read_text(encoding="utf-8")
        )
        with np.load(self.dir / "heads.npz", allow_pickle=False) as a:
            self.heads = {k: a[k].copy() for k in a.files}
        self.encoder = Encoder(ROOT / "models/mobilenetv2_features.onnx", view="full")

    def predict_features(self, features, mode="contrast"):
        """Apply learned cricket weights, then the unchanged validation temperature."""
        z = image_descriptor(features, mode)
        return softmax(
            z @ self.heads[mode + "_coef"].T + self.heads[mode + "_intercept"],
            self.config["temperatures"][mode],
        )

    def predict(self, path):
        t = time.perf_counter()
        image = load_image(path)
        views = image_views(image)
        features = self.encoder.encode([np.asarray(v) for v in views])
        p = self.predict_features(features)
        pg, pf = [self.predict_features(features, m) for m in ("global", "focus")]
        agree = bool(pg.argmax() == pf.argmax())
        consistent = bool(agree and pg.argmax() == p.argmax())
        ci = int(p.argmax())
        threshold = self.config["review_threshold"]
        accepted = bool(p.max() >= threshold and consistent)
        return {
            "label": self.config["classes"][ci],
            "display_label": LABELS[self.config["classes"][ci]],
            "score_type": "temperature-scaled softmax",
```

src/cricketshot/images.py

```
            "score_note": "A relative score among four supported labels, not a guarantee of correctness. Test calibration worsened after temperature scaling.",
            "confidence": float(p.max()),
            "accepted": accepted,
            "decision": "Prediction" if accepted else "Needs review",
            "probabilities": dict(zip(self.config["classes"], map(float, p))),
            "views_agree": agree,
            "prediction_consistent": consistent,
            "global_label": LABELS[self.config["classes"][int(pg.argmax())]],
            "focus_label": LABELS[self.config["classes"][int(pf.argmax())]],
            "threshold": threshold,
            "seconds": time.perf_counter() - t,
            "input_size": list(image.size),
            "scope": "Four-class still-image classification. Unknown classes can receive confident scores.",
        }, views
```

static/app.js

```
'use strict';
const $ = (id) => document.getElementById(id);
const names = {cut:'Cut',defence:'Defence',glance:'Glance',offdrive:'Off drive',ondrive:'On drive',pull:'Pull',sweep:'Sweep',unorthodox:'Unorthodox'};
let selectedFile = null, lastResult = null, busy = false;
function setFile(file) {
  selectedFile = file;
  $('file-label').textContent = file ? file.name : 'Drop your video here';
  $('analyze-button').disabled = !file || busy;
}
$('video-input').addEventListener('change', e => setFile(e.target.files[0]));
$('dropzone').addEventListener('dragover', e => {e.preventDefault(); $('dropzone').classList.add('drag');});
$('dropzone').addEventListener('dragleave', () => $('dropzone').classList.remove('drag'));
$('dropzone').addEventListener('drop', e => {e.preventDefault(); $('dropzone').classList.remove('drag'); setFile(e.dataTransfer.files[0]);});
function showState(state) {
  for (const name of ['empty-state','loading-state','error-state','prediction']) $(name).hidden = name !== state;
}
async function analyze(url, options) {
  if (busy) return;
  busy = true; $('analyze-button').disabled = true;
  document.querySelectorAll('.example').forEach(b => b.disabled = true);
  showState('loading-state');
  try {
    const response = await fetch(url, options);
    const data = await response.json();
    if (!response.ok) throw new Error(data.error || 'Prediction failed. Please retry.');
    lastResult = data; renderResult(data); showState('prediction');
  } catch (error) {
    $('error-state').textContent = error.message; showState('error-state');
  } finally {
    busy = false; $('analyze-button').disabled = !selectedFile;
    document.querySelectorAll('.example').forEach(b => b.disabled = false);
  }
}
$('upload-form').addEventListener('submit', e => {
  e.preventDefault(); if (!selectedFile) return;
  const body = new FormData(); body.append('video', selectedFile);
  analyze('/api/predict', {method:'POST',body});
});
function renderResult(data) {
  $('decision').textContent = data.decision.toUpperCase();
  $('shot-label').textContent = data.display_label;
  $('confidence').textContent = `${(data.confidence*100).toFixed(1)}%`;
```

static/app.js

```
  $('prediction-context').textContent = data.example ? `Test example: ${names[data.example.label]}. ${data.example.note}` : 'Prediction for your uploaded clip';
  $('review-note').textContent = data.accepted ? 'Score exceeds the review threshold. Check the clip and the alternatives before relying on it.' : 'Score is below the review threshold. The label above is the top candidate and needs human review.';
  $('review-note').className = `review-note${data.accepted?'':' warn'}`;
  const scores = $('scores'); scores.replaceChildren();
  for (const [label,p] of Object.entries(data.probabilities).sort((a,b)=>b[1]-a[1])) {
    const row=document.createElement('div'); row.className=`class-row${label===data.label?' top':''}`;
    const line=document.createElement('div'); line.className='class-label';
    const a=document.createElement('span'); a.textContent=names[label]||label;
    const b=document.createElement('span'); b.textContent=`${(p*100).toFixed(1)}%`;
    line.append(a,b);
    const track=document.createElement('div'); track.className='class-track';
    const fill=document.createElement('div'); fill.className='class-fill'; fill.style.width=`${p*100}%`;
    track.append(fill); row.append(line,track); scores.append(row);
  }
  $('timing').textContent = `${data.seconds.toFixed(2)} s on CPU`;
  const strip=$('phase-strip'); strip.replaceChildren();
  ['Early','Middle','Late'].forEach((name,phase) => {
    const group=document.createElement('div'); group.className='phase';
    const imgs=document.createElement('div'); imgs.className='phase-images';
    for(let i=0;i<3;i++) {const im=document.createElement('img'); im.src=data.previews[phase*3+i]; im.alt=`${name} sample ${i+1}`; imgs.append(im);}
    const caption=document.createElement('p'); caption.textContent=name; group.append(imgs,caption); strip.append(group);
  });
}
$('download-result').addEventListener('click', () => {
  if(!lastResult) return;
  const {previews,...exported}=lastResult;
  const url=URL.createObjectURL(new Blob([JSON.stringify(exported,null,2)],{type:'application/json'}));
  const a=document.createElement('a'); a.href=url; a.download='cricketshot-prediction.json'; a.click(); setTimeout(()=>URL.revokeObjectURL(url),1000);
});
async function initialize() {
  try {
    const [examplesResponse,evidenceResponse]=await Promise.all([fetch('/api/examples'),fetch('/api/evidence')]);
    const examples=await examplesResponse.json();
    for(const e of examples) {
      const button=document.createElement('button'); button.className='example'; button.type='button';
      button.setAttribute('aria-label',`Analyze ${names[e.label]} test example`);
      const img=document.createElement('img'); img.src=`/api/example/${e.id}/thumbnail`; img.alt=`${names[e.label]} sample`;
      const caption=document.createElement('span'); caption.textContent=names[e.label]; button.append(img,caption);
      button.addEventListener('click',()=> {if(busy)return; document.querySelectorAll('.example').forEach(b=>b.classList.remove('selected')); button.classList.add('selected'); analyze(`/api/example/${e.id}`,{method:'POST'});});
      $('examples').append(button);
    }
    if(evidenceResponse.ok) {
      const evidence=await evidenceResponse.json(), m=evidence.models.phase_delta;
      const items=[[`${(m.test.accuracy*100).toFixed(1)}%`,'Test accuracy'],[`${(m.test.macro_f1*100).toFixed(1)}%`,'Macro F1'],[String(m.test.n),'Test clips']];
      for(const [value,label] of items) {const box=document.createElement('div');box.className='metric';const n=document.createElement('strong');n.textContent=value;const l=document.createElement('span');l.textContent=label;box.append(n,l);$('metrics').append(box);}
    }
  }catch(error){$('examples').textContent='Examples unavailable. Check the setup instructions.';}
}
initialize();
```

static/images.css

static/images.css

```
.header-right a{color:#4c7042;text-underline-offset:4px;margin-right:20px}.image-views{display:grid;grid-template-columns:1fr 1fr;gap:16px;margin-top:12px}.model-view{display:flex;align-items:center;gap:10px}.model-view img{height:100px;width:100px;object-fit:contain;background:#edf1e7}.model-view p{font-size:10px;line-height:1.6;color:#77866e}.model-view strong{display:block;font-size:11px;color:#3c633a;font-weight:500}.prediction-heading h3{font-size:30px}.examples{grid-template-columns:repeat(4,1fr)}.example img{height:63px}.example span{font-size:8px}.scores{margin:20px 0}.hero-fact{margin-right:0;padding-left:35px}.hero-fact p{font-size:12px}@media(max-width:480px){.model-view{display:block}.model-view img{width:100%;height:110px}.header-right a{margin-right:3px}.header-right{gap:5px}}
.focus-demo header,.focus-demo .hero,.focus-demo .evidence,.focus-demo .method,.focus-demo footer{display:none}.focus-demo main{padding:28px;max-width:none}.focus-demo .workspace{min-height:600px}.focus-demo .model-view img{width:140px;height:140px}
.input-section,.model-view{min-width:0}.image-views{grid-template-columns:repeat(2,minmax(0,1fr))}.prediction-heading>div:first-child{min-width:0}.score{flex-shrink:0}@media(max-width:850px){.workspace{grid-template-columns:minmax(0,1fr)}.input-section,.result-section{padding:24px}.focus-demo main{padding:16px}.focus-demo .model-view img{width:100%;height:120px}.example-heading>span{font-size:8px}.prediction-heading h3{font-size:26px}.score strong{font-size:31px}}
.examples{grid-template-columns:repeat(4,minmax(0,1fr))}.example{min-width:0}

footer{gap:24px;align-items:flex-start;line-height:1.8}footer>span:first-child{max-width:780px}footer a{color:inherit}
```

static/images.js

```
'use strict';
const $=id=>document.getElementById(id);
const labels={'drive':'Drive','legglance-flick':'Leg glance / flick','pullshot':'Pull','sweep':'Sweep'};
let file=null,busy=false,result=null;
function choose(f){file=f;$('file-label').textContent=f?f.name:'Drop a cricket image here';$('analyze-button').disabled=!f||busy;}
$('image-input').addEventListener('change',e=>choose(e.target.files[0]));
$('dropzone').addEventListener('dragover',e=>{e.preventDefault();$('dropzone').classList.add('drag');});
$('dropzone').addEventListener('dragleave',()=>$('dropzone').classList.remove('drag'));
$('dropzone').addEventListener('drop',e=>{e.preventDefault();$('dropzone').classList.remove('drag');choose(e.dataTransfer.files[0]);});
function state(active){for(const id of ['empty-state','loading-state','error-state','prediction'])$(id).hidden=id!==active;}
async function predict(url,options){
  if(busy)return;busy=true;$('analyze-button').disabled=true;document.querySelectorAll('.example').forEach(b=>b.disabled=true);state('loading-state');
  try{const response=await fetch(url,options),data=await response.json();if(!response.ok)throw new Error(data.error||'Recognition failed.');result=data;render(data);state('prediction');}
  catch(error){$('error-state').textContent=error.message;state('error-state');}
  finally{busy=false;$('analyze-button').disabled=!file;document.querySelectorAll('.example').forEach(b=>b.disabled=false);}
}
$('image-form').addEventListener('submit',e=>{e.preventDefault();if(!file)return;const body=new FormData();body.append('image',file);predict('/image-api/predict',{method:'POST',body});});
function render(data){
  $('decision').textContent=data.decision.toUpperCase();$('shot-label').textContent=data.display_label;$('confidence').textContent=`${(data.confidence*100).toFixed(1)}%`;
  $('prediction-context').textContent=data.example?`Dataset label: ${labels[data.example.label]}. ${data.example.note}`:'Prediction for your image';
  const reason=!data.prediction_consistent?'The separate heads do not all agree with this prediction. Human review is needed.':data.confidence<data.threshold?'The score is below the review threshold. Human review is needed.':'All heads agree and the score exceeds the review threshold. A confident answer can still be wrong.';
  $('review-note').textContent=reason;$('review-note').className=`review-note${data.accepted?'':' warn'}`;
  const scores=$('scores');scores.replaceChildren();
  for(const [label,p] of Object.entries(data.probabilities).sort((a,b)=>b[1]-a[1])){const
```

static/images.js

```
row=document.createElement('div');row.className=`class-row${label===data.label?' top':''}`;const
line=document.createElement('div');line.className='class-label';const
name=document.createElement('span');name.textContent=labels[label];const
val=document.createElement('span');val.textContent=`${(p*100).toFixed(1)}%`;line.append(name,val);const
track=document.createElement('div');track.className='class-track';const fill=document.createElement('div');fill.
className='class-fill';fill.style.width=`${p*100}%`;track.append(fill);row.append(line,track);scores.append(row)
;}
  const h=data.hog_baseline; $('hog-result').textContent=h?`HOG baseline: ${labels[h.label]}
(${(100*h.score).toFixed(1)}% uncalibrated score). This uses grayscale gradient histograms. Neither model score
is a guarantee of correctness.`:'';
  $('timing').textContent=`${(data.seconds*1000).toFixed(0)} ms on CPU`;
  const views=$('image-views');views.replaceChildren();
  ['Global view','Central crop'].forEach((name,i)=>{const
box=document.createElement('div');box.className='model-view';const
image=document.createElement('img');image.src=data.previews[i];image.alt=name;const
p=document.createElement('p');const strong=document.createElement('strong');strong.textContent=name;p.append(str
ong,document.createTextNode(i===0?data.global_label:data.focus_label));box.append(image,p);views.append(box);});
  $('view-agreement').textContent='The global and crop heads are separate diagnostic classifiers. Their
agreement controls the review decision. Displayed pixels match the model crops before normalization.';
}
$('download-result').addEventListener('click',()=>{if(!result)return;const {previews,...data}=result;const
url=URL.createObjectURL(new Blob([JSON.stringify(data,null,2)],{type:'application/json'}));const a=document.crea
teElement('a');a.href=url;a.download='cricketshot-image-prediction.json';a.click();setTimeout(()=>URL.revokeObje
ctURL(url),1000);});
async function init(){try{const [er,mr]=await
Promise.all([fetch('/image-api/examples'),fetch('/image-api/evidence')]);const examples=await
er.json();if(!examples.length)$('examples').textContent='Upload your own image, or restore selected examples
using scripts/prepare_demo_examples.py (see README).';for(const e of examples){const
b=document.createElement('button');b.className='example';b.type='button';b.setAttribute('aria-label',`Analyze
${labels[e.label]} example`);const
im=document.createElement('img');im.src=`/image-api/example/${e.id}/thumbnail`;im.alt=labels[e.label];const t=do
cument.createElement('span');t.textContent=e.short_label||labels[e.label];b.append(im,t);b.addEventListener('cli
ck',()=>{if(busy)return;document.querySelectorAll('.example').forEach(x=>x.classList.remove('selected'));b.class
List.add('selected');predict(`/image-api/example/${e.id}`,{method:'POST'});});$('examples').append(b);}if(mr.ok)
{const r=await mr.json(),m=r.models.contrast.test;for(const [value,label] of
[[`${(m.accuracy*100).toFixed(1)}%`,'Test accuracy'],[`${(m.macro_f1*100).toFixed(1)}%`,'Macro
F1'],[String(m.n),'Test images']]){const box=document.createElement('div');box.className='metric';const
strong=document.createElement('strong');strong.textContent=value;const span=document.createElement('span');span.
textContent=label;box.append(strong,span);$('metrics').append(box);}}const params=new URLSearchParams(location.s
earch),example=params.get('example');if(params.get('focus')==='1')document.body.classList.add('focus-demo');if(e
xamples.some(e=>e.id===example))await predict(`/image-api/example/${encodeURIComponent(example)}`,{method:'POST'
});}catch(error){$('examples').textContent='Examples unavailable. See README.md for setup.';}}
init();
```

static/style.css

```
@font-face{font-family:system;src:local('Segoe
UI')}*{box-sizing:border-box}body{margin:0;background:#f6f7f2;color:#182c25;font-family:system,'Segoe
UI',Arial,sans-serif}header{height:88px;border-bottom:1px solid
#dfe4d9;display:flex;align-items:center;justify-content:space-between;padding:0 5.8%}.wordmark{font-size:25px;fo
nt-weight:750;letter-spacing:-1px;text-decoration:none;color:#182c25;display:flex;align-items:center}.wordmark>s
pan:first-child{font-weight:400}.wordmark-dot{display:block;width:8px;height:8px;background:#c26644;margin:12px
0 0 5px;border-radius:50%}.header-right{display:flex;align-items:center;gap:8px;font-size:12px;color:#5b6c61}.lo
cal-dot{width:6px;height:6px;background:#547c47;border-radius:50%}.course{margin-left:38px;font-size:11px;letter
-spacing:1px}main{max-width:1400px;margin:0 auto;padding:0
5.8%}.hero{display:flex;justify-content:space-between;align-items:center;padding:60px 0
48px}.eyebrow{font-size:10px;letter-spacing:2px;font-weight:700;color:#5e775d;margin:0 0
18px}h1{font-size:64px;letter-spacing:-3px;line-height:1.05;font-weight:550;margin:0 0 24px}h1 em{font-family:Ge
orgia,serif;font-weight:400;color:#60825c}.intro{font-size:15px;line-height:1.75;color:#667368;max-width:470px;m
argin:0}.hero-fact{border-left:1px solid #ced7c9;padding-left:45px;margin-right:30px}.fact-number{font-family:Ge
orgia,serif;font-size:88px;line-height:1}.fact-number
span{font-size:15px;font-family:system,Arial;margin-left:12px;color:#6f7e6b}.hero-fact
p{font-size:13px}.muted{color:#7b877a;line-height:1.7}.workspace{display:grid;grid-template-columns:42%
```

static/style.css

```
58%;background:#fff;border:1px solid #dfe4d9;min-height:560px}.input-section{padding:28px 30px;border-right:1px solid #dfe4d9}.result-section{padding:28px 32px;min-width:0}.section-title{display:flex;align-items:center;gap:12px;margin-bottom:25px}.step{font-size:10px;color:#899481;border:1px solid #dbe1d3;border-radius:50%;width:27px;height:27px;display:grid;place-content:center}h2{font-size:19px;letter-spacing:-.5px;font-weight:600;margin:0}.dropzone{display:flex;flex-direction:column;align-items:center;justify-content:center;min-height:170px;border:1px dashed #b7c9ad;background:#f9fbf5;cursor:pointer;gap:7px;text-align:center}.dropzone.drag{background:#eaf1df}.upload-symbol{color:#79936a;font-size:32px;font-weight:200}.dropzone strong{font-size:13px}.dropzone span:not(.upload-symbol){font-size:12px;color:#73836d}.dropzone small{font-size:9px;color:#8a9584;margin-top:10px}.primary-button{width:100%;background:#244f38;color:#fff;border:0;padding:15px 18px;font:600 12px system,Arial;cursor:pointer;margin-top:12px;display:flex;justify-content:space-between}.primary-button:disabled{background:#d5dfcd;cursor:default;color:#65795c}.primary-button:hover:enabled{background:#326d4b}.input-note{font-size:10px;line-height:1.65;color:#8a9486;margin:12px 0 24px}.example-heading{display:flex;justify-content:space-between;align-items:center;margin-bottom:10px}.example-heading h3{font-size:12px;font-weight:600;margin:0}.example-heading >span{font-size:9px;color:#88947f}.examples{display:grid;grid-template-columns:repeat(4,1fr);gap:7px}.example{padding:0;background:transparent;border:1px solid #e0e6da;cursor:pointer;text-align:left;color:#355235}.example:hover,.example.selected{outline:2px solid #7d9c67;outline-offset:1px}.example img{width:100%;height:55px;object-fit:cover;display:block}.example span{display:block;padding:6px 4px;font-size:9px}.empty-state{text-align:center;padding:60px 0 40px}.empty-nine{font:100px Georgia;color:#e1e9d8;line-height:1}.empty-state h3{font-size:17px;font-weight:500;margin:18px 0 10px}.empty-state p,.loading-state p{font-size:12px;line-height:1.8;color:#99a090}.loading-state{text-align:center;padding:110px 0}.loading-state h3{font-size:18px;font-weight:500}.spinner{border:2px solid #e1e8d7;border-top-color:#547d43;border-radius:50%;width:30px;height:30px;margin:0 auto 20px;animation:spin .7s linear infinite}@keyframes spin{to{transform:rotate(360deg)}}.error-state{padding:20px;background:#fff1e9;color:#9d4a28;font-size:13px;line-height:1.6}.prediction-heading{display:flex;justify-content:space-between;align-items:center}.prediction-heading .eyebrow{margin:0 0 8px}.prediction-heading h3{font-size:35px;font-weight:550;letter-spacing:-1px;margin:0}.score{display:flex;flex-direction:column;align-items:end}.score strong{font:38px Georgia;color:#48733c}.score span{font-size:9px;color:#939d8b}.prediction-context{font-size:11px;color:#7b8a71;margin:12px 0}.review-note{font-size:11px;line-height:1.6;background:#f0f4e9;padding:10px;color:#607650}.review-note.warn{background:#fff2df;color:#927038}.scores{display:grid;grid-template-columns:1fr 1fr;gap:8px 16px;margin:18px 0}.class-row{font-size:10px}.class-label{display:flex;justify-content:space-between;margin-bottom:4px;color:#788771}.class-track{height:3px;background:#edf1e7}.class-fill{height:100%;background:#adc19a}.class-row.top .class-fill{background:#3a7147}.class-row.top .class-label{color:#2d5c37;font-weight:650}.frames-heading{display:flex;justify-content:space-between;align-items:center;border-top:1px solid #e5eadd;padding-top:15px;margin-top:20px}.frames-heading h4{font-size:12px;margin:0}.frames-heading span{font-size:9px;color:#8a9681}.phase-strip{display:grid;grid-template-columns:repeat(3,1fr);gap:10px;margin-top:12px}.phase{min-width:0}.phase-images{display:grid;grid-template-columns:repeat(3,1fr);gap:2px}.phase img{width:100%;height:48px;object-fit:cover}.phase p{font-size:9px;color:#78896d;margin:6px 0}.frame-note{font-size:9px;line-height:1.7;color:#97a18e}.text-button{background:none;border:0;padding:5px 0;font-size:10px;color:#557847;cursor:pointer;text-decoration:underline;text-underline-offset:3px}.evidence{display:flex;justify-content:space-between;gap:30px;margin:42px 0;padding-bottom:32px;border-bottom:1px solid #dce3d3}.evidence .eyebrow{margin-bottom:12px}.evidence p:not(.eyebrow){font-size:11px;line-height:1.7;color:#87937e;max-width:340px}.metrics{display:flex;gap:36px;align-items:center}.metric strong{display:block;font:28px Georgia;color:#325b36;margin-bottom:8px}.metric span{font-size:10px;color:#87937e;display:block}.method{margin-bottom:40px}.method summary{font-size:12px;cursor:pointer;display:flex;justify-content:space-between;list-style:none}.method-body{display:grid;grid-template-columns:repeat(3,1fr);gap:28px;font-size:11px;line-height:1.9;color:#75826d;padding-top:15px}footer{border-top:1px solid #dce3d3;padding:20px 0 30px;display:flex;justify-content:space-between;color:#95a08c;font-size:9px}button:focus-visible,a:focus-visible,label:focus-within,summary:focus-visible{outline:3px solid #bc784f;outline-offset:3px}[hidden]{display:none!important}@media(max-width:850px){.hero-fact{display:none}h1{font-size:50px}.workspace{grid-template-columns:1fr}.input-section{border-right:0;border-bottom:1px solid #dfe4d9}.evidence{flex-direction:column}.metrics{justify-content:space-between}.method-body{grid-template-columns:1fr;gap:0}.header-right{font-size:10px}.course{display:none}.hero{padding-top:38px}footer{gap:20px}.result-section{min-height:450px}}@media(prefers-reduced-motion:reduce){.spinner{animation:none}}
```

templates/images.html

```
1  <!doctype html>
2  <html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,
   initial-scale=1"><title>CricketShot | Image recognition</title><link rel="stylesheet"
   href="/static/style.css"><link rel="stylesheet" href="/static/images.css"><script src="/static/images.js"
   defer></script></head>
```

templates/images.html

```
<body><header><a class="wordmark" href="/">Cricket<span>Shot</span><span class="wordmark-dot"></span></a><div class="header-right"><a href="/video">Video pilot</a><span class="local-dot"></span> Local inference <span class="course">EE604 / 2026</span></div></header>
<main><div class="hero"><div><p class="eyebrow">CRICKET SHOT RECOGNITION</p><h1>Read the shape<br>of a <em>shot.</em></h1><p class="intro">Two views of one image. A lightweight classifier compares the full view with a central crop, then shows its prediction and uncertainty.</p></div><div class="hero-fact"><div class="fact-number">04<span>shot classes</span></div><p>Drive. Leg glance / flick. Pull. Sweep.</p><p class="muted">Real images. Measured results.<br>Every prediction stays on your computer.</p></div></div>
<div class="workspace"><section class="input-section"><div class="section-title"><span class="step">01</span><h2>Choose an image</h2></div><form id="image-form"><label class="dropzone" id="dropzone" for="image-input"><span class="upload-symbol">+</span><strong id="file-label">Drop a cricket image here</strong><span>or click to browse</span><small>JPEG, PNG, WebP, BMP / up to 24 megapixels</small></label><input id="image-input" type="file" accept="image/jpeg,image/png,image/webp,image/bmp" hidden><button type="submit" id="analyze-button" class="primary-button" disabled>Recognize shot <span aria-hidden="true"> </span></button></form><p class="input-note">Use one visible batter playing a shot. The model supports four dataset labels and can be wrong on unfamiliar images.</p><div class="example-heading"><h3>Explore held-out examples</h3><span>Includes a failure case</span></div><div id="examples" class="examples"></div></section>
<section class="result-section" aria-live="polite"><div class="section-title"><span class="step">02</span><h2>Inspect the result</h2></div><div id="empty-state" class="empty-state"><div class="empty-nine">2</div><h3>One image, two perspectives</h3><p>See class scores, the two input views<br>and whether the model requests review.</p></div><div id="loading-state" class="loading-state" hidden><div class="spinner"></div><h3>Comparing the views</h3><p>Extracting visual features on CPU...</p></div><div id="error-state" class="error-state" role="alert" hidden></div><div id="prediction" hidden><div class="prediction-heading"><div><p id="decision" class="eyebrow"></p><h3 id="shot-label"></h3></div><div class="score"><strong id="confidence"></strong><span>model score</span></div></div><p id="prediction-context" class="prediction-context"></p><p id="review-note" class="review-note"></p><div id="scores" class="scores"></div><div class="frames-heading"><h4>Actual model inputs</h4><span id="timing"></span></div><div id="image-views" class="image-views"></div><p id="view-agreement" class="frame-note"></p><p id="hog-result" class="frame-note"></p><button class="text-button" id="download-result">Download prediction JSON ↓</button></div></section></div>
<section class="evidence"><div><p class="eyebrow">MEASURED, WITH LIMITS</p><h2>Evidence behind the prediction</h2><p>Exact duplicates collapse before splitting. Audited visual groups stay together. The 945 test images belong to 70 groups; match and player identities are unavailable.</p></div><div id="metrics" class="metrics"></div></section><details class="method"><summary>Method, scope and limitations <span>+</span></summary><div class="method-body"><p>A frozen MobileNetV2 extracts two feature vectors. The classifier combines them with their absolute difference. Only the linear cricket classifiers are trained. The fixed crop describes a region; it does not detect or segment the batter. A HOG baseline compares grayscale gradients on the same image split. The contrast feature is a modest course-level extension of established methods.</p><p>The review rule requires a high enough model score and agreement between the two single-view heads. It is a secondary diagnostic. Test calibration worsened after temperature scaling (ECE 0.043 to 0.094). A model score is not a guarantee of correctness or an unknown-class detector.</p><p>The dataset has no verified match IDs. Closely related frames may remain across splits despite visual grouping. The separate eight-class video pilot performs much worse and is available through the link above.</p></div></details><footer><span>CricketShot / EE604 course project<br><a href="{{ submission.github_url }}">GitHub repository</a><br>{% for member in submission.members %}{{ member.name }} ({{ member.roll_number }}){% if not loop.last %} / {% endif %}{% endfor %}</span><span>Local inference. No cloud upload. Predictions need judgment.</span></footer></main></body></html>
```

templates/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1">
  <title>CricketShot | EE604</title>
  <link rel="stylesheet" href="/static/style.css">
  <script src="/static/app.js" defer></script>
</head>
<body>
<header><a class="wordmark" href="/">Cricket<span>Shot</span><span class="wordmark-dot"></span></a><div class="header-right"><span class="local-dot"></span> <a href="/">Main image project</a> <span class="course">EE604 / 2026</span></div></header>
```

templates/index.html

```
<main>
  <div class="hero"><div><p class="eyebrow">EXPLORATORY VIDEO BACKUP</p><h1>Every shot has<br>a <em>sequence.</em></h1><p class="intro">This pilot scores 30.2% accuracy, equal to the majority baseline, and 21.3% macro F1. It is retained as an exploratory result. The main project classifies one image into four labels.</p></div><div class="hero-fact"><div class="fact-number">09<span>frames</span></div><p>Preparation. Contact. Follow-through.</p><p class="muted">Three temporal phases, one prediction.<br>Eight classes from real broadcast clips.</p></div></div>
  <div class="workspace">
    <section class="input-section" aria-labelledby="input-title"><div class="section-title"><span class="step">01</span><h2 id="input-title">Choose a clip</h2></div>
      <form id="upload-form"><label class="dropzone" id="dropzone" for="video-input"><span class="upload-symbol">+</span><strong id="file-label">Drop your video here</strong><span>or click to browse</span><small>AVI, MP4, MOV, MKV, WebM / up to 30 s / 40 MB</small></label><input id="video-input" type="file" accept="video/*,.avi,.mkv" hidden><button type="submit" id="analyze-button" class="primary-button" disabled>Analyze clip <span aria-hidden="true"> </span></button></form>
      <p class="input-note">Use one trimmed shot in a typical end-on broadcast view, with the batter near the upper center. The model uses a fixed crop.</p>
      <div class="example-heading"><h3>Try the test set</h3><span>Ground-truth labels included</span></div><div id="examples" class="examples"></div>
    </section>
    <section class="result-section" aria-labelledby="result-title" aria-live="polite"><div class="section-title"><span class="step">02</span><h2 id="result-title">Read the prediction</h2></div>
      <div id="empty-state" class="empty-state"><div class="empty-nine">9</div><h3>A shot unfolds over time</h3><p>Your prediction, class scores and sampled<br>frames will appear here.</p></div>
      <div id="loading-state" class="loading-state" hidden><div class="spinner"></div><h3>Reading the sequence</h3><p>Sampling frames and comparing temporal phases...</p></div>
      <div id="error-state" class="error-state" role="alert" hidden></div>
      <div id="prediction" hidden><div class="prediction-heading"><div><p id="decision" class="eyebrow"></p><h3 id="shot-label"></h3></div><div class="score"><strong id="confidence"></strong><span>model score</span></div></div><p id="prediction-context" class="prediction-context"></p><p id="review-note" class="review-note"></p><div id="scores" class="scores"></div><div class="frames-heading"><h4>What the model saw</h4><span id="timing"></span></div><div id="phase-strip" class="phase-strip"></div><p class="frame-note">Uniform samples divided into early, middle and late phases. These are temporal bins, not detected contact events.</p><button class="text-button" id="download-result">Download prediction JSON ↓</button></div>
    </section>
  </div>
  <section class="evidence"><div><p class="eyebrow">MEASURED, WITH LIMITS</p><h2>Evidence behind the demo</h2><p>Clips from a source group stay in one split. Test scores below come from saved predictions.</p></div><div id="metrics" class="metrics"></div></section>
  <details class="method"><summary>How it works, and where it can fail <span>+</span></summary><div class="method-body"><p>A fixed crop emphasizes the upper center of each broadcast frame. A frozen MobileNetV2 encoder and a small classifier use ordered phase averages, late-minus-early differences and temporal variation. The strip shows these crops.</p><p>A validation-fitted temperature adjusts confidence. Low scores request review. This rule cannot reliably detect non-cricket footage or unknown shot classes, and a confident prediction can still be wrong.</p><p>Eight labels come from a small, imbalanced archive. The crop revision followed a failed full-frame pilot, so the reported scores are exploratory results on a reused holdout. Different camera layouts may place the batter outside the crop.</p></div></details>
  <footer><span>CricketShot / EE604 course project<br><a href="{{ submission.github_url }}">GitHub repository</a><br>{% for member in submission.members %}{{ member.name }} ({{ member.roll_number }}){% if not loop.last %} / {% endif %}{% endfor %}</span><span>Local inference. No cloud upload. No claim of coaching accuracy.</span></footer>
</main></body></html>
```

tests/test_images.py

```
from pathlib import Path
import csv
import io
import json
import sys
import numpy as np
import pytest
```

tests/test_images.py

```
from PIL import Image
ROOT=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(ROOT))
from cricketshot.images import ImageRecognizer,image_descriptor,image_views,load_image
from app import create_app

def test_contrast_descriptor_dimensions():
    x=np.random.default_rng(4).normal(size=(3,2,1280)).astype('float32')
    assert image_descriptor(x,'contrast').shape==(3,3840)
    assert image_descriptor(x,'concat').shape==(3,2560)
    assert np.allclose(np.linalg.norm(image_descriptor(x,'contrast'),axis=1),1)

def test_image_group_and_hash_disjointness():
    rows=list(csv.DictReader((ROOT/'data/images/processed/manifest.csv').open(encoding='utf-8',newline='')))
    for key in ('group','rgb_sha256'):
        sets={s:{r[key] for r in rows if r['split']==s} for s in ('train','val','test')}
        assert not sets['train']&sets['test'] and not sets['train']&sets['val'] and not sets['val']&sets['test']
    assert len(rows)==len({r['rgb_sha256'] for r in rows})

def test_image_export_and_metrics_parity():
    model=ImageRecognizer();r=json.loads((ROOT/'results/images/metrics.json').read_text())
    with np.load(ROOT/'data/images/processed/features.npz',allow_pickle=False) as a:
        test=a['features'][a['split']=='test']
    with np.load(ROOT/'results/images/test_predictions.npz',allow_pickle=False) as a:
        for mode in ('global','focus','concat','contrast'):
            p=model.predict_features(test,mode)
            assert np.allclose(p,a['p_'+mode],atol=1e-7)
            assert np.allclose(p.sum(1),1)
            assert abs(np.mean(p.argmax(1)==a['y'])-r['models'][mode]['test']['accuracy'])<1e-12
        keep=(a['p_contrast'].max(1)>=model.config['review_threshold'])&a['agreement']
        assert np.array_equal(keep,a['accepted'])

def test_fresh_image_inference_matches_cached():
    rows=list(csv.DictReader((ROOT/'data/images/processed/manifest.csv').open(encoding='utf-8',newline='')))
    row=next(r for r in rows if r['split']=='test')
    result,_=ImageRecognizer().predict(ROOT/row['path'])
    with np.load(ROOT/'results/images/test_predictions.npz',allow_pickle=False) as a:
        i=list(a['ids']).index(row['id']);assert
np.allclose(list(result['probabilities'].values()),a['p_contrast'][i],atol=1e-6)

def test_invalid_images(tmp_path):
    p=tmp_path/'bad.jpg';p.write_bytes(b'not image')
    with pytest.raises(ValueError):load_image(p)
    p=tmp_path/'tiny.png';Image.new('RGB',(3,3)).save(p)
    with pytest.raises(ValueError):load_image(p)

def test_image_api_upload_and_examples():
    c=create_app().test_client();assert c.get('/').status_code==200
    assert c.post('/image-api/predict').status_code==400
    assert c.post('/image-api/predict',data={'image':(io.BytesIO(b'bad'),'bad.jpg')}).status_code==400
    assert c.post('/image-api/predict',data={'image':(io.BytesIO(b'bad'),'bad.svg')}).status_code==400
    e=c.get('/image-api/examples').json;assert len(e)>=3
    assert all(x['split']=='test' and 'path' not in x for x in e)
    p=c.post('/image-api/example/'+e[0]['id']);assert p.status_code==200 and len(p.json['previews'])==2
    assert c.get('/image-api/example/not-real/thumbnail').status_code==404
    # Exercise the real upload endpoint using an actual test example, not a mock.
    sample=json.loads((ROOT/'demo/image_examples.json').read_text())[0]
    with (ROOT/sample['path']).open('rb') as f:
        upload=c.post('/image-api/predict',data={'image':(io.BytesIO(f.read()),'sample.jpg')})
    assert upload.status_code==200
    assert abs(sum(upload.json['probabilities'].values())-1)<1e-8
```

tests/test_project.py

tests/test_project.py

```
"""Tests target leakage, inference parity, malformed inputs and measured evidence."""
from pathlib import Path
import csv
import io
import json
import sys
import numpy as np
import pytest
ROOT=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(ROOT))
from cricketshot.core import descriptor, softmax, Recognizer, decode_video
from app import create_app

def test_probabilities_stable_and_normalized():
    p=softmax(np.array([[10000,10001,-10000],[0,0,0]]))
    assert np.isfinite(p).all()
    assert np.allclose(p.sum(1),1)
    assert p[0].argmax()==1
    with pytest.raises(ValueError):softmax([1,2],0)

def test_temporal_descriptor_obeys_order():
    x=np.random.default_rng(4).normal(size=(2,9,1280)).astype('float32')
    assert descriptor(x,'phase_delta').shape==(2,6400)
    assert np.allclose(descriptor(x,'mean'),descriptor(x[:,::-1],'mean'),atol=1e-6)
    assert not np.allclose(descriptor(x,'phase_delta'),descriptor(x[:,::-1],'phase_delta'))

@pytest.fixture(scope='module')
def evidence():
    path=ROOT/'data/processed/manifest.csv'
    if not path.exists():pytest.skip('Dataset has not been prepared')
    return list(csv.DictReader(path.open(encoding='utf-8',newline='')))

def test_split_has_no_shared_sources_or_hashes(evidence):
    for key in ('source','group','sha256'):
        sets={s:{r[key] for r in evidence if r['split']==s} for s in ('train','val','test')}
        assert not sets['train'] & sets['test']
        assert not sets['train'] & sets['val']
        assert not sets['val'] & sets['test']
    classes={r['label'] for r in evidence}
    assert len(classes)==8
    for s in ('train','val','test'):
        assert {r['label'] for r in evidence if r['split']==s}==classes

def test_saved_metrics_recompute_from_predictions():
    r=json.loads((ROOT/'results/metrics.json').read_text())
    with np.load(ROOT/'results/test_predictions.npz',allow_pickle=False) as a:
        for mode,info in r['models'].items():
            p=a['p_'+mode]
            assert np.isfinite(p).all() and np.allclose(p.sum(1),1)
            assert abs(np.mean(p.argmax(1)==a['y'])-info['test']['accuracy'])<1e-12
            assert np.sum(info['test']['confusion'])==len(a['y'])
        assert len(np.unique(a['ids']))==len(a['ids'])

def test_numeric_export_matches_training_predictions(evidence):
    model=Recognizer()
    with np.load(ROOT/'data/processed/features.npz',allow_pickle=False) as a:
        pp=model.predict_features(a['features'][a['split']=='test'])
    with np.load(ROOT/'results/test_predictions.npz',allow_pickle=False) as a:
        assert np.allclose(pp,a['p_phase_delta'],atol=1e-7)

def test_live_decode_and_feature_parity(evidence):
```

tests/test_project.py

```
    row=next(r for r in evidence if r['split']=='test')
    result,frames=Recognizer().predict(ROOT/row['path'])
    with np.load(ROOT/'results/test_predictions.npz',allow_pickle=False) as a:
        i=list(a['ids']).index(row['id']);expected=a['p_phase_delta'][i]
    assert len(frames)==9
    assert np.allclose(list(result['probabilities'].values()),expected,atol=1e-6)

def test_invalid_video_rejected(tmp_path):
    bad=tmp_path/'bad.avi';bad.write_bytes(b'not a video')
    with pytest.raises(ValueError):decode_video(bad)

def test_local_api_errors_and_templates():
    client=create_app().test_client()
    assert client.get('/').status_code==200
    assert client.get('/api/status').json['ready']
    assert client.post('/api/predict').status_code==400
    bad=client.post('/api/predict',data={'video':(io.BytesIO(b'wrong'),'bad.exe')})
    assert bad.status_code==400
    invalid=client.post('/api/predict',data={'video':(io.BytesIO(b'wrong'),'bad.mp4')})
    assert invalid.status_code==400
    assert client.post('/api/example/../../app.py').status_code==404
    assert client.post('/api/example/not-real').status_code==404

def test_example_api_runs_real_inference():
    client=create_app().test_client()
    examples=client.get('/api/examples').json
    assert examples
    for e in examples:
        assert e['split']=='test' and 'path' not in e
    response=client.post('/api/example/'+examples[0]['id'])
    assert response.status_code==200
    data=response.json
    assert len(data['previews'])==9
    assert abs(sum(data['probabilities'].values())-1)<1e-8
    assert data['accepted']==(data['confidence']>=data['threshold'])
```

tests/test_revision.py

```
"""Checks for HOG vote arithmetic, cache binding, export and API parity."""
from pathlib import Path
import csv
import io
import json
import hashlib
import numpy as np
from PIL import Image
from cricketshot.hog import (PARAMETERS, HOGRecognizer, cell_histograms,
                             gradients, hog_descriptor)
from cricketshot.images import load_image
ROOT = Path(__file__).resolve().parents[1]


def test_hog_votes_are_magnitude_weighted_and_interpolated():
    magnitude = np.array([[90.0]])
    angle = np.array([[50.0]])
    cells = cell_histograms(magnitude, angle, cell_size=1)
    assert np.allclose(cells.sum(), 90)
    # Nine unsigned bins have centers at 0,20,40,...,160 degrees.
    assert cells[0, 0, 2] == 45 and cells[0, 0, 3] == 45
    wrap = cell_histograms(np.array([[20.0]]), np.array([[170.0]]), cell_size=1)
    assert wrap[0, 0, 8] == 10 and wrap[0, 0, 0] == 10


```

tests/test_revision.py

```
def test_hog_geometry_flat_image_and_unsigned_edges():
    flat = Image.new("RGB", (320, 240), (40, 40, 40))
    x = hog_descriptor(flat)
    assert x.shape == (6084,) and np.isfinite(x).all() and np.count_nonzero(x) == 0
    ramp = np.tile(np.linspace(0, 1, 224), (224, 1)).astype(np.float32)
    m, a = gradients(ramp)
    assert np.allclose(a[:, 1:-1], 0) and np.all(m[:, 1:-1] > 0)
    mr, ar = gradients(1-ramp)
    assert np.allclose(mr, m, atol=1e-6) and np.allclose(ar[:, 1:-1], 0)


def test_hog_saved_features_and_export_match_fresh_inputs():
    meta = json.loads((ROOT / "data/images/processed/hog_features.json").read_text())
    assert meta["parameters"] == PARAMETERS
    for key, path in [("manifest_sha256", "data/images/processed/manifest.csv"),
                      ("implementation_sha256", "src/cricketshot/hog.py"),
                      ("protocol_sha256", "configs/hog_experiment.json")]:
        assert meta[key] == hashlib.sha256((ROOT/path).read_bytes()).hexdigest()
    rows = list(csv.DictReader((ROOT / "data/images/processed/manifest.csv").open(encoding="utf-8",
newline="")))
    model = HOGRecognizer()
    with np.load(ROOT / "data/images/processed/hog_features.npz", allow_pickle=False) as cache:
        features = cache["features"]
        assert cache["ids"].tolist() == [r["id"] for r in rows]
        for index in np.linspace(0, len(rows)-1, 8, dtype=int):
            fresh = hog_descriptor(load_image(ROOT / rows[index]["path"]))
            assert np.allclose(fresh, features[index], atol=1e-7)
        test = np.array([r["split"] == "test" for r in rows])
        with np.load(ROOT / "results/hog/test_predictions.npz", allow_pickle=False) as predictions:
            p = model.predict_features(features[test])
            assert np.allclose(p, predictions["p"], atol=1e-7)
            assert np.allclose(p.sum(1), 1)


def test_uploaded_image_compares_both_real_models():
    from app import create_app
    client = create_app().test_client()
    example = json.loads((ROOT / "demo/image_examples.json").read_text())[2]
    payload = (ROOT / example["path"]).read_bytes()
    response = client.post("/image-api/predict", data={"image": (io.BytesIO(payload), "failure.png")})
    assert response.status_code == 200
    result = response.json
    assert "hog_baseline" in result and result["score_type"] == "temperature-scaled softmax"
    expected = HOGRecognizer().predict(ROOT / example["path"])
    assert result["hog_baseline"]["label"] == expected["label"]
    assert np.allclose(list(result["hog_baseline"]["probabilities"].values()),
list(expected["probabilities"].values()))


def test_missing_optional_examples_are_hidden(monkeypatch):
    import app as demo_app
    absent = "demo/images/not_distributed.png"
    original = Path.is_file
    monkeypatch.setattr(Path, "is_file", lambda p: False if p.as_posix().endswith(absent) else original(p))
    original_read = Path.read_text
    def read(p, *args, **kwargs):
        if p.name == "image_examples.json":
            return json.dumps([{"id": "absent", "path": absent}])
        return original_read(p, *args, **kwargs)
    monkeypatch.setattr(Path, "read_text", read)
    client = demo_app.create_app().test_client()
    assert client.get("/image-api/examples").json == []
    assert client.post("/image-api/example/absent").status_code == 404
```